ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA KHOA HỌC VÀ KĨ THUẬT MÁY TÍNH



BÁO CÁO

ĐỒ ÁN CHUYÊN NGÀNH

Hệ thống quản lý các hoạt động Đoàn - Hội và hỏi đáp hoạt động ngoại khóa Trường Đại học Bách khoa

NGÀNH: KHOA HỌC MÁY TÍNH

HỘI ĐỒNG: Hội đồng 4L Đồ án chuyên ngành

GVHD: ThS. Trần Ngọc Bảo Duy

TKHĐ: CN. Lê Bình Đẳng

—o0o—

SVTH1: Nguyễn Phương Duy (2210526)

SVTH2: Nguyễn Ngọc Quý (2212870)

SVTH3: Trần Quốc Toàn (2213540)

TP. HỒ CHÍ MINH, 12/2025

Lời cảm ơn

Đầu tiên, chúng em xin bày tỏ lòng biết ơn sâu sắc và chân thành cảm ơn **ThS. Trần Ngọc Bảo Duy** – Giảng viên Bộ môn Khoa học Máy tính, người Thầy đã trực tiếp hướng dẫn, định hướng đề tài và tận tình chỉ bảo, giúp đỡ chúng em tháo gỡ những khó khăn trong suốt thời gian thực hiện và hoàn thành đồ án *"Hệ thống quản lý các hoạt động Đoàn - Hội và hỏi đáp hoạt động ngoại khóa Trường Đại học Bách khoa"*.

Chúng em xin chân thành cảm ơn **Trường Đại học Bách khoa (ĐHQG-HCM)** và Quý Thầy Cô thuộc **Khoa Khoa học và Kỹ thuật Máy tính** đã tận tình truyền đạt những kiến thức nền tảng quý báu, tạo điều kiện thuận lợi và môi trường học tập tốt nhất để chúng em có đủ hành trang thực hiện đồ án này.

Cuối cùng, chúng em xin cảm ơn gia đình, bạn bè và những người thân đã luôn động viên, ủng hộ, chia sẻ và là chỗ dựa tinh thần vững chắc giúp chúng em tập trung nghiên cứu và hoàn thành báo cáo đúng hạn.

Mặc dù đã nỗ lực hết mình, nhưng do kiến thức và thời gian có hạn, đồ án khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp quý báu của Thầy để đồ án ngày càng hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

Lời cam đoan

Chúng em xin cam đoan đồ án này là do chính nhóm tự thực hiện dưới sự hướng dẫn của ThS. Trần Ngọc Bảo Duy và không sao chép các công trình nghiên cứu của người khác để làm sản phẩm của riêng mình.

Các thông tin thứ cấp và tài liệu tham khảo sử dụng trong đồ án đều có nguồn gốc rõ ràng và được trích dẫn đầy đủ theo quy định.

Chúng em xin hoàn toàn chịu trách nhiệm về tính xác thực và nguyên bản của đồ án này trước Hội đồng đánh giá và Nhà trường.

TP. Hồ Chí Minh, tháng 12 năm 2025

Nhóm sinh viên thực hiện

Nguyễn Phương Duy
Nguyễn Ngọc Quý
Trần Quốc Toàn

Tóm tắt

Nội dung của báo cáo được tổ chức thành 6 chương chính, trình bày chi tiết quy trình xây dựng hệ thống từ giai đoạn khảo sát, phân tích đến thiết kế và định hướng phát triển, cụ thể như sau:

Chương 1: Giới thiệu

Chương này trình bày tổng quan về bối cảnh thực hiện đề tài tại Trường Đại học Bách khoa (ĐHQG-HCM), nêu rõ các vấn đề bất cập trong công tác quản lý hoạt động ngoại khóa hiện tại. Từ đó, xác định mục tiêu, phạm vi nghiên cứu và đề xuất hướng giải quyết thông qua việc xây dựng hệ thống phần mềm quản lý tập trung.

Chương 2: Cơ sở lý thuyết và công nghệ

Tập trung giới thiệu các nền tảng lý thuyết và công nghệ chủ chốt được áp dụng trong đồ án. Nội dung bao gồm các quy trình nghiệp vụ Đoàn - Hội và bộ công nghệ (Tech Stack) được lựa chọn để phát triển ứng dụng Web cũng như công nghệ AI tiên tiến (RAG) phục vụ cho tính năng Chatbot.

Chương 3: Yêu cầu về hệ thống

Phân tích chi tiết các yêu cầu đối với hệ thống phần mềm, bao gồm yêu cầu chức năng (các nghiệp vụ quản lý thành viên, hoạt động, tài chính, biểu mẫu...) và yêu cầu phi chức năng (bảo mật, hiệu năng, trải nghiệm người dùng), làm cơ sở cho các bước thiết kế tiếp theo.

Chương 4: Mô hình hóa hệ thống

Chương này mô tả hệ thống dưới góc độ kỹ thuật thông qua ngôn ngữ mô hình hóa UML. Các biểu đồ Use Case, Activity và Sequence được xây dựng nhằm làm rõ các tác nhân, quy trình nghiệp vụ và luồng tương tác dữ liệu bên trong hệ thống.

Chương 5: Phân tích và Thiết kế hệ thống

Đi sâu vào thiết kế kiến trúc tổng thể và cấu trúc dữ liệu (ERD) của hệ thống. Đặc biệt, chương này trình bày chi tiết về kiến trúc và cơ chế hoạt động của Chatbot AI sử dụng kỹ thuật RAG (Retrieval-Augmented Generation), từ khâu xử lý dữ liệu đầu vào (Vector Embedding) đến quy trình truy vấn và phản hồi thông tin tự động.

Chương 6: Tổng kết

Đánh giá lại những kết quả đạt được của đồ án so với mục tiêu ban đầu, nhìn nhận các ưu điểm và hạn chế còn tồn tại. Đồng thời, chương này cũng đề xuất lộ trình phát triển, nâng cấp và bảo trì hệ thống trong tương lai.

Contents

Tóm tắt **2**

1 Giới thiệu **1**
1.1 Đặt vấn đề . . . 1
1.2 Các hướng giải quyết liên quan . . . 1

2 Cơ sở lý thuyết và công nghệ **3**
2.1 Cơ sở lý thuyết . . . 3
2.2 Công nghệ . . . 3

3 Yêu cầu về hệ thống **5**
3.1 Yêu cầu chức năng . . . 5
3.1.1 Quản lý Người dùng và Tổ chức . . . 5
3.1.2 Quản lý Hoạt động CTXH . . . 5
3.1.3 Quản lý Đóng Phí . . . 6
3.1.4 Quản lý Biểu mẫu và Chứng từ . . . 7
3.1.5 Chatbot hỏi đáp hoạt động công tác xã hội . . . 7
3.2 Yêu cầu phi chức năng . . . 8
3.2.1 Bảo mật và An toàn thông tin . . . 8
3.2.2 Hiệu năng . . . 8
3.2.3 Khả năng triển khai và Mở rộng . . . 9
3.2.4 Giám sát và Logging . . . 9
3.2.5 Giao diện và Trải nghiệm người dùng . . . 10
3.2.6 Code Quality và Maintainability . . . 10

4 Mô hình hóa hệ thống **11**
4.1 Mô hình hóa Use Case . . . 11
4.1.1 Quản lý thành viên . . . 11
4.1.2 Quản lý đơn vị . . . 17
4.1.3 Quản lý lớp học . . . 19
4.1.4 Quản lý tổ chức . . . 21
4.1.5 Quản lý phân quyền (RBAC) . . . 23
4.1.6 Quản lý đóng phí . . . 25
4.1.7 Quản lý hoạt động công tác xã hội . . . 27
4.1.8 Quản lý biểu mẫu . . . 39
4.1.9 Hỏi đáp thông tin công tác xã hội qua Chatbot . . . 45
4.2 Mô hình hóa Activity Diagram . . . 47
4.2.1 Activity Diagram - Quản lý tổ chức . . . 47
4.2.2 Activity Diagram - Quản lý phân quyền . . . 49
4.2.3 Activity Diagram - Module Công tác Xã hội . . . 50
4.2.4 Activity Diagram - Module Biểu mẫu . . . 53
4.2.5 Activity Diagram - Chức năng tạo đợt đóng phí . . . 56
4.2.6 Activity Diagram - Chức năng thêm dữ liệu từ file Excel . . . 57
4.3 Mô hình hóa Sequence Diagram . . . 59
4.3.1 Sequence Diagram - Module Quản lý tổ chức . . . 59
4.3.2 Sequence Diagram - Module Phân quyền . . . 60
4.3.3 Sequence Diagram - Module Công tác Xã hội . . . 62

4.3.4 Sequence Diagram - Module Biểu mẫu . . . 65
4.3.5 Sequence Diagram - Chatbot hỏi-đáp hoạt động CTXH . . . 67

5 Phân tích & Thiết kế hệ thống 69
5.1 Thiết kế dữ liệu . . . 69
5.1.1 ERD tổng thể . . . 69
5.1.2 Dữ liệu phục vụ Chatbot RAG . . . 73
5.2 Thiết kế kiến trúc hệ thống . . . 74
5.3 Thiết kế Chatbot RAG . . . 76
5.3.1 Tổng quan Chatbot . . . 76
5.3.2 Tổng quan giải pháp Chatbot RAG . . . 76
5.3.3 Quy trình tự động chuyển đổi dữ liệu sang vector embedding . . . 77
5.3.4 Quy trình xử lý câu hỏi của người dùng với AI . . . 77
5.3.5 Kết quả đạt được . . . 79
5.3.6 Giới hạn của Chatbot . . . 80
5.3.7 Kết luận . . . 80
5.3.8 Định hướng phát triển Chatbot . . . 80

6 Tổng kết 81
6.1 Nhận xét . . . 81
6.1.1 Ưu điểm . . . 81
6.1.2 Hạn chế . . . 82
6.2 Kế hoạch phát triển . . . 82
6.2.1 Hoàn thiện và tối ưu các tính năng hiện có (Tuần 1-6) . . . 82
6.2.2 Triển khai Testing toàn diện (Tuần 7-10) . . . 83
6.2.3 Cải thiện DevOps và Deployment (Tuần 11-14) . . . 83
6.2.4 Documentation và Knowledge Transfer (Tuần 15-16) . . . 84
6.2.5 Các mục tiêu đo lường được . . . 84

7 Tài liệu tham khảo 84

List of Figures

1 UC01 - Quản lý thành viên . . . 11
2 UC02 - Quản lý đơn vị . . . 17
3 UC03 - Quản lý lớp học . . . 19
4 UC04 - Quản lý tổ chức . . . 21
5 UC05 - Gán vai trò và Phân quyền . . . 23
6 UC04 - Quản lý đóng phí . . . 25
7 UC07 - Quản lý hoạt động Công tác Xã hội (bao gồm UC7.4 - Cấu hình giới hạn, UC7.5 - Xem báo cáo cá nhân, UC7.6 - Nhập dữ liệu hệ thống cũ) . . . 28
8 UC7.1 - Quản lý danh mục công tác xã hội . . . 29
9 UC7.2 - Quản lý người tham gia hoạt động . . . 32
10 UC7.3 - Quản lý hoạt động công tác xã hội . . . 35
11 UC8.1 - Quản lý biểu mẫu . . . 40
12 UC8.2 - Điền thông tin vào biểu mẫu . . . 43
13 UC09 - Tra cứu hoạt động Công tác xã hội qua Chatbot . . . 45
14 Activity Diagram - Quản lý tổ chức . . . 47
15 Activity Diagram - Quản lý phân quyền . . . 49
16 Activity Diagram - Tạo và chỉnh sửa hoạt động . . . 51
17 Activity Diagram - Chấm điểm sinh viên từ file Excel . . . 52
18 Activity Diagram - Tạo cấu hình giới hạn ngày CTXH . . . 53
19 Activity Diagram - Tạo và cấu hình biểu mẫu . . . 54
20 Activity Diagram - Điền thông tin vào biểu mẫu . . . 55
21 Activity Diagram - Quản lý đóng phí . . . 56
22 Activity Diagram - Thêm nhiều thành viên từ file Excel . . . 57
23 Sequence Diagram - Luồng hoạt động của quản lý tổ chức . . . 59
24 Sequence Diagram - Luồng hoạt động cơ chế phân quyền . . . 61
25 Sequence Diagram - Tạo và chỉnh sửa hoạt động CTXH . . . 63
26 Sequence Diagram - Chấm điểm sinh viên từ file Excel . . . 64
27 Sequence Diagram - Tạo và cấu hình biểu mẫu . . . 66
28 Sequence Diagram - Pipeline tự động chuyển đổi dữ liệu CTXH sang dạng vector embedding) . . . 67
29 Sequence Diagram - Luồng hoạt động của Chatbot RAG) . . . 68
30 Entity-Relationship Model cho toàn bộ hệ thống . . . 72
31 Relational Data Model của toàn bộ hệ thống . . . 72
32 Component Diagram- Kiến trúc hệ thống . . . 74

List of Tables

1 Bảng mô tả Use Case UC01 – Quản lý thành viên 12
2 Bảng mô tả Use Case UC01.1 - Thêm một thành viên từ Form 13
3 Bảng mô tả Use Case UC01.2 - Thêm nhiều thành viên từ file Excel 14
4 Bảng mô tả Use Case UC01.3 - Gán vai trò cho thành viên 16
5 Bảng mô tả Use Case UC02 - Quản lý đơn vị 18
6 Bảng mô tả Use Case UC03 - Quản lý lớp học 20
7 Bảng mô tả Use Case UC04 - Quản lý tổ chức 22
8 Bảng mô tả Use Case UC05 - Gán vai trò và Phân quyền 24
9 Bảng mô tả Use Case UC04 - Quản lý đóng phí 27
10 Bảng mô tả Use Case UC7.1 - Quản lý danh mục công tác xã hội 31
11 Bảng mô tả Use Case UC7.2 - Quản lý người tham gia hoạt động 34
12 Bảng mô tả Use Case UC7.3 - Quản lý hoạt động công tác xã hội 36
13 Bảng mô tả Use Case UC7.4 - Quản lý hạn mức ngày CTXH theo chức 37
14 Bảng mô tả Use Case UC7.5 - Xem báo cáo ngày công tác xã hội cá nhân 38
15 Bảng mô tả Use Case UC7.6 - Nhập dữ liệu ngày CTXH từ hệ thống cũ 39
16 Bảng mô tả Use Case UC8.1 - Quản lý biểu mẫu 42
17 Bảng mô tả Use Case UC8.2 - Điền thông tin vào biểu mẫu 45
18 Bảng mô tả Use Case UC09 - Tra cứu hoạt động Công tác xã hội qua Chatbot 46
19 Cấu trúc dữ liệu CtxhActivity 73
20 So sánh giữa LLM thuần túy và RAG 77

1 Giới thiệu

1.1 Đặt vấn đề

Trong bối cảnh giáo dục đại học tại Việt Nam, Trường Đại học Bách Khoa – Đại học Quốc gia TP.HCM (ĐH Bách Khoa) là một trong những cơ sở đào tạo hàng đầu, với quy mô sinh viên lên đến hơn 27.000 người. Đây không chỉ là con số lớn nhất tại TP.HCM mà còn phản ánh vị thế dẫn đầu về số lượng đoàn viên và hội viên tham gia các hoạt động của Đoàn Thanh niên và Hội Sinh viên.

Tuy nhiên, quy mô khổng lồ này lại đối mặt với một nghịch lý lớn: công tác quản lý hiện tại chủ yếu dựa vào các phương pháp thủ công và rời rạc, nguyên nhân là do đoàn hội hiện nay vẫn chưa có hệ thống quản lý thông tin tập trung.

Cụ thể, toàn bộ dữ liệu liên quan đến người dùng (đoàn viên, hội viên), hoạt động và tài chính không được lưu trữ trên một máy chủ thống nhất mà phân tán trong tài liệu giấy được lưu trong kho và các tệp Word, Excel trên máy tính cá nhân của cán bộ. Việc tra cứu, tổng hợp báo cáo hay liên kết dữ liệu đòi hỏi nhiều thao tác thủ công, gây tốn kém thời gian và nhân lực. Sinh viên phải "bơi" trong biển thông tin trên mạng xã hội để tìm kiếm hoạt động phù hợp, trong khi cán bộ gặp nhiều khó khăn trong công tác quản lý và thống kê.

Từ thực tế trên, thông qua quá trình khảo sát tại Đoàn trường và Hội Sinh viên, có thể xác định ba vấn đề cốt lõi cần được giải quyết:

- **Quản lý Người dùng và Tổ chức:** Chưa tồn tại hệ thống định danh số hóa cho đoàn viên và hội viên. Việc chứng minh tư cách chủ yếu dựa trên sổ đoàn hoặc hồ sơ giấy tờ thủ công, dễ thất lạc hoặc hư hỏng, dẫn đến việc xác minh thông tin tốn nhiều thời gian và nhân lực (ví dụ: kiểm tra sổ đoàn gốc hoặc biên lai đóng hội phí).
- **Quản lý Hoạt động:** Thông tin về các chương trình, phong trào chủ yếu được đăng tải rải rác trên fanpage hoặc mạng xã hội, gây ra tình trạng trùng lặp, chồng chéo dữ liệu và thiếu công cụ lọc, thống kê tự động. Sinh viên phải tự tìm kiếm thông tin, trong khi cán bộ khó theo dõi và tổng hợp số lượng hoạt động theo từng giai đoạn.
- **Quản lý Tài chính:** Quy trình thu – chi chủ yếu dựa trên tiền mặt, với dòng tiền đi từ dưới lên (lớp thu, nộp khoa; khoa tổng hợp, nộp trường). Cách thức này tiềm ẩn nhiều rủi ro như thất thoát, sai lệch số liệu, thiếu minh bạch, đồng thời tạo áp lực và trách nhiệm lớn cho các cán bộ trung gian.

Những hạn chế nêu trên không chỉ làm giảm hiệu quả hoạt động của Đoàn – Hội mà còn ảnh hưởng trực tiếp đến quyền lợi và mức độ tham gia của sinh viên. Do đó, việc xây dựng một giải pháp chuyển đổi số toàn diện là yêu cầu cấp thiết nhằm nâng cao tính minh bạch, hiệu quả quản lý và khả năng mở rộng trong tương lai.

1.2 Các hướng giải quyết liên quan

Xuất phát từ các vấn đề cốt lõi đã được phân tích, có thể nhận thấy rằng những hạn chế trong công tác quản lý Đoàn – Hội không thể được khắc phục bằng các giải pháp rời rạc hay mang tính cục bộ. Thay vào đó, cần một hướng tiếp cận tổng thể dựa trên nền tảng chuyển đổi số, nhằm xây dựng một hệ thống quản lý thống nhất, đồng bộ và có khả năng mở rộng trong tương lai.

Đối với vấn đề quản lý người dùng và tổ chức, hướng giải quyết trọng tâm là số hóa dữ liệu định danh của Đoàn viên và Hội viên. Việc xây dựng hồ sơ điện tử cho mỗi sinh viên cho phép lưu trữ tập trung các thông tin về tư cách Đoàn viên, Hội viên cũng như lịch sử tham gia hoạt động. Qua đó, quá trình xác minh thông tin được thực hiện nhanh chóng, chính xác và giảm thiểu sự phụ thuộc vào hồ sơ giấy truyền thống.

Đối với vấn đề quản lý hoạt động, hướng giải quyết được đặt ra là tập trung hóa và chuẩn hóa thông tin hoạt động trên một nền tảng duy nhất. Các chương trình, phong trào và hoạt động xã hội sẽ được đăng tải, quản lý và theo dõi tập trung, giúp sinh viên dễ dàng tiếp cận và đăng ký tham gia, đồng thời hỗ trợ cán bộ trong việc thống kê, theo dõi và đánh giá hiệu quả tổ chức hoạt động theo từng giai đoạn.

Đối với vấn đề quản lý tài chính, hướng giải quyết chủ yếu là số hóa quy trình thu – chi và từng bước giảm sự phụ thuộc vào tiền mặt. Việc quản lý các khoản thu, chi thông qua hệ thống điện tử giúp đảm bảo tính minh bạch, khả năng truy xuất và kiểm soát dòng tiền, đồng thời hạn chế các rủi ro liên quan đến thất thoát và sai lệch số liệu. Bên cạnh đó, giải pháp này còn góp phần giảm áp lực và trách nhiệm cho các cán bộ trung gian trong quá trình thu nộp.

Trên cơ sở các hướng giải quyết nêu trên, đề tài hướng đến việc xây dựng một cổng thông tin điện tử tập trung dành cho hoạt động Đoàn – Hội, đóng vai trò là nền tảng quản lý thống nhất cho dữ liệu người dùng, hoạt động và tài chính. Đặc biệt, việc tích hợp công nghệ trí tuệ nhân tạo thông qua AI Chatbot hỗ trợ hỏi đáp tự động được kỳ vọng sẽ giúp sinh viên tiếp cận thông tin nhanh chóng, giảm tải công việc cho cán bộ và nâng cao hiệu quả vận hành của hệ thống.

2 Cơ sở lý thuyết và công nghệ

2.1 Cơ sở lý thuyết

Dự án được xây dựng dựa trên các khái niệm cốt lõi của Hệ thống thông tin quản lý (MIS) và Chuyển đổi số, cụ thể bao gồm:

- **Mô hình quản lý dữ liệu tập trung (Centralized Data Management):**
  - Xây dựng hệ thống lưu trữ thống nhất cho dữ liệu Đoàn viên, Hội viên, hoạt động và tài chính.
  - Giải quyết tình trạng dữ liệu phân mảnh, trùng lặp trên sổ sách giấy và mạng xã hội.
  - Tăng tốc độ tra cứu và đảm bảo tính chính xác của thông tin.
- **Lý thuyết chuyển đổi số trong tổ chức giáo dục (Digital Maturity Model - DMM):**
  - Số hóa quy trình nghiệp vụ để nâng cao hiệu quả hoạt động.
  - Đảm bảo nguyên tắc *accountability* trong quản lý tài chính thông qua minh bạch hóa số liệu.
  - Ứng dụng Xử lý ngôn ngữ tự nhiên (NLP) để tự động hóa hỗ trợ sinh viên 24/7.
- **An toàn thông tin và Kiểm soát truy cập:**
  - Áp dụng mô hình **CIA Triad** (Confidentiality - Integrity - Availability) để bảo vệ tính bảo mật, toàn vẹn và sẵn sàng của hệ thống định danh điện tử.
  - Sử dụng cơ chế **Phân quyền dựa trên vai trò (RBAC)**: Đảm bảo cán bộ chỉ truy cập dữ liệu phù hợp với chức năng, giảm thiểu rủi ro sai lệch dữ liệu.

2.2 Công nghệ

Dựa trên nền tảng lý thuyết, dự án áp dụng kiến trúc ứng dụng web hiện đại với các công nghệ sau:

- **Giao diện người dùng (Front-end): ReactJS**
  - Xây dựng giao diện web động, phản hồi tốt trên đa thiết bị.
  - Tái sử dụng thành phần (components) giúp sinh viên tiếp cận thông tin tập trung.
  - Kết hợp **Redux** (quản lý trạng thái) và **Material-UI** (giao diện trực quan).
- **Xử lý nghiệp vụ (Back-end): NestJS & TypeORM**
  - **NestJS**: Framework Node.js (TypeScript) với kiến trúc module, tối ưu cho API RESTful quy mô lớn.
  - **TypeORM**: Ánh xạ dữ liệu (*User, Activity, Finance*) vào cơ sở dữ liệu, tự động hóa xác minh định danh và thống kê.
- **Lưu trữ dữ liệu (Database): PostgreSQL**
  - Hệ quản trị CSDL quan hệ mã nguồn mở, độ tin cậy cao.
  - Hỗ trợ truy vấn phức tạp và đánh chỉ mục hiệu quả, thay thế hoàn toàn Excel thủ công.
- **Triển khai & Vận hành (DevOps): Docker & GitLab CI/CD**
  - **Docker**: Đóng gói ứng dụng (containerization) đảm bảo môi trường nhất quán.
  - **GitLab CI/CD**: Tự động hóa quy trình xây dựng, kiểm thử và triển khai (update) hệ thống.
- **Thanh toán điện tử: PayOS API**

  - Tích hợp cổng thanh toán trực tuyến cho hội phí và các khoản thu chi.
  - Minh bạch hóa dòng tiền, giảm rủi ro thất thoát tiền mặt.

- **Trí tuệ nhân tạo: AI Chatbot**
  - Sử dụng mô hình Chat Bot RAG (Retrieval-Augmented Generation - Tạo sinh Tăng cường Truy xuất) với cốt lõi là framework LangChain.js
  - Tự động giải đáp thắc mắc về quy chế, hoạt động, giảm tải cho cán bộ quản lý.

3 Yêu cầu về hệ thống

3.1 Yêu cầu chức năng

Dựa trên phân tích các vấn đề cốt lõi trong công tác quản lý Đoàn – Hội tại Trường Đại học Bách Khoa, hệ thống cần đáp ứng các yêu cầu chức năng sau:

3.1.1 Quản lý Người dùng và Tổ chức

- **Quản lý hồ sơ điện tử:**
  - Hệ thống phải lưu trữ và quản lý thông tin định danh của Đoàn viên và Hội viên bao gồm: mã số sinh viên (MSSV), họ tên, email, số điện thoại, ngày sinh, giới tính, tình trạng Đoàn viên/Hội viên.
  - Hệ thống phải lưu trữ thông tin học vụ: khoa, bộ môn, lớp, niên khóa, chương trình đào tạo.
  - Hệ thống phải tự động tích lũy điểm CTXH khi sinh viên hoàn thành hoạt động và được chấm điểm.
  - Hệ thống phải hỗ trợ import hàng loạt từ file Excel (18 cột) với khả năng tự động tạo tổ chức/khoa/lớp nếu chưa tồn tại trong hệ thống.
- **Quản lý cơ cấu tổ chức:**
  - Hệ thống phải hỗ trợ phân cấp tổ chức theo mô hình: Trường → Khoa (Đoàn/Hội cơ sở) → Lớp (Chi đoàn/Chi hội).
  - Hệ thống phải hỗ trợ 8 loại tổ chức: Đoàn trường, Đoàn cơ sở, Chi đoàn, Hội sinh viên trường, Liên chi hội, Chi hội, CLB trường, CLB khoa.
  - Hệ thống phải cho phép quản lý khoa (Department) và lớp (Class) độc lập, đồng thời liên kết với tổ chức tương ứng.
  - Hệ thống phải cho phép gán sinh viên vào nhiều tổ chức với các vai trò: member, admin, hoặc super_admin.
- **Phân quyền truy cập (RBAC):**
  - Hệ thống phải áp dụng mô hình Role-Based Access Control với context theo tổ chức.
  - Hệ thống phải phân chia quyền theo 2 cấp độ: System (toàn hệ thống) và Organization (theo đơn vị cụ thể).
  - Hệ thống phải đảm bảo cán bộ chỉ được truy cập và chỉnh sửa dữ liệu thuộc phạm vi quản lý (organization scope).
  - Hệ thống phải hỗ trợ gán nhiều vai trò cho cùng một user ở các tổ chức khác nhau.
  - Hệ thống phải cung cấp các vai trò mặc định: Super Admin, Quản trị viên tổ chức, Quản lý, Biên tập viên, Người xem.

3.1.2 Quản lý Hoạt động CTXH

- **Tạo và quản lý hoạt động:**
  - Hệ thống phải cho phép cán bộ tạo, chỉnh sửa, xóa mềm (soft delete) thông tin hoạt động bao gồm: tên, mô tả, thời gian bắt đầu/kết thúc, địa điểm, số lượng tham gia, tổng số ngày CTXH.
  - Hệ thống phải quản lý 4 trạng thái hoạt động: DRAFT (Nháp), ACTIVE (Đang hoạt động), COMPLETED (Đã kết thúc), CANCELLED (Đã hủy).
  - Hệ thống phải kiểm tra ràng buộc: chỉ cho phép thêm sinh viên khi hoạt động ở trạng thái ACTIVE; chỉ cho phép chấm điểm khi hoạt động ở trạng thái ACTIVE hoặc COMPLETED.

- Hệ thống phải hỗ trợ phân loại hoạt động theo danh mục (Category) với cấu trúc phân cấp cây.

- **Quản lý danh sách tham gia:**
  - Hệ thống phải cho phép cán bộ thêm sinh viên vào hoạt động theo 2 cách:
    1. Thêm thủ công từng sinh viên bằng cách nhập mã số sinh viên.
    2. Upload file Excel chứa danh sách sinh viên (hệ thống tự động parse và validate từng dòng).
  - Hệ thống phải theo dõi 2 trạng thái tham gia: PENDING (Chưa chấm điểm), GRADED (Đã chấm điểm).
  - Hệ thống phải ghi nhận các thông tin: ngày đăng ký tham gia, ngày chấm điểm, số ngày CTXH thực tế, ghi chú, mã người chấm điểm.

- **Chấm điểm CTXH:**
  - Hệ thống phải cho phép cán bộ chấm điểm thủ công bằng cách nhập số ngày thực tế (actual_days) và ghi chú cho từng sinh viên.
  - Hệ thống phải hỗ trợ upload file Excel để chấm điểm hàng loạt, kèm validation: số ngày không được vượt quá total_days của hoạt động, không cho phép chấm trùng lặp.
  - Hệ thống phải cung cấp chức năng tải template Excel chứa danh sách sinh viên đã tham gia để cán bộ điền điểm.
  - Hệ thống phải tự động cộng dồn điểm CTXH vào tổng điểm trong hồ sơ sinh viên khi chấm điểm thành công.

- **Thống kê và báo cáo:**
  - Báo cáo số lượng hoạt động theo danh mục, tổ chức, trạng thái.
  - Thống kê tỷ lệ hoàn thành của sinh viên theo hoạt động.
  - Xuất báo cáo điểm CTXH cá nhân và tổng hợp của hoạt động (file Excel).

3.1.3 Quản lý Đóng Phí

- **Tạo đợt đóng phí:**
  - Cán bộ tạo Payment Campaign với thông tin: tên đợt, mô tả, thời gian bắt đầu/kết thúc, tổ chức phụ trách.
  - Quản lý trạng thái đợt: DRAFT, ACTIVE, COMPLETED, CANCELLED.
  - Cấu hình mức phí (Fee Level) theo đối tượng: sinh viên, cán bộ, hoặc custom với số tiền khác nhau.

- **Phân công đóng phí:**
  - Cán bộ phân công sinh viên vào đợt đóng phí (Payment Assignment) với mức phí tương ứng.
  - Hỗ trợ phân công hàng loạt theo lớp, khoa, hoặc danh sách MSSV.
  - Theo dõi trạng thái thanh toán: PENDING, PAID, CANCELLED.
  - Ghi nhận thông tin: hạn đóng, số tiền, ghi chú.

- **Thanh toán trực tuyến qua PayOS:**
  - Sinh viên xem danh sách các khoản phí được phân công trong trang ”Đóng phí của tôi”.
  - Click ”Thanh toán ngay” để chuyển đến cổng PayOS.
  - Hỗ trợ các phương thức: chuyển khoản ngân hàng, ví điện tử (VNPay, Momo, ZaloPay).

  - Webhook tự động cập nhật trạng thái thanh toán sau khi giao dịch thành công.

- **Cấu hình PayOS theo từng đợt:**
  - Mỗi đợt đóng phí có thể cấu hình riêng: Client ID, API Key, Checksum Key.
  - Hỗ trợ kế thừa cấu hình từ tổ chức cha nếu không tự cấu hình.
  - Lưu lịch sử giao dịch (Payment History): mã giao dịch, số tiền, trạng thái, thời gian.

- **Báo cáo thu phí:**
  - Theo dõi tiến độ thu phí: tổng số người được phân công, số người đã đóng, tỷ lệ hoàn thành.
  - Thống kê doanh thu theo đợt, theo tổ chức, theo thời gian.
  - Xuất báo cáo Excel danh sách sinh viên đã/chưa đóng phí.

3.1.4 Quản lý Biểu mẫu và Chứng từ

- **Upload và parse template:**
  - Quản trị viên upload file Word (.docx) chứa các placeholder dạng `{{field_name}}`.
  - Hệ thống tự động parse file bằng thư viện Docxtemplater, trích xuất danh sách merge fields.
  - Upload file template lên S3 (Long Vân), lưu URL vào Database.

- **Cấu hình fields:**
  - Quản trị viên cấu hình từng field: tên hiển thị, kiểu dữ liệu (TEXT, NUMBER, DATE, SELECT...), required, validation rules.
  - Hỗ trợ các field đặc biệt: DATE_PARTS (tách ngày/tháng/năm), SELECT với options tùy chỉnh.
  - Quản lý trạng thái template: PROCESSING (đang xử lý), ACTIVE (sẵn sàng sử dụng), INACTIVE (vô hiệu hóa).

- **Điền và nộp biểu mẫu:**
  - Sinh viên chọn template, hệ thống hiển thị form động dựa trên cấu hình fields.
  - Validate dữ liệu trên client (ReactJS) và server (NestJS) trước khi submit.

- **Generate document:**
  - Khi submit, hệ thống download template từ S3, load vào Docxtemplater.
  - Transform dữ liệu: chuyển DATE thành DD/MM/YYYY, tạo field_day/field_month/field_year cho DATE_PARTS.
  - Render document với dữ liệu đã transform, generate buffer.
  - Upload file output lên S3 với đường dẫn: `submissions/{userId}/{timestamp}_{fileName}.docx`.
  - Cập nhật submission: status = COMPLETED, outputFileUrl, completedAt.

- **Lưu trữ và tra cứu:**
  - Lưu metadata submission trong Database: templateId, userId, formData (JSON), status, timestamps.
  - Sinh viên xem danh sách biểu mẫu đã nộp, tải lại file đã generate.
  - Hỗ trợ preview trực tuyến qua Office Online Viewer (presigned URL từ S3).

3.1.5 Chatbot hỏi đáp hoạt động công tác xã hội

- Trả lời câu hỏi về thông tin hoạt động công tác xã hội hiện có trong hệ thống

3.2 Yêu cầu phi chức năng

Hệ thống được thiết kế với các yêu cầu phi chức năng sau đây, dựa trên công nghệ đã triển khai và nhu cầu thực tế của đối tượng sử dụng:

3.2.1 Bảo mật và An toàn thông tin

- **Xác thực và phân quyền:**
  - Hệ thống phải mã hóa mật khẩu bằng thuật toán bcrypt với salt rounds = 10 trước khi lưu vào cơ sở dữ liệu.
  - Hệ thống phải xác thực người dùng qua JWT với thời gian hết hạn access token là 15 phút và refresh token là 7 ngày.
  - Hệ thống phải phân quyền theo mô hình RBAC với context tổ chức, đảm bảo mỗi user chỉ được truy cập dữ liệu thuộc phạm vi được gán.
  - Hệ thống phải kiểm tra quyền trước khi cho phép thực thi mọi API liên quan đến tạo, sửa, xóa dữ liệu quan trọng.
- **Bảo vệ dữ liệu:**
  - Hệ thống phải validate và sanitize mọi input từ người dùng trên cả client và server để phòng chống SQL Injection, XSS.
  - Hệ thống phải sử dụng ORM (TypeORM) để xây dựng query, tránh concatenate SQL string trực tiếp.
  - Hệ thống phải áp dụng soft delete cho các entity quan trọng (users, activities, organizations) để có thể khôi phục khi cần thiết.
  - Hệ thống phải lưu trữ timestamp (createdAt, updatedAt, deletedAt) và thông tin người thao tác cho mọi entity quan trọng phục vụ audit trail.
- **Triển khai bảo mật:**
  - Hệ thống phải sử dụng HTTPS với chứng chỉ SSL/TLS hợp lệ trên môi trường production.
  - Hệ thống phải cấu hình CORS chỉ cho phép request từ domain chính thức của ứng dụng.
  - Hệ thống phải kiểm tra extension và MIME type khi upload file, chỉ chấp nhận .xlsx và .docx với kích thước tối đa 10MB.

3.2.2 Hiệu năng

- **Tối ưu hóa cơ sở dữ liệu:**
  - Hệ thống phải đánh index trên các trường tra cứu thường xuyên (code, email, student_code) để đảm bảo thời gian truy vấn < 100ms với bảng có $< 100{,}000$ bản ghi.
  - Hệ thống phải áp dụng pagination cho mọi danh sách với kích thước trang mặc định 10-50 bản ghi.
  - Hệ thống phải sử dụng eager/lazy loading hợp lý để tránh N+1 query problem, đảm bảo số lượng query ≤ 10 cho mỗi request.
  - Hệ thống phải xử lý batch operations (import Excel > 1000 dòng) theo chunk để tránh timeout.
- **Xử lý file:**
  - Hệ thống phải upload file lên S3 storage thay vì lưu trực tiếp vào Database để tránh quá tải.
  - Hệ thống phải parse file Excel theo chunk (200 dòng/lần) để xử lý file lớn mà không quá tải bộ nhớ.

  - Hệ thống phải generate document Word bất đồng bộ và cập nhật trạng thái khi hoàn thành, timeout tối đa 60 giây.

- **Tối ưu Frontend:**
  - Hệ thống phải sử dụng client-side caching để giảm số lần gọi API lặp lại cho cùng dữ liệu (cache time: 5 phút).
  - Hệ thống phải lazy load các module và route để đảm bảo initial bundle size $<$ 500KB (gzipped).
  - Hệ thống phải tối ưu re-render để đảm bảo thời gian phản hồi UI $<$ 100ms khi người dùng tương tác.

3.2.3 Khả năng triển khai và Mở rộng

- **Containerization:**
  - Đóng gói ứng dụng bằng Docker: backend (NestJS), frontend (ReactJS + Nginx), database (PostgreSQL).
  - Sử dụng Docker Compose để orchestrate các container trong môi trường development và production.
  - Multi-stage build trong Dockerfile để giảm kích thước image production.
- **Kiến trúc modular:**
  - Tách biệt các module: Users, Organizations, CTXH, Payment, Templates, Auth.
  - Mỗi module có structure riêng: entities, services, controllers, DTOs.
  - Dependency injection với NestJS để dễ dàng test và thay thế implementation.
- **Lưu trữ phân tán:**
  - File storage sử dụng S3-compatible (Long Vân), tách biệt khỏi database server.
  - Presigned URL cho file private, cho phép download trực tiếp từ S3 mà không qua backend.

3.2.4 Giám sát và Logging

- **Application monitoring:**
  - Tích hợp Prometheus để thu thập metrics: request count, response time, error rate.
  - Visualize metrics với Grafana dashboard, theo dõi real-time.
  - Cấu hình alerts khi phát hiện bất thường (error rate tăng cao, memory leak).
- **Logging:**
  - Sử dụng NestJS Logger để ghi log với các level: error, warn, info, debug.
  - Log các thao tác quan trọng: login, create/update/delete user, chấm điểm CTXH, thanh toán.
  - Lưu trữ logs để phục vụ debugging và audit.
- **Error handling:**
  - Global exception filter để bắt và format lỗi thống nhất.
  - Trả về error message rõ ràng cho frontend hiển thị (không expose stack trace ở production).
  - Transaction rollback tự động khi có lỗi trong các thao tác liên quan nhiều bảng.

3.2.5 Giao diện và Trải nghiệm người dùng

- **Responsive design:**
  - Sử dụng Tailwind CSS với breakpoints để hỗ trợ desktop, tablet, mobile.
  - Test giao diện trên các kích thước màn hình phổ biến (1920x1080, 1366x768, 375x667).
- **Dark mode:**
  - Hỗ trợ chế độ sáng/tối, tự động lưu preference của user.
  - Áp dụng dark mode classes cho tất cả component với Tailwind CSS.
- **Đa ngôn ngữ:**
  - Tích hợp i18next để hỗ trợ tiếng Việt và tiếng Anh.
  - Tách file translation theo module để dễ quản lý và mở rộng.
- **UX improvements:**
  - Loading states cho các thao tác mất thời gian (upload file, generate document).
  - Toast notifications cho feedback tức thì khi thành công/lỗi.
  - Confirmation dialog cho các thao tác nguy hiểm (xóa user, hủy hoạt động).

3.2.6 Code Quality và Maintainability

- **Coding standards:**
  - Backend: tuân thủ NestJS style guide, sử dụng TypeScript strict mode.
  - Frontend: ESLint + Prettier để format code tự động, maintain consistency.
  - Naming convention rõ ràng: camelCase cho biến/hàm, PascalCase cho class/component.
- **Documentation:**
  - API documentation tự động với Swagger UI (`/api-docs`).
  - Inline comments cho logic phức tạp trong code.
  - README files cho từng module, giải thích cách sử dụng và dependencies.
- **Version control:**
  - Sử dụng Git với branch strategy: main (production), develop (staging), feature branches.
  - Commit message theo format: type(scope): description (VD: feat(ctxh): add grading from Excel).

4 Mô hình hóa hệ thống

4.1 Mô hình hóa Use Case

4.1.1 Quản lý thành viên

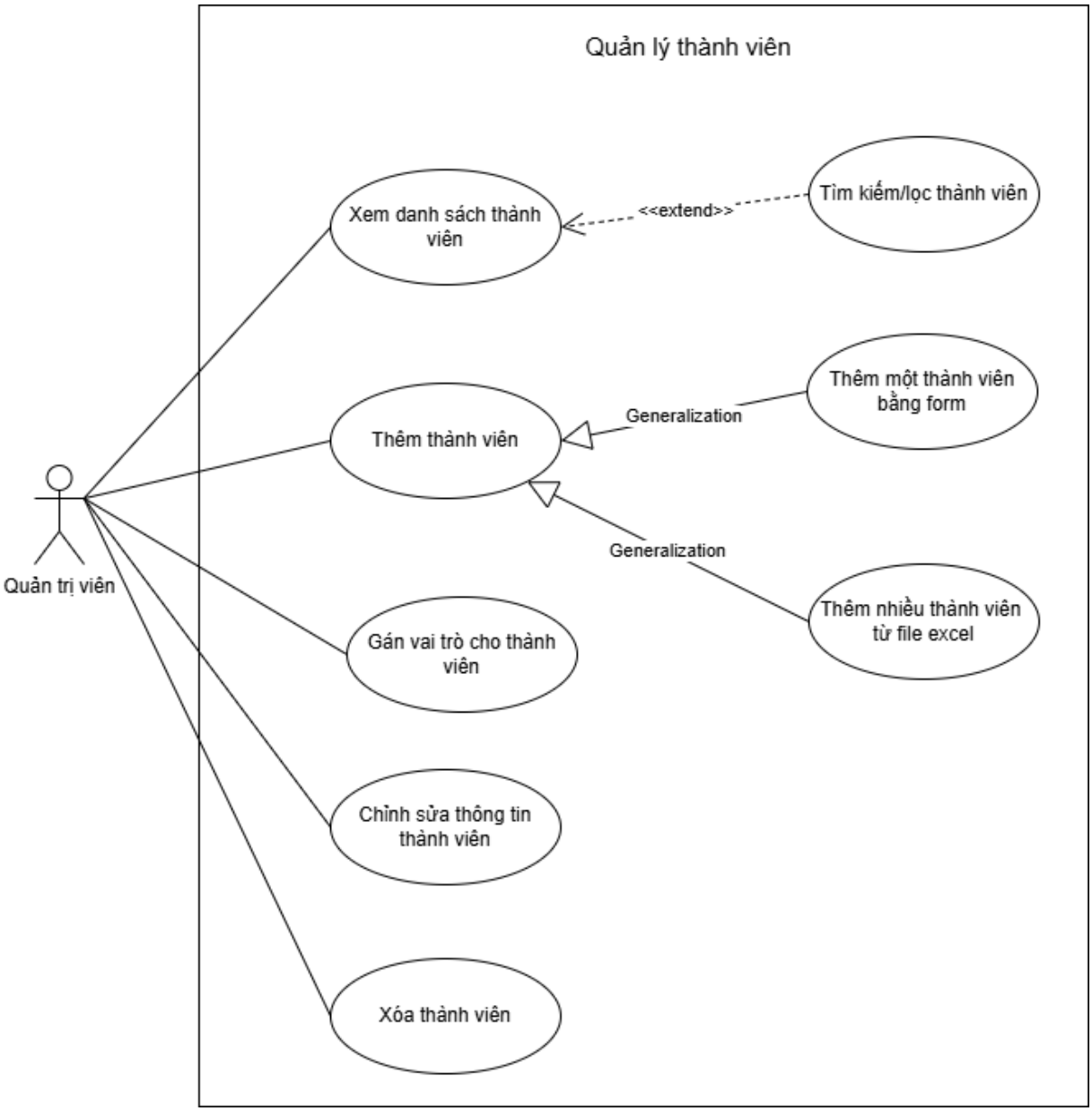


Figure 1: *UC01 - Quản lý thành viên*

Use-case Name	**UC01. Quản lý thành viên**
Actors	Quản trị viên
Description	Use case này cho phép quản trị viên thực hiện các thao tác quản lý thông tin thành viên trong hệ thống, bao gồm: – Xem danh sách thành viên – Tìm kiếm và lọc thành viên – Thêm, sửa, xóa thông tin thành viên – Gán vai trò (phân quyền) cho thành viên
Trigger	Quản trị viên truy cập chức năng Quản lý thành viên từ giao diện hệ thống.
Pre-Conditions	Quản trị viên đã đăng nhập vào hệ thống bằng tài khoản có quyền quản lý thành viên.
Post-Conditions	Thông tin thành viên được hiển thị và cập nhật chính xác theo các thao tác đã thực hiện.
Normal Flow	1. Quản trị viên truy cập module *Quản lý thành viên*. 2. Hệ thống hiển thị danh sách thành viên. 3. Quản trị viên chọn một trong các thao tác: – Xem chi tiết thông tin thành viên – Tìm kiếm hoặc lọc thành viên – Thêm mới thành viên – Chỉnh sửa thông tin thành viên – Xóa thành viên – Gán vai trò (phân quyền) cho thành viên 4. Hệ thống xử lý yêu cầu và cập nhật dữ liệu. 5. Hệ thống hiển thị kết quả thao tác cho quản trị viên.
Alternative Flow	Ở bước 3: – **3a.** Quản trị viên thực hiện tìm kiếm hoặc lọc thành viên, hệ thống hiển thị danh sách thành viên phù hợp với điều kiện tìm kiếm. – **3b.** Quản trị viên chọn hủy thao tác thêm/sửa, hệ thống quay lại danh sách thành viên mà không thay đổi dữ liệu.
Exception	Ở bước 1 – **1a.** Nếu quản trị viên không có quyền truy cập chức năng quản lý thành viên, hệ thống chặn truy cập và không hiển thị module này trên giao diện. Ở bước 4: – **4a.** Nếu dữ liệu nhập không hợp lệ, hệ thống hiển thị thông báo lỗi và yêu cầu quản trị viên nhập lại.

Table 1: *Bảng mô tả Use Case UC01 – Quản lý thành viên*

Use-case Name	**UC01.1. Thêm một thành viên từ Form**
Actors	Quản trị viên
Description	Quản trị viên thêm một thành viên mới vào hệ thống thông qua form nhập liệu.
Trigger	Quản trị viên bấm nút *"Thêm thành viên mới"* trong trang quản lý người dùng.
Pre-Conditions	– Quản trị viên đã đăng nhập vào hệ thống – Quản trị viên có quyền tạo thành viên mới
Post-Conditions	– Thành viên mới được tạo và lưu vào cơ sở dữ liệu – Mật khẩu mặc định được tạo tự động từ email hoặc mã số – Thành viên được gán vào tổ chức (nếu có) – Thành viên được gán vai trò (nếu có) – Hệ thống gửi thông báo về mật khẩu mặc định cho thành viên
Normal Flow	1. Quản trị viên bấm nút *"Thêm thành viên mới"*. 2. Hệ thống hiển thị form nhập thông tin thành viên với các trường: – **Bắt buộc**: Họ và tên đệm, Tên, Email, Mã số sinh viên/cán bộ – **Tùy chọn**: Số điện thoại, Mật khẩu, Tổ chức, Loại thành viên, Trạng thái 3. Quản trị viên điền thông tin vào form. 4. Quản trị viên chọn tổ chức (nếu có). 5. Quản trị viên chọn vai trò và tổ chức áp dụng vai trò (nếu có). 6. Quản trị viên bấm nút *"Tạo mới"*. 7. Hệ thống kiểm tra tính hợp lệ của dữ liệu. 8. Hệ thống tự động tạo mật khẩu mặc định từ email hoặc mã số. 9. Hệ thống lưu thông tin thành viên. 10. Hệ thống hiển thị thông báo *"Tạo thành viên thành công"*. 11. Hệ thống chuyển hướng về trang danh sách thành viên.
Alternative Flow	– **8.a.** Không cung cấp mật khẩu: + Hệ thống tạo mật khẩu từ phần trước ký tự `@` của email + Nếu không có email, sử dụng mã số sinh viên/cán bộ + Mật khẩu được mã hóa trước khi lưu
Exception	– **7.a.** Email hoặc mã số đã tồn tại: + Hiển thị lỗi *"Email hoặc mã số đã tồn tại trong hệ thống"* + Quay lại bước 3 – **7.b.** Thiếu trường bắt buộc: + Hiển thị lỗi *"Vui lòng điền đầy đủ thông tin bắt buộc"* + Quay lại bước 3

Table 2: *Bảng mô tả Use Case UC01.1 - Thêm một thành viên từ Form*

Use-case Name	**UC01.2. Thêm nhiều thành viên từ file Excel**
Actors	Quản trị viên
Description	Quản trị viên thêm hàng loạt thành viên từ file Excel theo template chuẩn gồm 18 cột dữ liệu.
Trigger	Quản trị viên cần thêm nhiều thành viên cùng lúc (ví dụ: đầu năm học, import danh sách sinh viên mới).
Pre-Conditions	– Quản trị viên đã đăng nhập vào hệ thống – Quản trị viên có quyền import thành viên – File Excel đã được chuẩn bị theo template chuẩn
Post-Conditions	– Danh sách thành viên được import thành công – Các tổ chức được tự động tạo nếu chưa tồn tại – Mật khẩu mặc định được tạo cho mỗi thành viên – Hệ thống ghi log chi tiết quá trình import – Báo cáo kết quả import được tạo
Normal Flow	1. Quản trị viên bấm nút *"Thêm từ File"*. 2. Hệ thống hiển thị hướng dẫn và nút tải file mẫu. 3. Quản trị viên tải template Excel (18 cột). 4. Quản trị viên điền dữ liệu vào template. 5. Quản trị viên upload file Excel. 6. Hệ thống nhận file và tạo session import. 7. Hệ thống parse file và trả về `sessionId`. 8. Quản trị viên bấm *"Validate dữ liệu"*. 9. Hệ thống thực hiện validation và hiển thị kết quả. 10. Quản trị viên bấm *"Thực hiện import"*. 11. Hệ thống import dữ liệu theo batch. 12. Hệ thống hiển thị tiến trình import real-time. 13. Hệ thống hiển thị báo cáo kết quả import.
Alternative Flow	– **13.a.** Tự động tạo tổ chức: + Phát hiện mã khoa/lớp chưa tồn tại + Tạo tổ chức mới + Gán thành viên vào tổ chức vừa tạo
Exception	– **6.a.** File không đúng định dạng (.xlsx) – **6.b.** File quá lớn (>10MB) – **6.c.** File không đúng template – **9.a.** 100% bản ghi có lỗi nghiêm trọng

Table 3: *Bảng mô tả Use Case UC01.2 - Thêm nhiều thành viên từ file Excel*

Use-case Name	**UC01.3. Gán vai trò cho thành viên**
Actors	Quản trị viên
Description	Quản trị viên gán một hoặc nhiều vai trò cho thành viên. Mỗi vai trò có thể được gán trong phạm vi tổ chức cụ thể hoặc trên toàn hệ thống.
Trigger	Quản trị viên cần phân quyền cho thành viên để họ thực hiện các chức năng trong hệ thống.
Pre-Conditions	– Quản trị viên đã đăng nhập vào hệ thống – Quản trị viên có quyền gán vai trò – Thành viên đã tồn tại trong hệ thống – Hệ thống đã có các vai trò được định nghĩa
Post-Conditions	– Thành viên được gán vai trò mới – Quyền hạn của thành viên được cập nhật theo vai trò – Lịch sử gán vai trò được ghi log – Thành viên có thể thực hiện các chức năng theo quyền mới
Normal Flow	1. Quản trị viên truy cập trang chi tiết thành viên. 2. Hệ thống hiển thị thông tin thành viên và danh sách vai trò hiện tại. 3. Quản trị viên bấm nút *"Gán vai trò"*. 4. Hệ thống hiển thị form gán vai trò với: – Dropdown chọn vai trò (Super Admin, Quản lý đóng phí, …) – Dropdown chọn tổ chức áp dụng (tùy chọn) 5. Quản trị viên chọn vai trò từ danh sách. 6. Quản trị viên chọn tổ chức áp dụng vai trò (nếu vai trò yêu cầu phạm vi). 7. Quản trị viên bấm nút *"Lưu"*. 8. Hệ thống kiểm tra vai trò–tổ chức không bị trùng lặp. 9. Hệ thống lưu thông tin vai trò mới cho thành viên. 10. Hệ thống cập nhật quyền hạn của thành viên. 11. Hệ thống hiển thị thông báo *"Gán vai trò thành công"*. 12. Danh sách vai trò của thành viên được cập nhật.

Use-case Name (tiếp)	UC01.3. Gán vai trò cho thành viên
Alternative Flow	– **6.a.** Không chọn tổ chức (vai trò toàn hệ thống): + Quản trị viên bỏ trống trường tổ chức + Vai trò được gán ở phạm vi toàn hệ thống + Thành viên có quyền hạn trong tất cả tổ chức – **7.a.** Gán nhiều vai trò cùng lúc: + Sau khi lưu vai trò đầu tiên, bấm *"Thêm vai trò khác"* + Lặp lại bước 5–7 cho mỗi vai trò + Hệ thống lưu tất cả các vai trò – **7.b.** Xóa vai trò hiện tại: + Bấm nút *"Xóa"* tại vai trò cần xóa + Hệ thống hiển thị xác nhận xóa + Quản trị viên xác nhận + Hệ thống xóa vai trò và cập nhật quyền hạn – **7.c.** Cập nhật toàn bộ vai trò: + Truy cập chế độ *"Chỉnh sửa vai trò"* + Thêm hoặc xóa các vai trò theo nhu cầu + Bấm *"Lưu tất cả"* + Hệ thống xóa toàn bộ vai trò cũ + Hệ thống gán lại các vai trò mới
Exception	– **8.a.** Vai trò–tổ chức đã tồn tại: + Phát hiện vai trò đã được gán cho thành viên trong tổ chức này + Hiển thị lỗi *"Thành viên đã có vai trò này trong tổ chức đã chọn"* + Quay lại bước 5 – **8.b.** Vai trò yêu cầu tổ chức nhưng không được chọn: + Hiển thị lỗi *"Vai trò này yêu cầu chọn tổ chức áp dụng"* + Quay lại bước 6 – **8.c.** Không đủ quyền gán vai trò: + Không có quyền gán vai trò cấp cao (ví dụ: Admin) + Hiển thị lỗi *"Bạn không có quyền gán vai trò này"* + Quay lại bước 5

Table 4: *Bảng mô tả Use Case UC01.3 - Gán vai trò cho thành viên*

4.1.2 Quản lý đơn vị

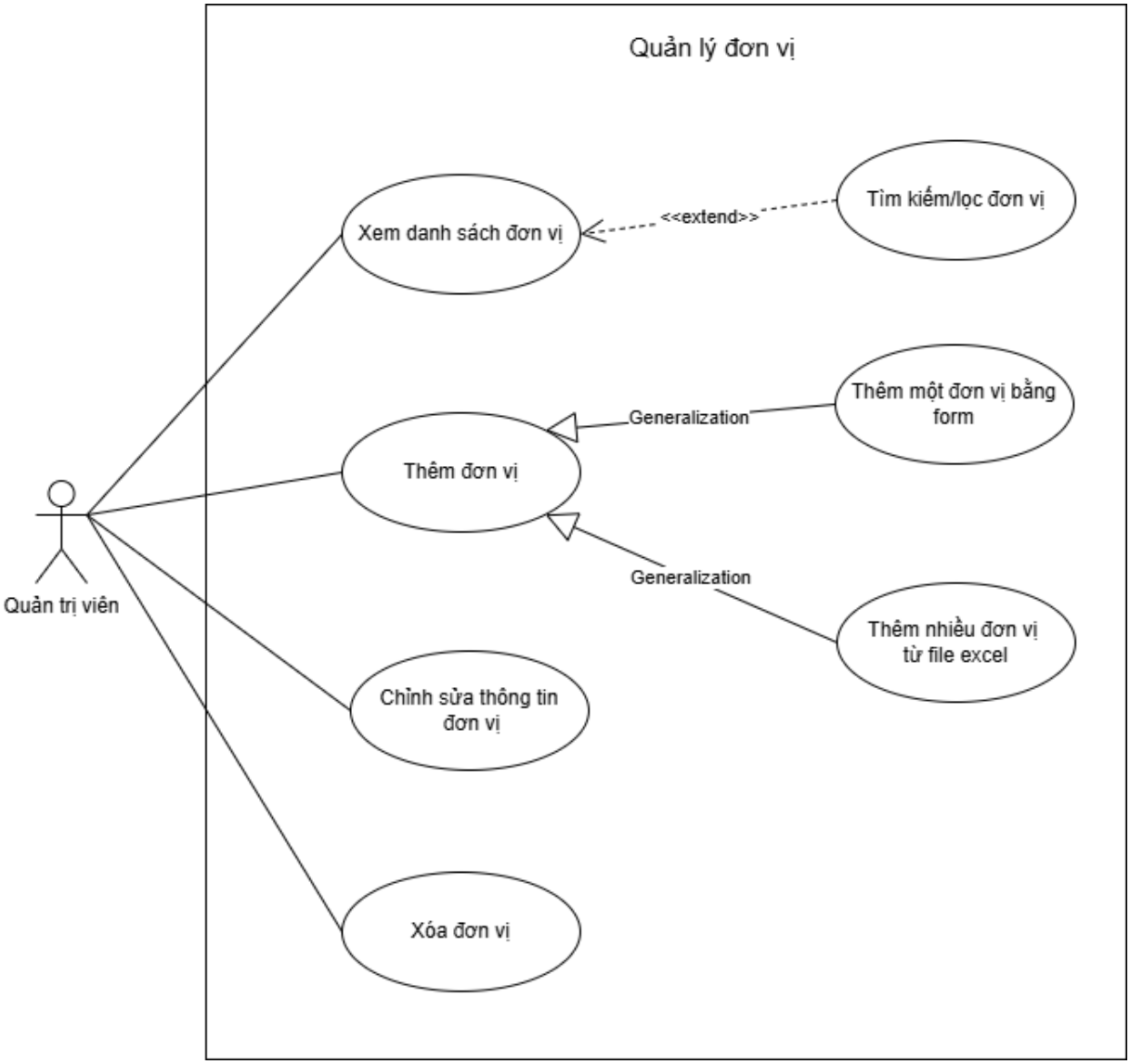


Figure 2: *UC02 - Quản lý đơn vị*

Use Case Name	**UC02. Quản lý đơn vị**
Actors	Quản trị viên
Description	Use case này cho phép quản trị viên thực hiện các thao tác quản lý thông tin đơn vị trong hệ thống bao gồm: • Xem danh sách đơn vị • Tìm và lọc đơn vị • Thực hiện các thao tác thêm, xóa, sửa thông tin đơn vị
Trigger	Quản trị viên truy cập chức năng quản lý đơn vị từ giao diện hệ thống.
Pre-Conditions	Quản trị viên đã đăng nhập vào hệ thống bằng tài khoản có phân quyền quản lý đơn vị.
Post-Conditions	Thông tin đơn vị được hiển thị và cập nhật đúng theo thao tác đã thực hiện.
Normal Flow	1. Quản trị viên truy cập module *Quản lý đơn vị*. 2. Hệ thống hiển thị danh sách đơn vị. 3. Quản trị viên chọn một trong các thao tác: • Xem danh sách đơn vị • Tìm và lọc đơn vị • Thực hiện các thao tác thêm, xóa, sửa thông tin đơn vị
Alternative Flow	Ở bước 3: • **3a.** Quản trị viên thực hiện tìm kiếm/lọc đơn vị, hệ thống hiển thị danh sách đơn vị phù hợp với điều kiện tìm kiếm.
Exception	Ở bước 1: • **1a.** Nếu quản trị viên không có phân quyền truy cập vào chức năng quản lý đơn vị thì hệ thống sẽ chặn truy cập và không hiển thị module này ở phía giao diện người dùng.

Table 5: *Bảng mô tả Use Case UC02 - Quản lý đơn vị*

4.1.3 Quản lý lớp học

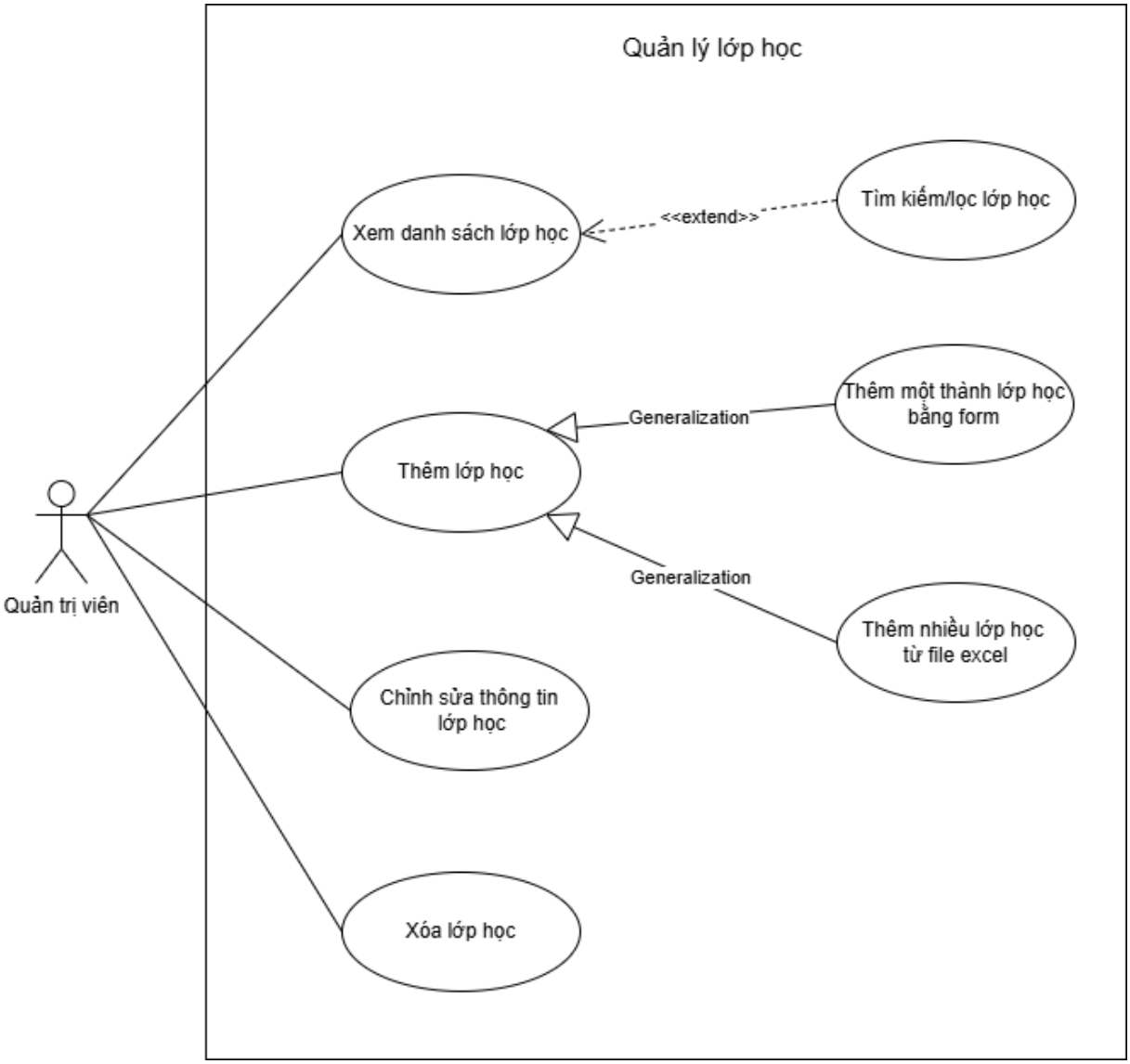


Figure 3: *UC03 - Quản lý lớp học*

Use Case Name	**UC03. Quản lý lớp học**
Actors	Quản trị viên
Description	Use case này cho phép quản trị viên thực hiện các thao tác quản lý thông tin lớp học trong hệ thống bao gồm: • Xem danh sách lớp học • Tìm và lọc lớp học • Thực hiện các thao tác thêm, xóa, sửa thông tin lớp học
Trigger	Quản trị viên truy cập chức năng quản lý lớp học từ giao diện hệ thống.
Pre-Conditions	Quản trị viên đã đăng nhập vào hệ thống bằng tài khoản có phân quyền quản lý lớp học.
Post-Conditions	Thông tin lớp học được hiển thị và cập nhật đúng theo thao tác đã thực hiện.
Normal Flow	1. Quản trị viên truy cập module *Quản lý lớp học*. 2. Hệ thống hiển thị danh sách lớp học. 3. Quản trị viên chọn một trong các thao tác: • Xem danh sách lớp học • Tìm và lọc lớp học • Thực hiện các thao tác thêm, xóa, sửa thông tin lớp học
Alternative Flow	Ở bước 3: • **3a.** Quản trị viên thực hiện tìm kiếm/lọc lớp học, hệ thống hiển thị danh sách lớp học phù hợp với điều kiện tìm kiếm.
Exception	Ở bước 1: • **1a.** Nếu quản trị viên không có phân quyền truy cập vào chức năng quản lý lớp học thì hệ thống sẽ chặn truy cập và không hiển thị module này ở phía giao diện người dùng.

Table 6: *Bảng mô tả Use Case UC03 - Quản lý lớp học*

4.1.4 Quản lý tổ chức

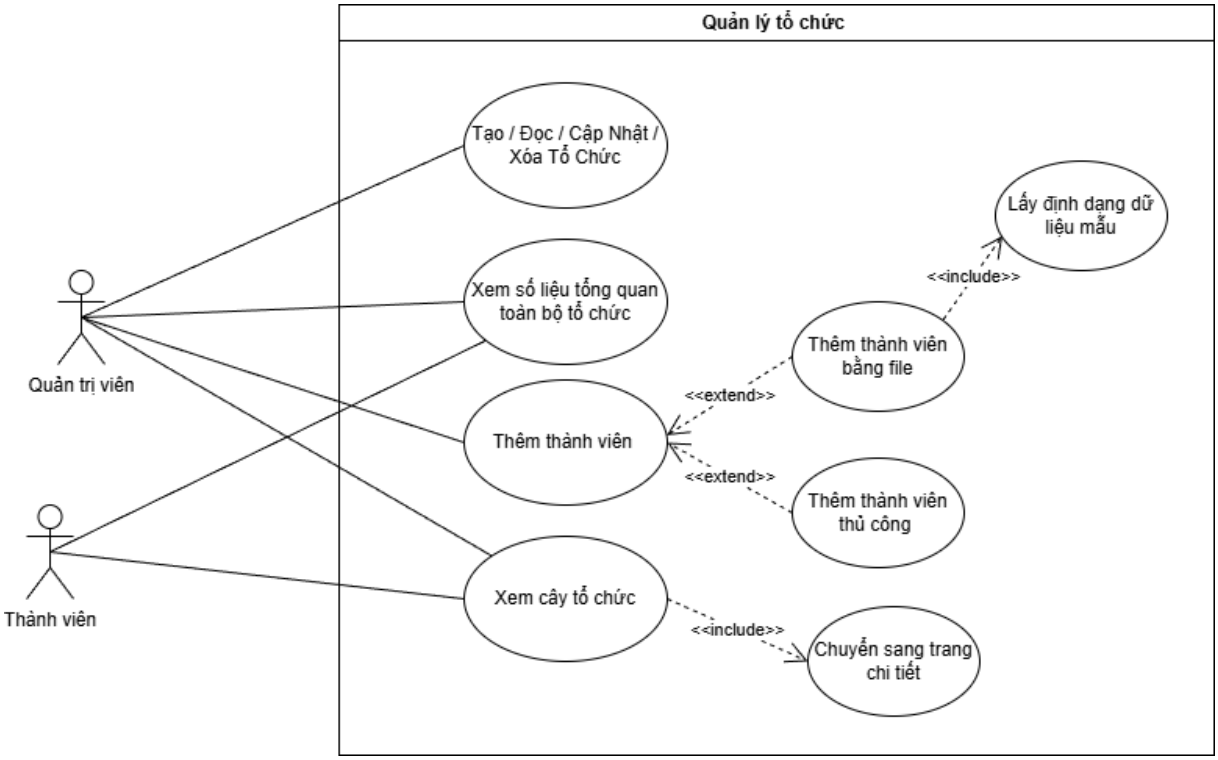


Figure 4: *UC04 - Quản lý tổ chức*

Use-case Name	**UC04. Quản lý tổ chức**
Actors	– Quản trị viên (Admin) – Thành viên (Member)
Description	Cho phép người dùng xem cấu trúc cây tổ chức và số liệu thống kê tổng quan. Đối với Quản trị viên, hệ thống cung cấp các chức năng quản lý nâng cao như: Tạo/Sửa/Xóa tổ chức và Thêm thành viên (hỗ trợ thêm thủ công hoặc nhập từ file).
Trigger	Người dùng truy cập vào trang ”Quản lý tổ chức” từ thanh điều hướng chính.
Pre-Conditions	– Người dùng đã đăng nhập vào hệ thống. – Dữ liệu về tổ chức và thành viên đã tồn tại trong cơ sở dữ liệu.
Post-Conditions	– Hiển thị chính xác cấu trúc cây tổ chức và số liệu thống kê. – Các thay đổi về cấu trúc tổ chức hoặc thành viên mới (nếu có) được cập nhật vào hệ thống.

Use-case Name (tiếp)	**UC04. Quản lý tổ chức**
Normal Flow	1. Người dùng yêu cầu xem danh sách tổ chức. 2. Hệ thống tải và hiển thị dữ liệu dưới dạng cây phân cấp (Tree View) kèm theo các số liệu thống kê tổng quan. 3. Người dùng tương tác mở rộng/thu gọn các nhánh của cây tổ chức. 4. Người dùng chọn một đơn vị tổ chức cụ thể. 5. **(Include)** Hệ thống thực hiện quy trình *"Chuyển sang trang chi tiết"* để hiển thị thông tin đầy đủ của tổ chức đó.
Alternative Flow	Tại bước 3 (Chức năng dành cho Quản trị viên - Thêm thành viên): • **3a.** Quản trị viên chọn chức năng "Thêm thành viên" vào tổ chức. • **3b.** Hệ thống cung cấp 2 tùy chọn: – **(Extend 1) Thêm thủ công:** Quản trị viên nhập thông tin từng thành viên vào form. – **(Extend 2) Thêm bằng file:** Quản trị viên chọn tải lên danh sách từ file Excel/CSV. – *Trong trường hợp chọn Thêm bằng file:* **(Include)** Hệ thống cung cấp chức năng *"Lấy định dạng dữ liệu mẫu"* để người dùng tải về template trước khi nhập liệu. Tại bước 3 (Chức năng quản lý cấu trúc): • **3c.** Quản trị viên thực hiện các thao tác CRUD: Tạo mới, Cập nhật thông tin hoặc Xóa một tổ chức khỏi cây.
Exception	• **Lỗi định dạng file:** Khi thực hiện "Thêm thành viên bằng file", nếu file không đúng mẫu, hệ thống báo lỗi và yêu cầu tải lại template chuẩn. • **Lỗi dữ liệu:** Khi "Tạo/Sửa tổ chức", nếu tên tổ chức bị trùng hoặc thiếu thông tin bắt buộc, hệ thống hiển thị thông báo lỗi tương ứng.

Table 7: *Bảng mô tả Use Case UC04 - Quản lý tổ chức*

4.1.5 Quản lý phân quyền (RBAC)

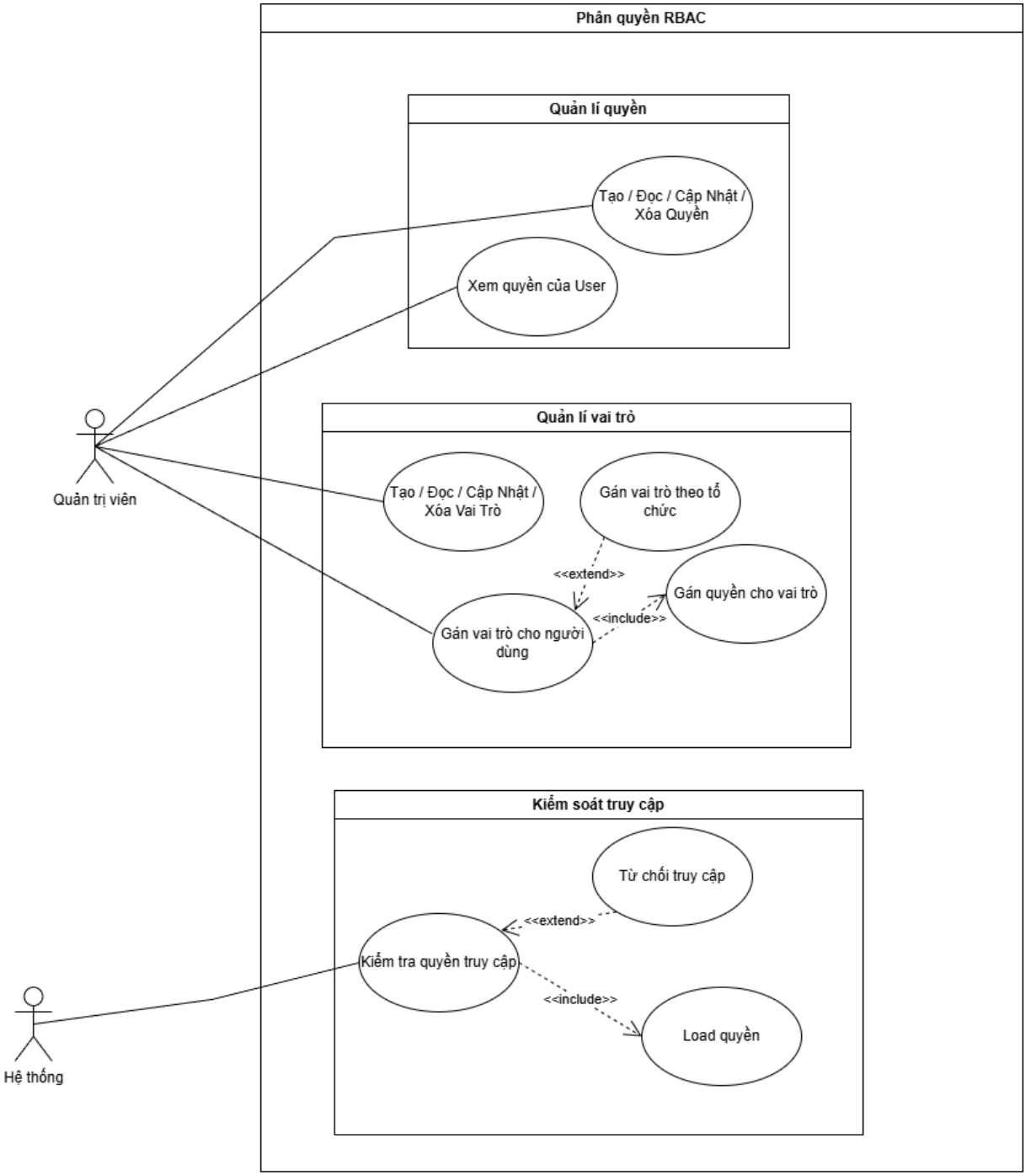


Figure 5: *UC05 - Gán vai trò và Phân quyền*

Use-case Name	**UC05. Gán vai trò và Phân quyền**
Actors	– Quản trị viên (Admin) – Hệ thống (System)
Description	Quản trị viên thực hiện cấp quyền cho người dùng bằng cách gán các vai trò (Roles) cụ thể. Hệ thống sẽ tự động liên kết các quyền hạn (Permissions) tương ứng với vai trò đó. Ngoài ra, hệ thống hỗ trợ việc gán vai trò trong phạm vi một tổ chức/đơn vị cụ thể nếu cần thiết.

Use-case Name (tiếp)	**UC05. Gán vai trò và Phân quyền**
Trigger	Quản trị viên chọn một tài khoản người dùng và nhấn nút truy cập vào màn hình ”Chọn vai trò vai trò”.
Pre-Conditions	– Quản trị viên đã đăng nhập và có quyền quản lý hệ thống. – Các Quyền (Permissions) đã được định nghĩa trong cơ sở dữ liệu. – Tài khoản người dùng cần gán đã tồn tại.
Post-Conditions	– Người dùng được cập nhật vai trò mới và có các quyền truy cập tương ứng. – Thông tin phân quyền được lưu vào cơ sở dữ liệu.
Normal Flow	1. Quản trị viên tìm kiếm và chọn người dùng cần phân quyền. 2. Hệ thống hiển thị danh sách các Vai trò khả dụng (Ví dụ: Admin, Mod, User...). 3. Quản trị viên chọn vai trò muốn gán cho người dùng. 4. **(Include)** Hệ thống thực hiện quy trình *”Gán quyền cho vai trò”*: Tự động ánh xạ các quyền hạn (Create/Read/Update/Delete) đi kèm với vai trò đã chọn. 5. Quản trị viên xác nhận lưu thay đổi. 6. Hệ thống cập nhật dữ liệu và thông báo thành công.
Alternative Flow	Tại bước 3 (Trường hợp gán theo phạm vi tổ chức): • **3a.** Nếu vai trò yêu cầu phạm vi cụ thể (ví dụ: Bí thư của tổ chức A), Quản trị viên chọn thêm thông tin Tổ chức/Đơn vị. • **3b. (Extend)** Hệ thống kích hoạt quy trình *”Gán vai trò theo tổ chức”* để kiểm tra và giới hạn phạm vi quyền hạn trong tổ chức đó. • **3c.** Tiếp tục sang bước 4.
Exception	Tại bước 6: • **6a.** Hệ thống gặp lỗi kết nối cơ sở dữ liệu. Hiển thị thông báo: *”Lưu thất bại, vui lòng thử lại sau.”* • **6b.** Tài khoản người dùng đang bị khóa hoặc không hợp lệ. Hệ thống từ chối phân quyền và báo lỗi.

Table 8: *Bảng mô tả Use Case UC05 - Gán vai trò và Phân quyền*

4.1.6 Quản lý đóng phí

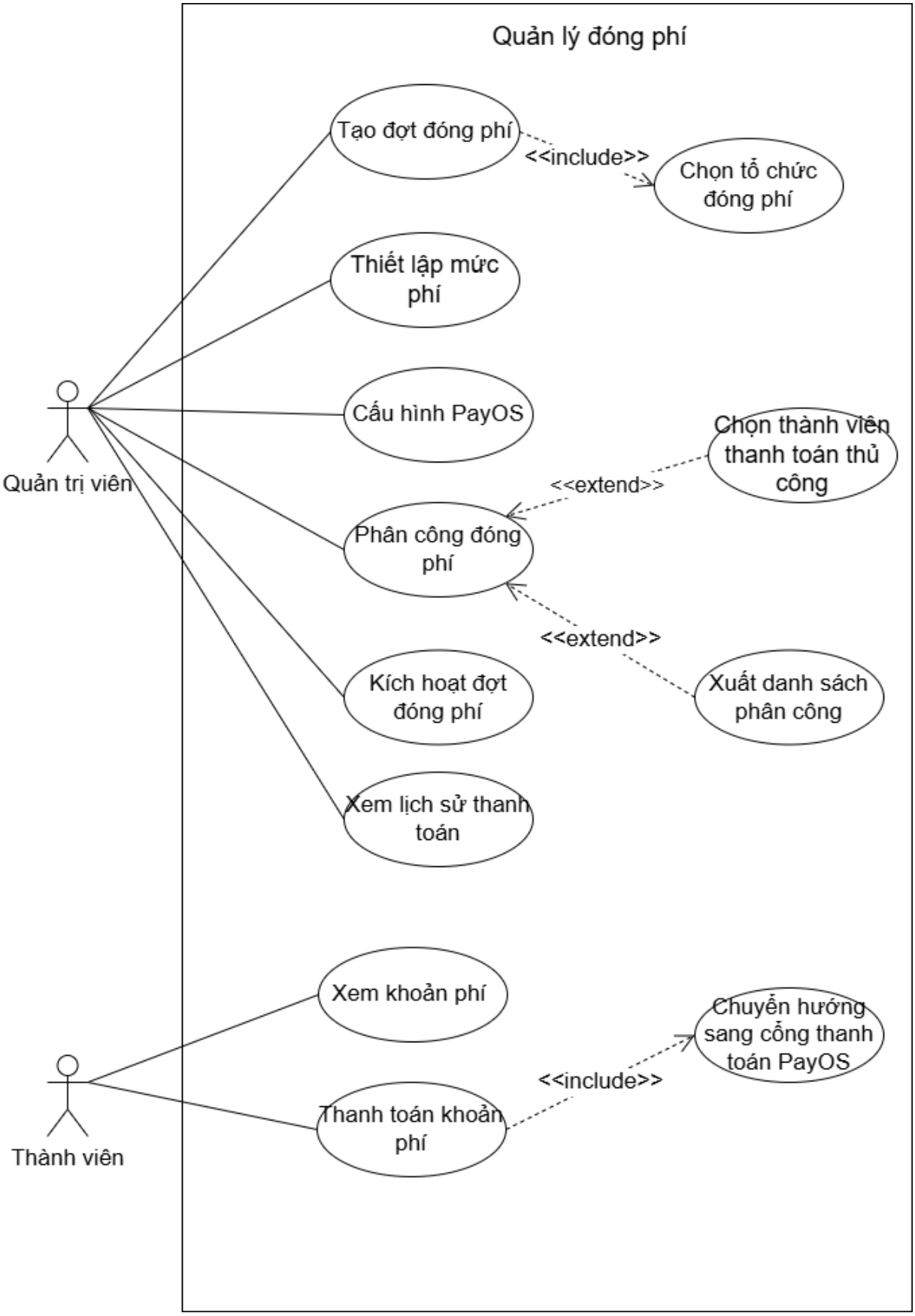


Figure 6: *UC04 - Quản lý đóng phí*

Use Case Name	**UC04. Quản lý đóng phí**
Actors	Quản trị viên, Thành viên
Description	Quản trị viên tạo một đợt đóng phí mới, thiết lập các thông tin cấu hình cần thiết sau đó kích hoạt đợt đóng phí để thành viên có thể bắt đầu thực hiện thanh toán.
Trigger	Quản trị viên muốn tạo và khởi động một đợt thu phí mới.
Pre-Conditions	• Quản trị viên đã đăng nhập vào hệ thống. • Hệ thống hiện đã có danh sách tổ chức và thành viên để tạo đợt đóng phí và phân công.
Post-Conditions	• Đợt đóng phí mới đã được tạo và lưu lại. • Đợt đóng phí được chuyển từ trạng thái *Nháp* sang *Đang hoạt động*. • Thông tin đợt đóng phí hiển thị trên trang của các thành viên được phân công. • Thành viên thực hiện thanh toán và hệ thống cập nhật trạng thái thanh toán.
Normal Flow	1. Quản trị viên chọn chức năng tạo đợt đóng phí mới. 2. Hệ thống hiển thị giao diện tạo đợt đóng phí, trạng thái mặc định là *Nháp*. 3. Quản trị viên điền các thông tin của đợt đóng phí và bấm nút *Lưu*. 4. Hệ thống lưu đợt đóng phí với trạng thái bản nháp và chuyển hướng đến trang chi tiết đợt đóng phí vừa tạo. 5. Quản trị viên truy cập tab *Mức phí*, nhập ít nhất một mức phí và lưu lại. 6. Quản trị viên truy cập tab *Phân công thành viên*, chọn và phân công ít nhất một thành viên và lưu lại. 7. Quản trị viên truy cập tab *Cấu hình PayOS*, thiết lập API key của cổng thanh toán PayOS và lưu lại. 8. Quản trị viên nhấn nút *Kích hoạt đợt đóng phí*. 9. Hệ thống chuyển trạng thái đợt đóng phí từ *Nháp* sang *Đang hoạt động* và hiển thị thông tin đóng phí trên trang *Đóng phí của tôi* của thành viên. 10. Thành viên truy cập trang *Đóng phí của tôi* và xem các khoản phí cần thanh toán. 11. Thành viên bấm nút *Thanh toán ngay*, hệ thống chuyển sang cổng thanh toán trực tuyến. 12. Sau khi thanh toán, hệ thống chuyển về trang kết quả thanh toán và cập nhật trạng thái thanh toán.

Use Case Name (tiếp)	UC04. Quản lý đóng phí
Alternative Flow	Ở bước 11: • **11a.** Thành viên không thanh toán trực tuyến mà đến văn phòng Đoàn để đóng tiền mặt. • **11b.** Tại trang phân công đóng phí, quản trị viên bấm chọn *Đóng phí thủ công* tại tên thành viên trong danh sách. • **11c.** Hệ thống cập nhật trạng thái thanh toán của thành viên là *Đã đóng phí (thủ công)*.
Exception	Ở bước 8: • **8a.** Nếu đợt đóng phí thiếu một trong hai cấu hình bắt buộc (*Mức phí* hoặc *Phân công*), nút *Kích hoạt* sẽ không hiển thị. • **8b.** Hệ thống hiển thị thông báo yêu cầu cấu hình đầy đủ mức phí và phân công thành viên trước khi kích hoạt.

Table 9: *Bảng mô tả Use Case UC04 - Quản lý đóng phí*

4.1.7 Quản lý hoạt động công tác xã hội

Module Công tác Xã hội cung cấp các chức năng quản lý toàn diện hoạt động CTXH của sinh viên, bao gồm 6 chức năng chính:

1. **Quản lý danh mục:** Tạo, cập nhật và phân loại các danh mục hoạt động CTXH.
2. **Quản lý hoạt động:** Tạo, chỉnh sửa thông tin hoạt động CTXH (tên, thời gian, địa điểm, số ngày CTXH).
3. **Quản lý người tham gia:** Thêm sinh viên (thủ công/Excel), chấm điểm ngày CTXH (thủ công/Excel), xuất danh sách.
4. **Cấu hình giới hạn:** Thiết lập số ngày CTXH tối đa theo tổ chức, tự động áp dụng cho tổ chức con.
5. **Nhập dữ liệu từ hệ thống cũ:** Nhập số ngày CTXH từ hệ thống cũ bằng file Excel.
6. **Xem báo cáo cá nhân:** Sinh viên xem tổng số ngày đã tích lũy, trạng thái đạt/chưa đạt yêu cầu 15 ngày.

Trong đó, 3 chức năng đầu (Quản lý danh mục, hoạt động, người tham gia) có use case diagram chi tiết riêng do độ phức tạp cao. 3 chức năng còn lại (Cấu hình giới hạn, Nhập dữ liệu cũ, Xem báo cáo cá nhân) được thể hiện trong diagram tổng quát và chỉ mô tả bằng bảng specification do tính đơn giản.

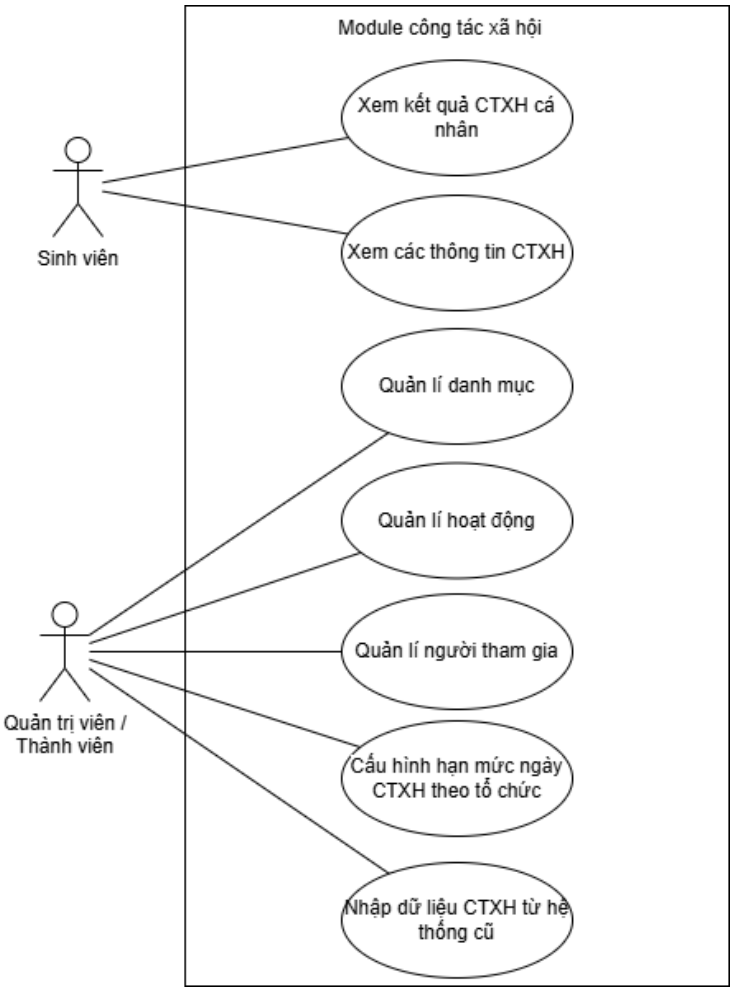


Figure 7: *UC07 - Quản lý hoạt động Công tác Xã hội (bao gồm UC7.4 - Cấu hình giới hạn, UC7.5 - Xem báo cáo cá nhân, UC7.6 - Nhập dữ liệu hệ thống cũ)*

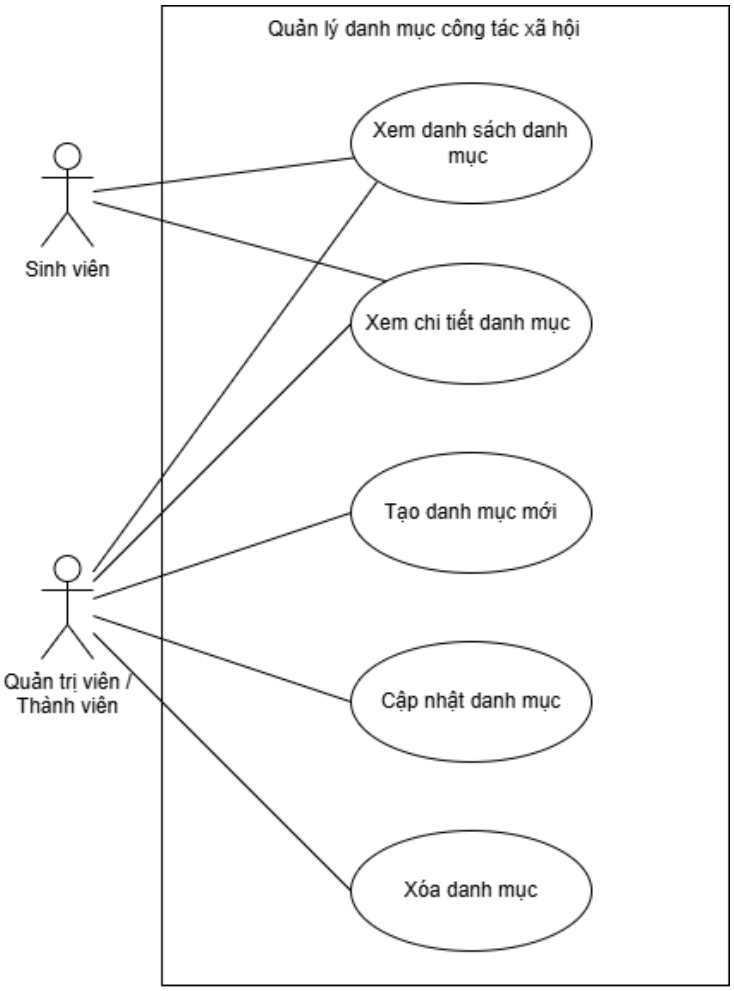


Figure 8: *UC7.1 - Quản lý danh mục công tác xã hội*

Use Case Name	**UC7.1. Quản lý danh mục công tác xã hội**
Actors	Quản trị viên, Người dùng
Description	Use case này cho phép Quản trị viên/Người dùng quản lý danh mục CTXH để phân loại các hoạt động công tác xã hội. Bao gồm: • Tạo danh mục mới • Xem danh sách danh mục • Xem chi tiết danh mục và danh sách hoạt động thuộc danh mục • Cập nhật thông tin danh mục • Xóa danh mục
Trigger	Quản trị viên/Người dùng muốn tạo, xem, chỉnh sửa hoặc xóa danh mục để phân loại hoạt động CTXH.
Pre-Conditions	• Người dùng đã đăng nhập vào hệ thống • Có các quyền xem và quản lí danh mục công tác xã hội
Post-Conditions	• Danh mục được tạo mới, cập nhật hoặc xóa (mềm) trong database • Có thể xem danh sách hoạt động thuộc danh mục
Normal Flow	**Tạo danh mục mới:** 1. Người dùng click ”Tạo danh mục mới”, nhập tên và mô tả 2. Hệ thống validate tên không trống, chưa tồn tại 3. Tạo danh mục và hiển thị thông báo thành công **Xem chi tiết:** 1. Người dùng click vào danh mục 2. Hệ thống hiển thị thông tin danh mục và danh sách hoạt động **Cập nhật:** 1. Người dùng click ”Sửa”, chỉnh sửa thông tin 2. Hệ thống validate và cập nhật danh mục **Xóa:** 1. Người dùng click ”Xóa” 2. Hệ thống kiểm tra: Nếu có hoạt động thì không cho xóa 3. Nếu không có hoạt động: Hiển thị xác nhận và xóa mềm

Alternative Flow	**Tạo mới:** • **4a.** Tên trống: Hiển thị ”Tên danh mục không được để trống”, quay lại bước 3 • **4b.** Tên đã tồn tại: Hiển thị ”Danh mục đã tồn tại”, quay lại bước 3 **Xem chi tiết:** • **2a.** Danh mục không có hoạt động nào: Hiển thị ”Chưa có hoạt động nào trong danh mục này” **Cập nhật:** • **2a.** Danh mục không tồn tại: Hiển thị lỗi không tìm thấy, quay về danh sách • **5a.** Tên mới đã tồn tại: Hiển thị ”Danh mục đã tồn tại”, quay lại bước 3 **Xóa:** • **2a.** Danh mục có hoạt động: – Hiển thị lỗi ”Không thể xóa danh mục đã có hoạt động. Vui lòng xóa tất cả hoạt động trước.” – Không cho phép xóa • **4a.** Người dùng click ”Hủy”: Đóng hộp thoại, không xóa • **5a.** Không có quyền delete: Hiển thị ”Bạn không có quyền xóa danh mục”
Exception	• Lỗi database: Hiển thị ”Không thể lưu danh mục, vui lòng thử lại” • Mất kết nối: Hiển thị ”Lỗi kết nối, vui lòng thử lại sau”

Table 10: *Bảng mô tả Use Case UC7.1 - Quản lý danh mục công tác xã hội*

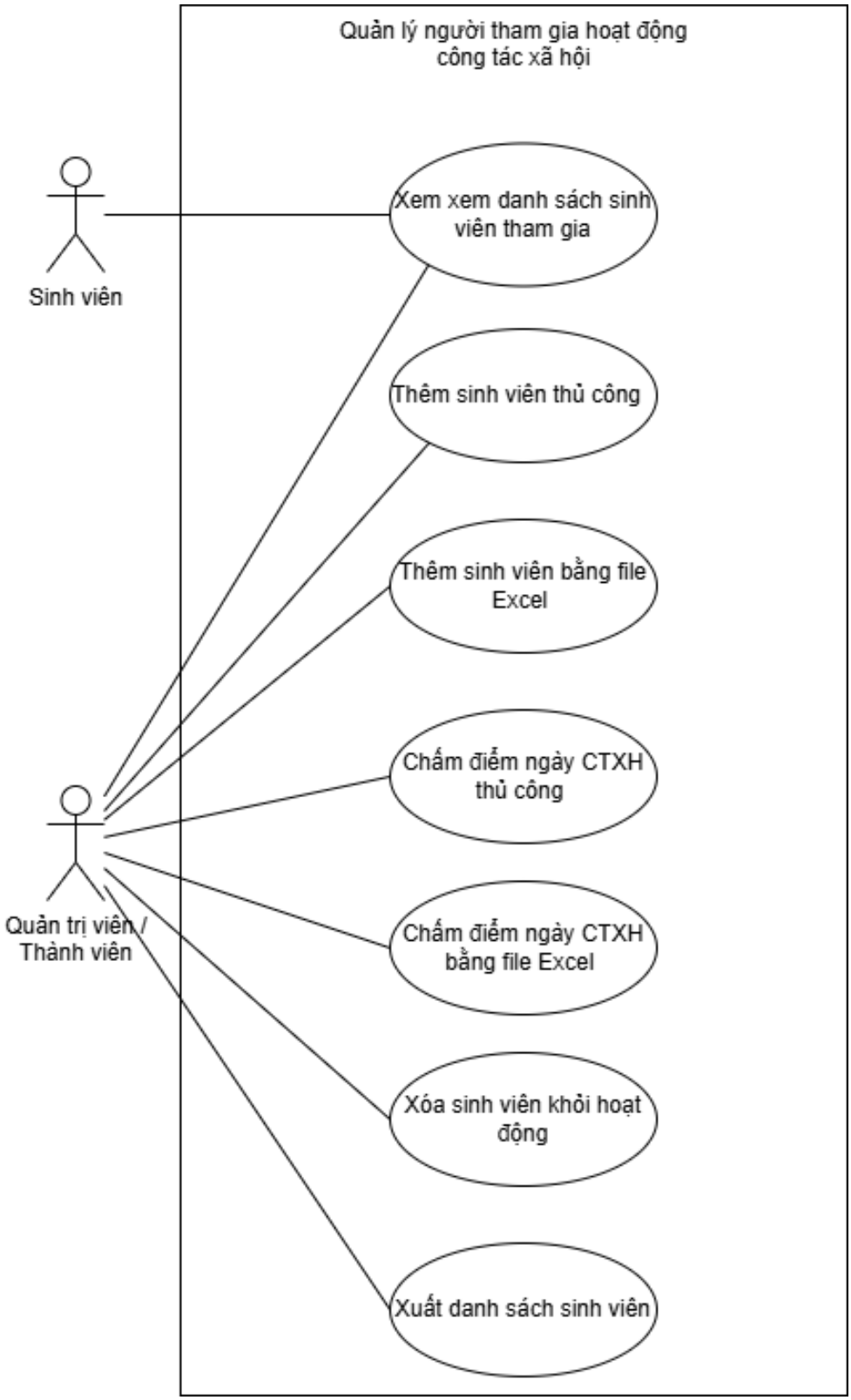


Figure 9: *UC7.2 - Quản lý người tham gia hoạt động*

Use Case Name	UC7.2. Quản lý người tham gia hoạt động
Actors	Quản trị viên, Người dùng
Description	Use case này cho phép quản lý danh sách sinh viên tham gia hoạt động CTXH, bao gồm: • Xem danh sách sinh viên • Thêm sinh viên thủ công (nhập mã số) • Thêm sinh viên bằng file Excel • Chấm điểm CTXH thủ công • Chấm điểm bằng file Excel • Xóa sinh viên khỏi hoạt động • Xuất danh sách sinh viên
Trigger	Quản trị viên/Người dùng muốn thêm sinh viên vào hoạt động hoặc chấm điểm sau khi hoạt động kết thúc.
Pre-Conditions	• Người dùng đã đăng nhập • Có quyền quản lý người tham gia hoạt động công tác xã hội • Hoạt động tồn tại với trạng thái ”Đang hoạt động’
Post-Conditions	• Bản ghi tham gia hoạt động được tạo hoặc cập nhật trong hệ thống • Số ngày CTXH được cộng vào tổng số ngày CTXH của sinh viên • Sinh viên thấy kết quả trong báo cáo cá nhân
Normal Flow	**Thêm sinh viên thủ công:** 1. Người dùng nhập mã số sinh viên 2. Hệ thống validate: sinh viên tồn tại, chưa tham gia 3. Tạo bản ghi tham gia với trạng thái ”Đã đăng ký” **Chấm điểm thủ công:** 1. Người dùng nhập số ngày (bội 0.5) 2. Hệ thống validate: số ngày hợp lệ, không vượt giới hạn 3. Cập nhật số ngày và cộng vào tổng CTXH của sinh viên **Chấm điểm Excel:** 1. Người dùng tải mẫu và tải lên file Excel 2. Hệ thống parse file (chunk 500 rows), validate từng row 3. Hiển thị preview: hợp lệ/lỗi 4. Cập nhật hàng loạt sau khi xác nhận

Alternative Flow	**Thêm sinh viên:** • **5a.** Không tìm thấy sinh viên: Hiển thị ”Không tìm thấy sinh viên”, quay lại bước 4 • **5b.** Sinh viên đã tham gia: Hiển thị ”Sinh viên đã tham gia”, quay lại bước 4 **Chấm điểm thủ công:** • **4a.** Số ngày không hợp lệ: Hiển thị lỗi cụ thể, quay lại bước 3 • **4b.** Số ngày vượt giới hạn: Hiển thị ”Vượt quá số ngày tối đa của hoạt động”, quay lại bước 3 **Chấm điểm Excel:** • **4a.** File sai định dạng: Hiển thị ”Vui lòng sử dụng file .xlsx”, quay lại bước 3 • **5a.** Có row lỗi: Đánh dấu lỗi trong preview, chỉ update row hợp lệ
Exception	• Hoạt động không ở trạng thái đang hoạt động: ”Chỉ thêm sinh viên vào hoạt động đang hoạt động” • File Excel > 10MB: ”File quá lớn, tối đa 10MB” • Lỗi database batch update: Rollback transaction, hiển thị lỗi

Table 11: *Bảng mô tả Use Case UC7.2 - Quản lý người tham gia hoạt động*

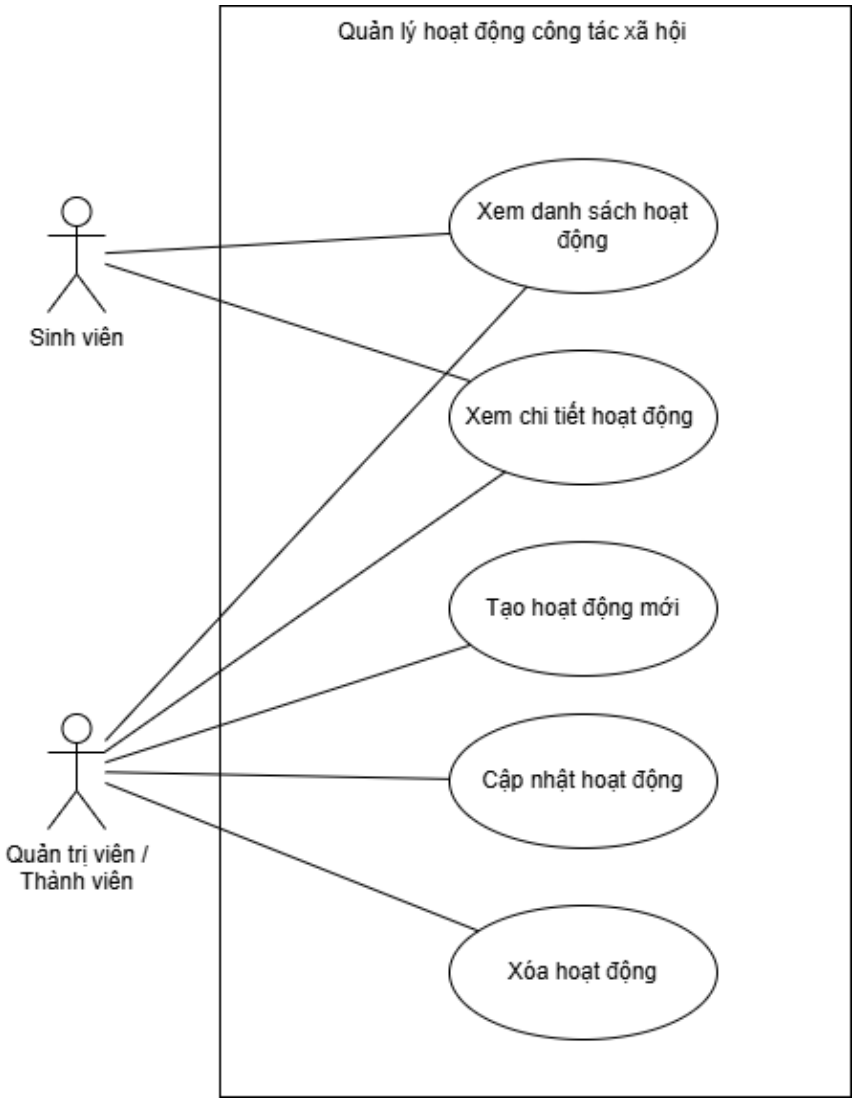


Figure 10: *UC7.3 - Quản lý hoạt động công tác xã hội*

Use Case Name	UC7.3. Quản lý hoạt động công tác xã hội
Actors	Quản trị viên, Người dùng
Description	Use case này cho phép quản lý hoạt động CTXH trong hệ thống: • Tạo hoạt động mới (Nháp/Đang hoạt động) • Xem danh sách hoạt động • Cập nhật thông tin hoạt động • Xóa hoạt động
Trigger	Quản trị viên/Người dùng muốn tạo hoạt động CTXH mới hoặc chỉnh sửa hoạt động hiện có.
Pre-Conditions	• Người dùng đã đăng nhập • Có quyền tạo mới hoặc cập nhật hoạt động công tác xã hội • Có ít nhất 1 danh mục trong hệ thống
Post-Conditions	• Hoạt động được tạo mới hoặc cập nhật • Trạng thái: Nháp hoặc Đang hoạt động tùy theo cấu hình • Hoạt động hiển thị trong danh sách
Normal Flow	**Tạo hoạt động mới:** 1. Người dùng nhập thông tin: tên, danh mục, địa điểm, thời gian, số ngày, trạng thái 2. Hệ thống validate: tên không trống, ngày kết thúc sau ngày bắt đầu, số ngày là bội 0.5 3. Kiểm tra cấu hình giới hạn: nếu vượt thì cảnh báo nhưng vẫn cho tạo 4. Lưu hoạt động vào cơ sở dữ liệu **Cập nhật:** 1. Người dùng chỉnh sửa thông tin hoạt động 2. Hệ thống validate (tương tự tạo mới) và cập nhật
Alternative Flow	**Tạo mới:** • **4a.** Tên trống: Hiển thị ”Tên không được để trống”, quay lại bước 3 • **4b.** Tên trùng: Hiển thị ”Hoạt động đã tồn tại trong danh mục”, quay lại bước 3 • **4c.** Không có quyền tổ chức: Hiển thị ”Không có quyền”, quay lại bước 3 • **4d.** Thời gian không hợp lệ: Hiển thị lỗi, quay lại bước 3 • **4e.** Số ngày không hợp lệ: ”Số ngày phải là bội của 0.5”, quay lại bước 3 • **4f.** Trạng thái đang hoạt động nhưng thiếu thông tin: ”Cần đầy đủ địa điểm, thời gian, số ngày”, quay lại bước 3 **Cập nhật:** • **2a.** Hoạt động không tồn tại: Hiển thị lỗi không tìm thấy, quay về danh sách • **5a.** Validate lỗi: Tương tự tạo mới
Exception	• Danh mục không tồn tại: Hiển thị lỗi không tìm thấy danh mục • Lỗi database: Hiển thị ”Không thể lưu hoạt động, vui lòng thử lại”

Table 12: *Bảng mô tả Use Case UC7.3 - Quản lý hoạt động công tác xã hội*

Use Case Name	UC7.4. Quản lý hạn mức ngày CTXH theo chức
Actors	Quản trị viên
Description	Quản trị viên thiết lập số ngày CTXH tối đa được phép cho từng tổ chức. Cấu hình tự động áp dụng cho tất cả tổ chức con chưa có cấu hình hoặc đang hoạt động.
Trigger	Quản trị viên muốn giới hạn số ngày một hoạt động có thể cấp cho sinh viên theo từng tổ chức.
Pre-Conditions	• Người dùng đã đăng nhập • Có quyền cấu hình giới hạn số ngày CTXH • Tổ chức tồn tại trong hệ thống
Post-Conditions	• Cấu hình giới hạn được tạo với thông tin tổ chức, số ngày tối đa và trạng thái • Tổ chức con tự động được cập nhật cấu hình
Normal Flow	1. Người dùng truy cập ”Cấu hình giới hạn”, click ”Tạo cấu hình” 2. Chọn tổ chức, nhập số ngày tối đa, chọn trạng thái 3. Hệ thống validate: số ngày > 0, tổ chức tồn tại 4. Tạo cấu hình cho tổ chức cha 5. Tự động áp dụng cho tổ chức con (cấp 1, 2): tạo mới hoặc cập nhật nếu đang hoạt động 6. Hiển thị ”Đã tạo cấu hình và áp dụng cho X tổ chức con”
Alternative Flow	• **7a.** Số ngày ≤ 0: Hiển thị ”Số ngày phải > 0”, quay lại bước 5 • **7b.** Tổ chức đã có cấu hình đang hoạt động: Hiển thị ”Tổ chức đã có cấu hình”
Exception	• Tổ chức không tồn tại: Hiển thị lỗi không tìm thấy tổ chức

Table 13: *Bảng mô tả Use Case UC7.4 - Quản lý hạn mức ngày CTXH theo chức*

Use Case Name	**UC7.5. Xem báo cáo ngày công tác xã hội cá nhân**
Actors	Sinh viên
Description	Sinh viên xem báo cáo CTXH cá nhân bao gồm tổng số ngày đã tích lũy, trạng thái đạt/chưa đạt, và danh sách chi tiết các hoạt động đã tham gia. Hệ thống tính từ cả ngày mới (hệ thống hiện tại) và ngày cũ (nhập từ hệ thống cũ).
Trigger	Sinh viên muốn kiểm tra tiến độ CTXH, xem đã đạt 15 ngày chưa.
Pre-Conditions	• Người dùng đã đăng nhập • Tài khoản sinh viên có mã số sinh viên trong hệ thống
Post-Conditions	Báo cáo CTXH cá nhân hiển thị đầy đủ thông tin
Normal Flow	1. Sinh viên click ”Báo cáo CTXH của tôi” 2. Hệ thống lấy mã số, truy vấn: số ngày mới + số ngày cũ, danh sách hoạt động đã tham gia 3. Tính tổng số ngày, kiểm tra đạt yêu cầu (>= 15 ngày) 4. Hiển thị báo cáo: thông tin, tổng ngày, trạng thái đạt/chưa đạt, danh sách hoạt động
Alternative Flow	• **2a.** Sinh viên chưa tham gia hoạt động nào: – Hiển thị ”Bạn chưa tham gia hoạt động CTXH nào” – Hiển thị button ”Xem danh sách hoạt động” • **5a.** Tổng ngày $\geq$ 15: – Hiển thị badge ”Da hoan thanh CTXH”
Exception	Không tìm thấy thông tin sinh viên: Hiển thị lỗi tài khoản không hợp lệ

Table 14: *Bảng mô tả Use Case UC7.5 - Xem báo cáo ngày công tác xã hội cá nhân*

Use Case Name	UC7.6. Nhập dữ liệu ngày CTXH từ hệ thống cũ
Actors	Quản trị viên
Description	Quản trị viên tải lên file Excel chứa mã số sinh viên và số ngày CTXH từ hệ thống cũ. Hệ thống phân tích, kiểm tra và cập nhật số ngày CTXH từ hệ thống cũ vào hồ sơ sinh viên. Sử dụng khi chuyển đổi dữ liệu từ hệ thống cũ sang hệ thống mới.
Trigger	Quản trị viên muốn nhập dữ liệu CTXH từ hệ thống cũ khi lần đầu triển khai.
Pre-Conditions	• Người dùng đã đăng nhập • Có quyền nhập dữ liệu từ hệ thống cũ • File Excel định dạng: Mã số sinh viên | Số ngày
Post-Conditions	• Bản ghi sinh viên được cập nhật số ngày CTXH từ hệ thống cũ • Số ngày CTXH từ hệ thống mới KHÔNG thay đổi • Không tạo bản ghi tham gia hoạt động (chỉ lưu tổng số ngày)
Normal Flow	1. Người dùng vào ”Nhập dữ liệu hệ thống cũ”, click ”Nhập” 2. Tải mẫu Excel (Header: Mã số | Số ngày), điền dữ liệu và tải lên 3. Hệ thống parse Excel, validate từng row: MSSV tồn tại, số ngày >= 0 4. Hiển thị preview: hợp lệ/lỗi 5. Xác nhận: cập nhật hàng loạt số ngày CTXH từ hệ thống cũ vào hồ sơ sinh viên 6. Hiển thị ”Đã nhập X sinh viên”
Alternative Flow	• **6a.** File sai định dạng: ”Vui lòng sử dụng mẫu”, quay lại bước 5 • **7a.** Không tìm thấy sinh viên: Đánh dấu lỗi, tiếp tục row tiếp • **7b.** Số ngày < 0: Đánh dấu lỗi, tiếp tục row tiếp
Exception	Xử lý hàng loạt lỗi: Tự động thử lại, hiển thị ”Lỗi khi nhập dữ liệu”

Table 15: *Bảng mô tả Use Case UC7.6 - Nhập dữ liệu ngày CTXH từ hệ thống cũ*

4.1.8 Quản lý biểu mẫu

Module Biểu mẫu cung cấp các chức năng quản lý và sử dụng biểu mẫu động cho các hoạt động của đoàn hội, bao gồm 2 chức năng chính:

1. **Quản lý biểu mẫu:** Tải lên file Word template, cấu hình trường dữ liệu động, quản lý danh sách biểu mẫu.
2. **Điền thông tin vào biểu mẫu:** Sinh viên/Thành viên điền biểu mẫu, hệ thống tự động sinh file Word/PDF từ template.

Module hỗ trợ parse file Word template với các placeholder động, cho phép tạo biểu mẫu tùy chỉnh mà không cần code.

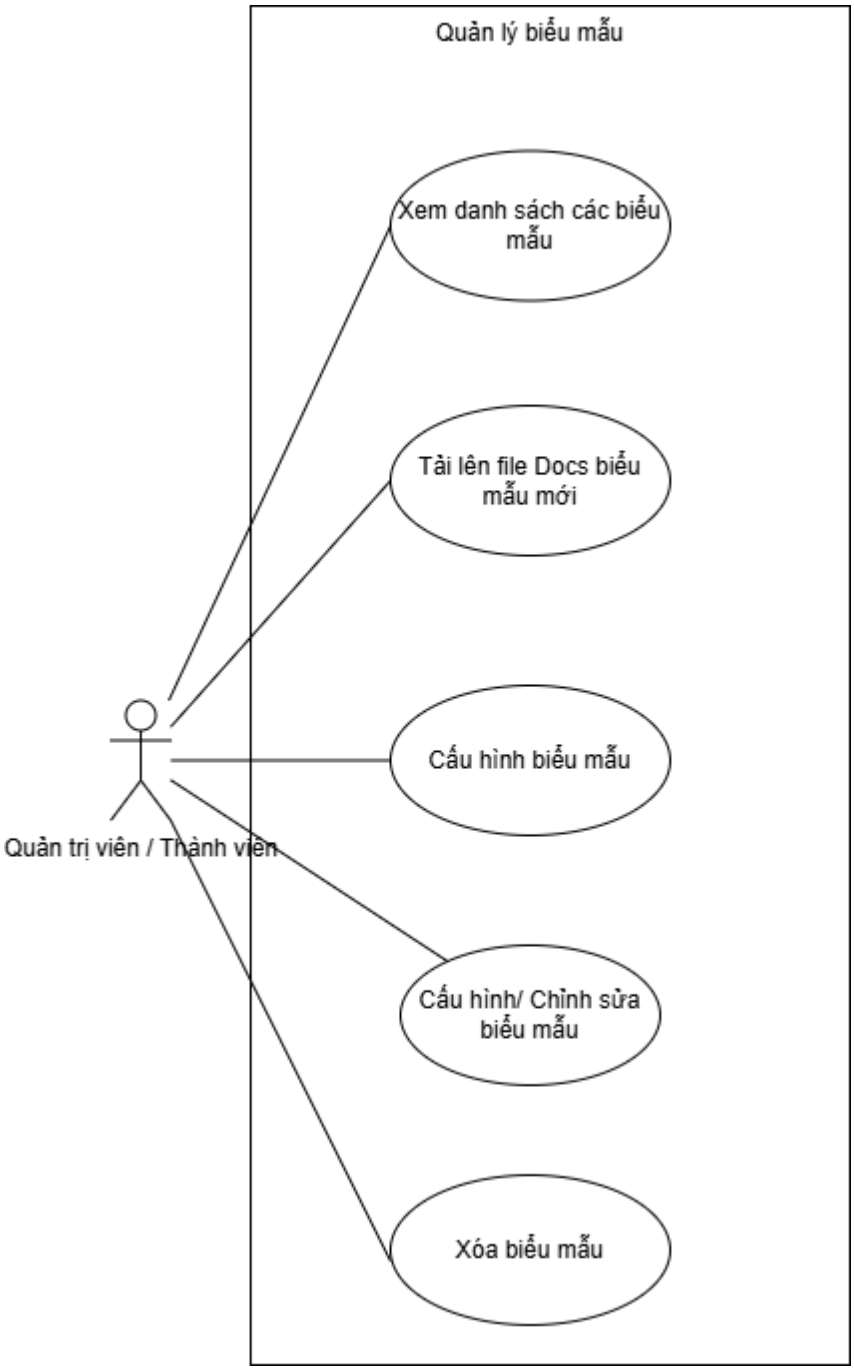


Figure 11: *UC8.1 - Quản lý biểu mẫu*

Use Case Name	**UC8.1. Quản lý biểu mẫu**
Actors	Quản trị viên, Người dùng
Description	Use case này cho phép quản lý biểu mẫu Word động trong hệ thống. Bao gồm: • Xem danh sách biểu mẫu • Tải lên file Word template mới • Cấu hình trường dữ liệu động (placeholder) • Chỉnh sửa cấu hình biểu mẫu • Xóa biểu mẫu
Trigger	Quản trị viên/Người dùng muốn tạo hoặc quản lý biểu mẫu cho các hoạt động.
Pre-Conditions	• Người dùng đã đăng nhập • Có quyền quản lý biểu mẫu
Post-Conditions	• Biểu mẫu được tạo mới, cấu hình hoặc xóa trong hệ thống • File template được lưu trên S3 • Cấu hình trường được lưu trong database
Normal Flow	**Tải lên biểu mẫu mới:** 1. Người dùng click ”Tạo biểu mẫu”, nhập tên và tải lên file Word 2. Hệ thống parse file, trích xuất các placeholder (tên_trường) 3. Hiển thị danh sách placeholder tìm thấy 4. Người dùng cấu hình từng placeholder: loại dữ liệu, nhãn hiển thị, bắt buộc/tùy chọn 5. Lưu template lên S3 và cấu hình vào database **Chỉnh sửa cấu hình:** 1. Người dùng click ”Cấu hình” trên biểu mẫu 2. Hệ thống hiển thị danh sách placeholder và cấu hình hiện tại 3. Người dùng chỉnh sửa cấu hình trường 4. Cập nhật cấu hình vào database **Xóa biểu mẫu:** 1. Người dùng click ”Xóa” 2. Hệ thống kiểm tra: nếu có bản ghi đã nộp thì không cho xóa 3. Nếu không có: xóa file trên S3 và xóa mềm trong database

Alternative Flow	**Tải lên:** • **2a.** File không phải .docx: Hiển thị ”Chỉ hỗ trợ file .docx”, quay lại bước 1 • **2b.** File > 5MB: Hiển thị ”File quá lớn, tối đa 5MB”, quay lại bước 1 • **3a.** Không tìm thấy placeholder: Hiển thị cảnh báo ”Template không có trường động”, vẫn cho lưu • **5a.** Lỗi tải lên S3: Hiển thị ”Lỗi khi lưu file, vui lòng thử lại” **Chỉnh sửa:** • **2a.** Biểu mẫu không tồn tại: Hiển thị lỗi không tìm thấy **Xóa:** • **2a.** Có bản ghi đã nộp: Hiển thị ”Không thể xóa biểu mẫu đã có người nộp”, không cho xóa • **3a.** Người dùng click ”Hủy”: Đóng hộp thoại, không xóa
Exception	• Lỗi phân tích file Word: Hiển thị ”File không đúng định dạng hoặc bị lỗi” • Lỗi kết nối S3: Hiển thị ”Lỗi lưu trữ, vui lòng thử lại sau” • Lỗi cơ sở dữ liệu: Hiển thị ”Không thể lưu cấu hình, vui lòng thử lại”

Table 16: *Bảng mô tả Use Case UC8.1 - Quản lý biểu mẫu*

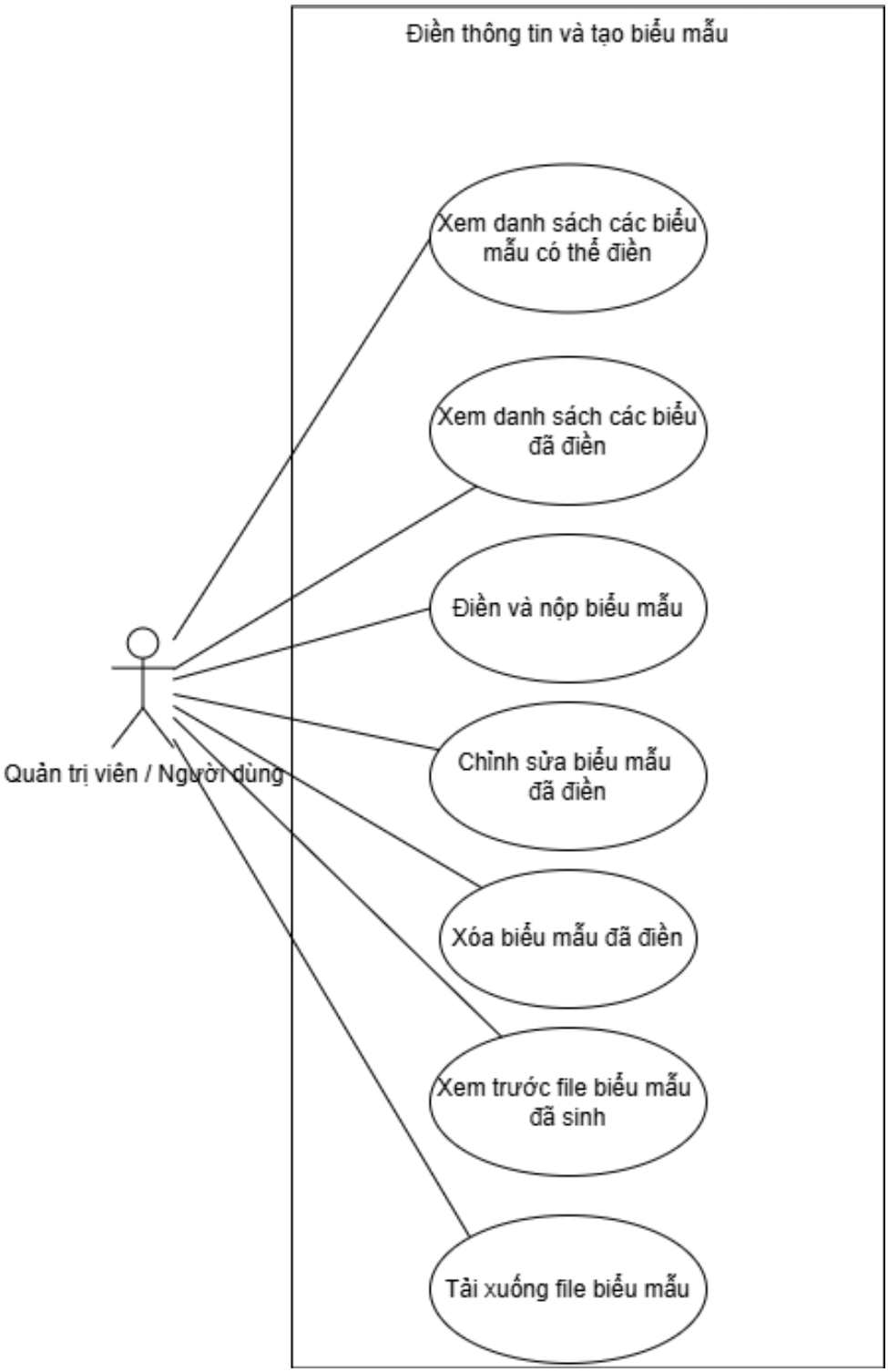


Figure 12: *UC8.2 - Điền thông tin vào biểu mẫu*

Use Case Name	UC8.2. Điền thông tin vào biểu mẫu
Actors	Quản trị viên, Người dùng, Thành viên
Description	Use case này cho phép người dùng điền thông tin vào biểu mẫu và hệ thống tự động sinh file Word/PDF. Bao gồm: • Xem danh sách biểu mẫu có thể điền • Xem danh sách biểu mẫu đã điền • Điền và nộp biểu mẫu mới • Chỉnh sửa biểu mẫu đã nộp • Xóa biểu mẫu đã nộp • Xem trước file đã sinh • Tải xuống file Word/PDF
Trigger	Người dùng muốn điền biểu mẫu hoặc quản lý các biểu mẫu đã nộp.
Pre-Conditions	• Người dùng đã đăng nhập • Có ít nhất 1 biểu mẫu trong hệ thống
Post-Conditions	• Bản ghi nộp được tạo/cập nhật trong database • File Word/PDF được sinh và lưu trên S3 • Người dùng có thể tải xuống file đã sinh
Normal Flow	**Điền biểu mẫu mới:** 1. Người dùng chọn biểu mẫu, click ”Điền biểu mẫu” 2. Hệ thống hiển thị form với các trường từ cấu hình 3. Người dùng điền thông tin và click ”Nộp” 4. Hệ thống validate, thay thế placeholder trong template, sinh file Word/PDF 5. Lưu bản ghi nộp và file lên S3, hiển thị ”Nộp thành công” **Chỉnh sửa bản ghi đã nộp:** 1. Người dùng click ”Chỉnh sửa” trên bản ghi đã nộp 2. Hệ thống hiển thị form với dữ liệu cũ 3. Người dùng sửa thông tin và click ”Cập nhật” 4. Sinh lại file mới, cập nhật bản ghi nộp **Xem trước/Tải xuống:** 1. Người dùng click ”Xem trước” hoặc ”Tải xuống” 2. Hệ thống lấy file từ S3 3. Xem trước trong browser hoặc tải xuống file

Alternative Flow	**Điền biểu mẫu:** • **4a.** Trường bắt buộc bỏ trống: Hiển thị lỗi validate, quay lại bước 3 • **4b.** Dữ liệu không đúng định dạng: Hiển thị lỗi cụ thể, quay lại bước 3 • **5a.** Lỗi sinh file: Hiển thị ”Không thể tạo file, vui lòng thử lại” **Chỉnh sửa:** • **1a.** Bản ghi nộp không tồn tại: Hiển thị lỗi không tìm thấy • **1b.** Không có quyền sửa (không phải người nộp): Hiển thị ”Bạn không có quyền” **Xem trước/Tải xuống:** • **2a.** File không tồn tại trên S3: Sinh lại file từ dữ liệu đã nộp • **2b.** Lỗi kết nối S3: Hiển thị ”Không thể tải file, vui lòng thử lại”
Exception	• Template bị xóa: Hiển thị ”Biểu mẫu không còn tồn tại” • Lỗi xử lý template: Hiển thị ”Lỗi xử lý biểu mẫu, vui lòng liên hệ quản trị viên” • Lỗi tải lên S3: Hiển thị ”Không thể lưu file, vui lòng thử lại”

Table 17: *Bảng mô tả Use Case UC8.2 - Điền thông tin vào biểu mẫu*

4.1.9 Hỏi đáp thông tin công tác xã hội qua Chatbot

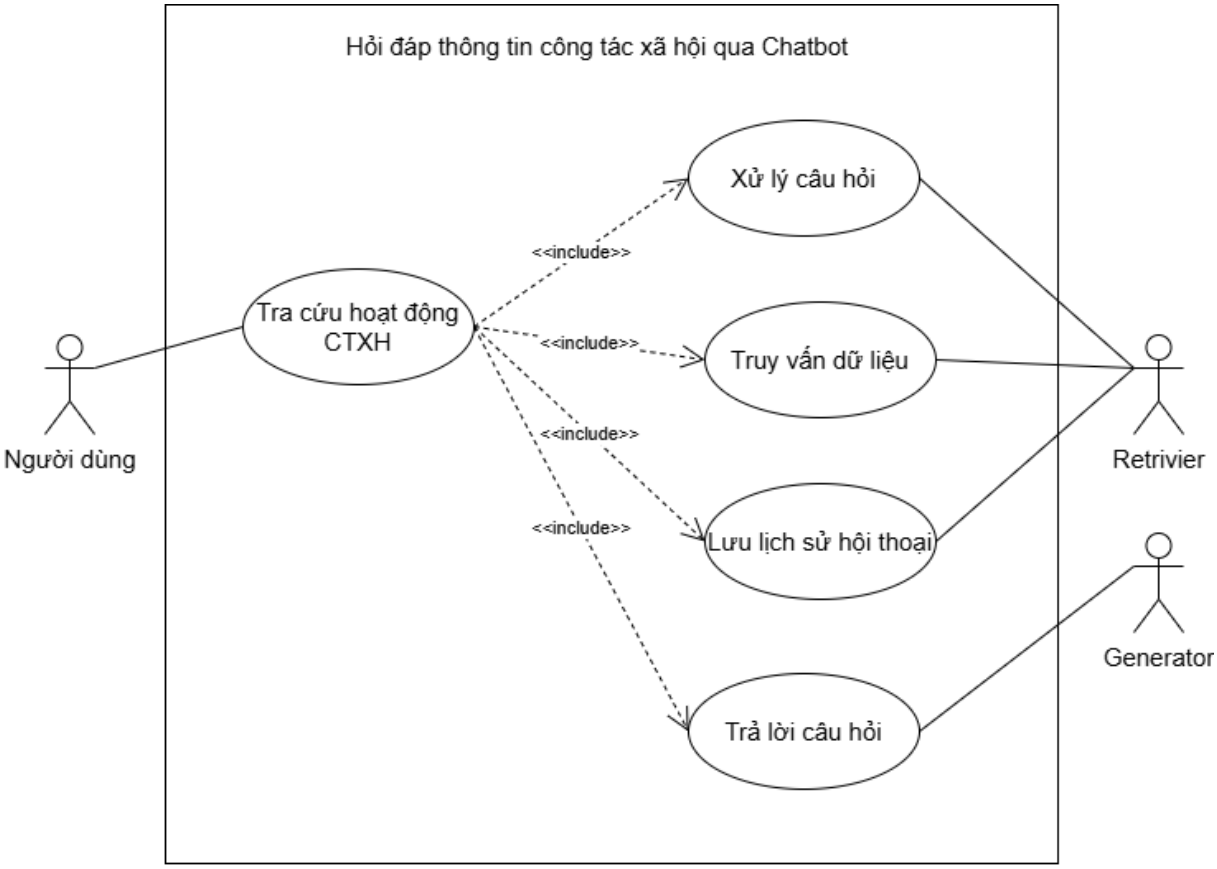


Figure 13: *UC09 - Tra cứu hoạt động Công tác xã hội qua Chatbot*

Use-case Name	UC09. Tra cứu hoạt động Công tác xã hội qua Chatbot
Actors	– Người dùng – Retriever (Hệ thống truy xuất) – Generator (Hệ thống tạo nội dung)
Description	Người dùng gửi câu hỏi về các hoạt động Công tác xã hội (CTXH). Hệ thống Chatbot thực hiện xử lý ngôn ngữ tự nhiên, truy xuất dữ liệu phù hợp từ cơ sở dữ liệu kiến thức và tạo ra câu trả lời tự nhiên để phản hồi cho người dùng, đồng thời lưu lại lịch sử cuộc trò chuyện nhằm duy trì ngữ cảnh hội thoại
Trigger	Người dùng nhập và gửi một câu hỏi liên quan đến hoạt động CTXH vào giao diện Chatbot.
Pre-Conditions	– Hệ thống Chatbot đang hoạt động – Cơ sở dữ liệu kiến thức về CTXH đã được chuẩn bị và sẵn sàng – Các dịch vụ AI LLM (sử dụng ở Retriever và Generator) đang ở trạng thái trực tuyến
Post-Conditions	– Người dùng nhận được câu trả lời phù hợp với câu hỏi đã gửi – Lịch sử hội thoại được ghi nhận vào hệ thống – Trạng thái phiên hội thoại được cập nhật để sẵn sàng cho câu hỏi tiếp theo
Normal Flow	1. Người dùng gửi câu hỏi về hoạt động CTXH. 2. Hệ thống thực hiện xử lý câu hỏi (phân tích ý định và nội dung). 3. Hệ thống phối hợp với *Retriever* để truy vấn dữ liệu từ cơ sở dữ liệu kiến thức. 4. Hệ thống lưu lịch sử hội thoại nhằm duy trì ngữ cảnh. 5. Hệ thống phối hợp với *Generator* để tổng hợp thông tin và tạo câu trả lời. 6. Hệ thống trả lời câu hỏi cho người dùng.
Alternative Flow	Ở bước 3: • **3a.** Hệ thống không truy xuất được dữ liệu phù hợp. Hệ thống thông báo không tìm thấy thông tin và gợi ý người dùng đặt câu hỏi khác. Ở bước 2: • **2a.** Hệ thống xác định câu hỏi không liên quan đến CTXH. Hệ thống từ chối trả lời và hướng dẫn người dùng về các chủ đề được hỗ trợ.
Exception	Ở bước 3 hoặc 5: • **3a. hoặc 5a.** Lỗi kết nối dịch vụ AI khiến hệ thống không thể kết nối tới Retriever hoặc Generator. Hệ thống hiển thị thông báo: *"Dịch vụ đang gặp sự cố, vui lòng thử lại sau."*

Table 18: *Bảng mô tả Use Case UC09 - Tra cứu hoạt động Công tác xã hội qua Chatbot*

4.2 Mô hình hóa Activity Diagram

4.2.1 Activity Diagram - Quản lý tổ chức

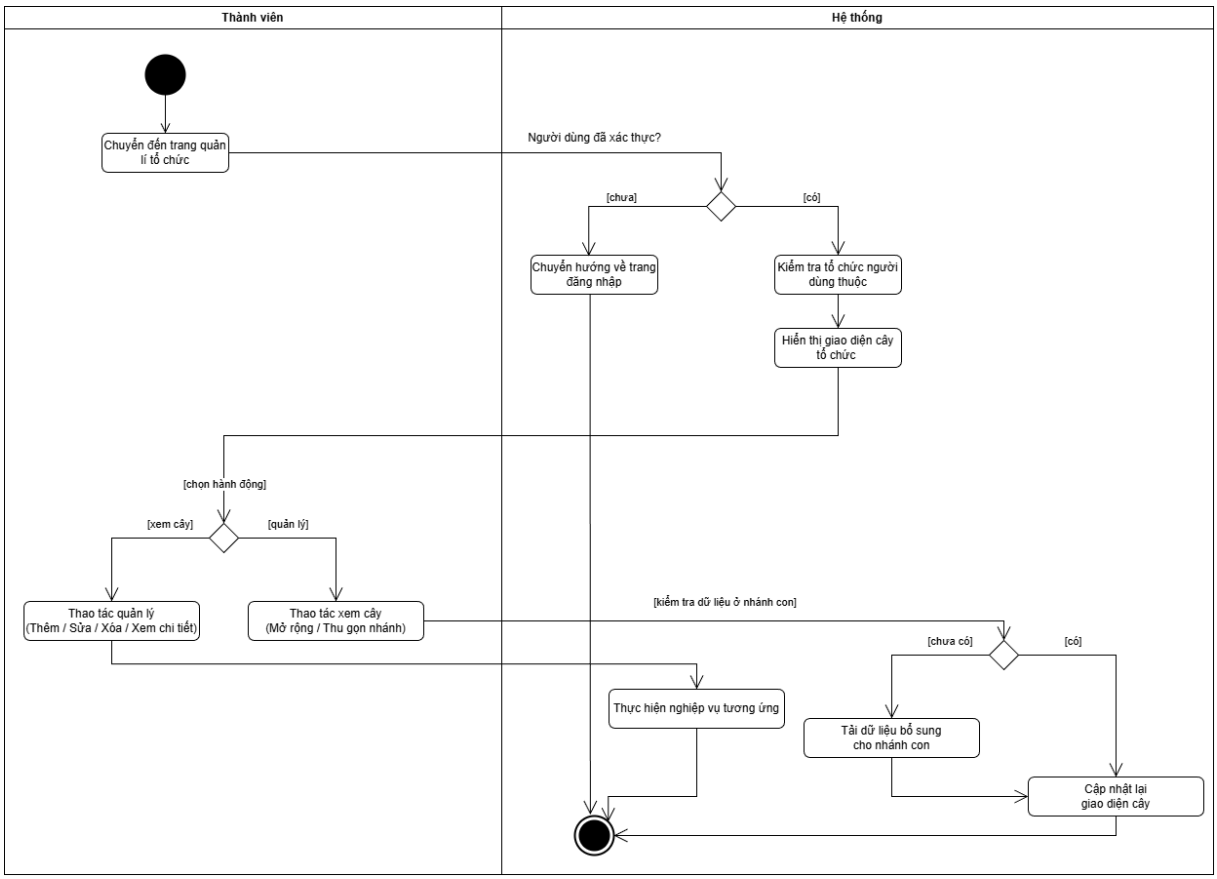


Figure 14: *Activity Diagram - Quản lý tổ chức*

Biểu đồ hoạt động tại Hình 14 mô tả luồng tương tác nghiệp vụ trên trang Quản lý tổ chức. Quy trình này không chỉ bao gồm các thao tác quản lý dữ liệu (CRUD) mà còn thể hiện chi tiết cơ chế tối ưu hóa trải nghiệm người dùng thông qua việc tải dữ liệu cây phân cấp theo nhu cầu. Quy trình được chia thành hai làn xử lý (*Swimlanes*) là **Thành viên** và **Hệ thống**, bao gồm các giai đoạn chính sau:

4.2.1.1 Khởi tạo và Kiểm soát truy cập

Khi người dùng truy cập vào trang, hệ thống thực hiện chuỗi kiểm tra tự động trước khi hiển thị dữ liệu:

- **Xác thực người dùng:** Hệ thống kiểm tra trạng thái đăng nhập. Nếu chưa xác thực, người dùng bị chuyển hướng về trang Đăng nhập.
- **Khởi tạo ngữ cảnh:** Nếu đã xác thực, hệ thống thực hiện bước *"Kiểm tra tổ chức người dùng thuộc"*. Bước này giúp hệ thống xác định và ưu tiên hiển thị các đơn vị mà người dùng đang là thành viên lên đầu danh sách, sau đó mới hiển thị giao diện cây tổ chức hoàn chỉnh.

4.2.1.2 Phân loại hành động người dùng

Tại giao diện chính, người dùng có thể thực hiện hai nhóm tác vụ, dẫn đến hai nhánh xử lý riêng biệt:

- **Nhánh Quản lý (CRUD):** Bao gồm các thao tác Thêm, Sửa, Xóa hoặc Xem chi tiết. Hệ thống sẽ tiếp nhận và thực hiện các nghiệp vụ backend tương ứng rồi kết thúc luồng.
- **Nhánh Xem cây (View Tree):** Bao gồm thao tác Mở rộng (+) hoặc Thu gọn (-) các nhánh con của cây tổ chức.

4.2.1.3 Cơ chế hiển thị và Lazy Loading

Khi người dùng chọn thao tác **Mở rộng nhánh** (thuộc nhóm thao tác xem cây), hệ thống áp dụng thuật toán tối ưu hóa dữ liệu:

1. **Kiểm tra dữ liệu cục bộ:** Hệ thống kiểm tra xem dữ liệu của nhánh con đó đã được tải về trình duyệt chưa (*Client-side Check*).
2. **Quyết định tải dữ liệu:**
   - *Trường hợp chưa có (Cache Miss):* Hệ thống thực hiện hành động *"Tải dữ liệu bổ sung cho nhánh con"* từ máy chủ.
   - *Trường hợp đã có (Cache Hit):* Hệ thống bỏ qua bước tải mạng và sử dụng ngay dữ liệu có sẵn.
3. **Cập nhật giao diện:** Cuối cùng, hệ thống cập nhật lại hiển thị cây (*Re-render*) để mở rộng nhánh mà người dùng vừa chọn.

4.2.2 Activity Diagram - Quản lý phân quyền

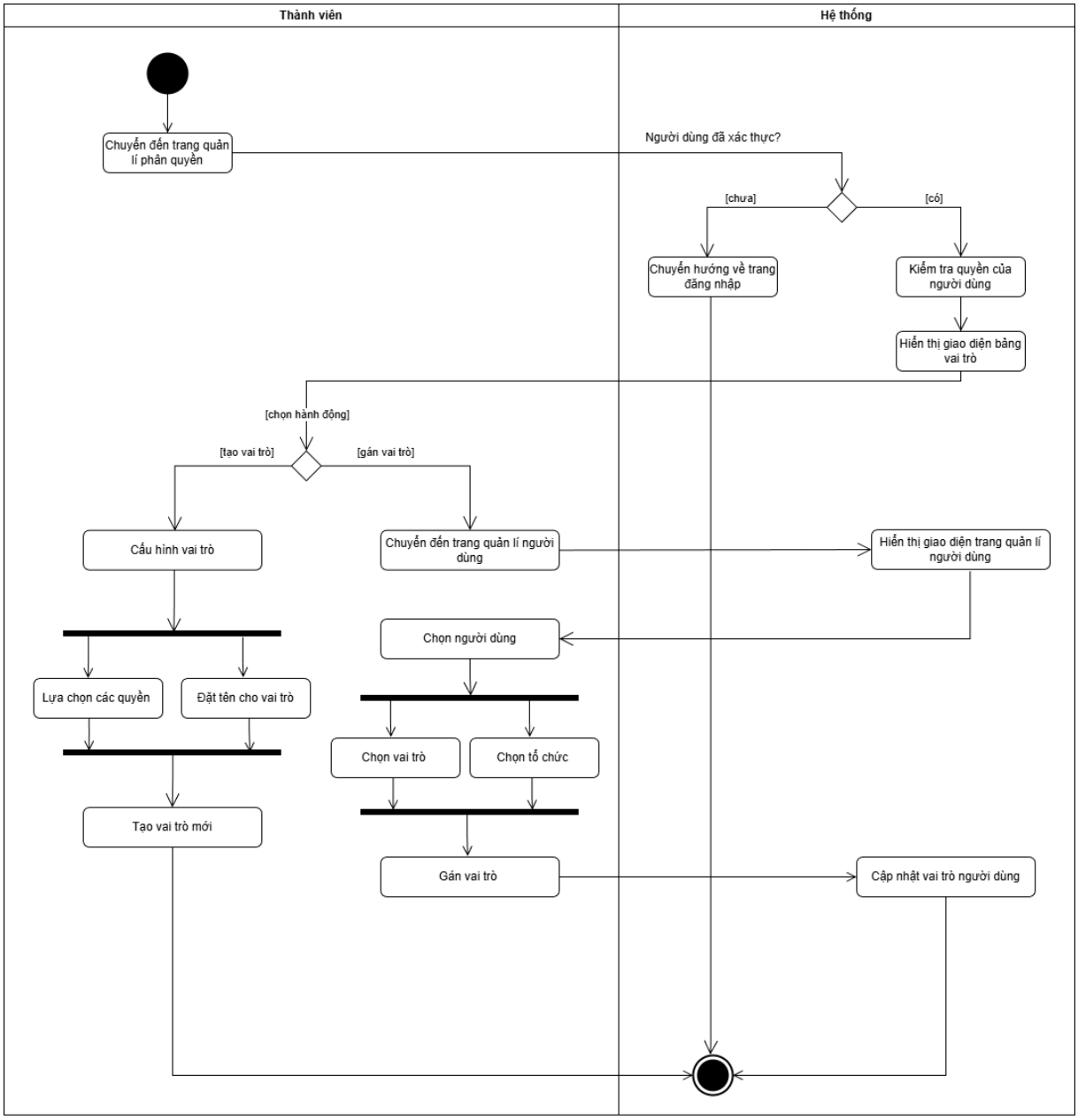


Figure 15: *Activity Diagram - Quản lý phân quyền*

Biểu đồ hoạt động (Activity Diagram) tại Hình 15 mô tả chi tiết quy trình nghiệp vụ quản lý phân quyền, bao gồm các bước kiểm soát truy cập và hai luồng chức năng chính: Cấu hình vai trò mới và Gán vai trò cho người dùng. Quy trình được chia thành hai làn xử lý (*Swimlanes*) là **Thành viên** (Quản trị viên) và **Hệ thống**, với các bước xử lý cụ thể như sau:

4.2.2.1 Kiểm soát truy cập ban đầu

Khi người dùng truy cập vào trang quản lý phân quyền, hệ thống thực hiện lớp bảo mật đầu tiên:

- **Kiểm tra xác thực:** Hệ thống xác minh xem phiên làm việc của người dùng có hợp lệ không.
- **Điều hướng:**
  - Nếu chưa xác thực: Hệ thống tự động chuyển hướng về trang Đăng nhập.

– Nếu đã xác thực: Hệ thống tiếp tục kiểm tra quyền hạn cụ thể của người dùng trước khi hiển thị giao diện bảng danh sách vai trò.

4.2.2.2 Các luồng nghiệp vụ chính

Tại giao diện quản lý, quản trị viên có thể lựa chọn một trong hai hành động, dẫn đến hai nhánh xử lý khác nhau:

- **Nhánh 1: Tạo vai trò mới (Create Role)** Quy trình này sử dụng cơ chế xử lý song song (*Parallel Processing*) để tối ưu hóa thao tác nhập liệu:
  1. Quản trị viên vào màn hình cấu hình vai trò.
  2. Hệ thống cho phép thực hiện đồng thời hai tác vụ: *Lựa chọn các quyền hạn (Permissions)* và *Đặt tên cho vai trò*.
  3. Sau khi hoàn tất cả hai thông tin trên, hệ thống tổng hợp và tạo vai trò mới trong cơ sở dữ liệu.
- **Nhánh 2: Gán vai trò cho người dùng (Assign Role)** Đây là quy trình cấp quyền truy cập cụ thể cho từng thành viên:
  1. Hệ thống chuyển hướng và hiển thị danh sách người dùng.
  2. Quản trị viên chọn một tài khoản người dùng cụ thể.
  3. Tương tự như quy trình tạo vai trò, hệ thống yêu cầu xác định song song hai yếu tố: *Vai trò cần gán* và *Tổ chức áp dụng* (Scope). Điều này đảm bảo tính linh hoạt của mô hình RBAC, cho phép một người dùng có thể giữ các vai trò khác nhau tại các tổ chức khác nhau.
  4. Cuối cùng, hệ thống cập nhật thông tin phân quyền vào cơ sở dữ liệu và kết thúc quy trình.

4.2.3 Activity Diagram - Module Công tác Xã hội

Để minh họa chi tiết luồng xử lý của các chức năng chính trong module Công tác Xã hội, báo cáo trình bày 3 Activity Diagram đại diện cho các pattern quan trọng trong hệ thống:

- **Quản lý trạng thái:** Tạo và chỉnh sửa hoạt động với logic chuyển đổi giữa trạng thái Nháp và Đang hoạt động (UC7.3)
- **Xử lý hàng loạt:** Chấm điểm sinh viên từ file Excel với pipeline parse - validate - preview - batch update (UC7.2)
- **Cấu hình cascade:** Tạo cấu hình giới hạn tự động áp dụng cho tổ chức con theo cấp bậc (UC7.4)

Mỗi diagram thể hiện luồng xử lý hoàn chỉnh từ đầu vào của người dùng đến kết quả cuối cùng, bao gồm các điểm phân nhánh (decision point) và xử lý ngoại lệ quan trọng.

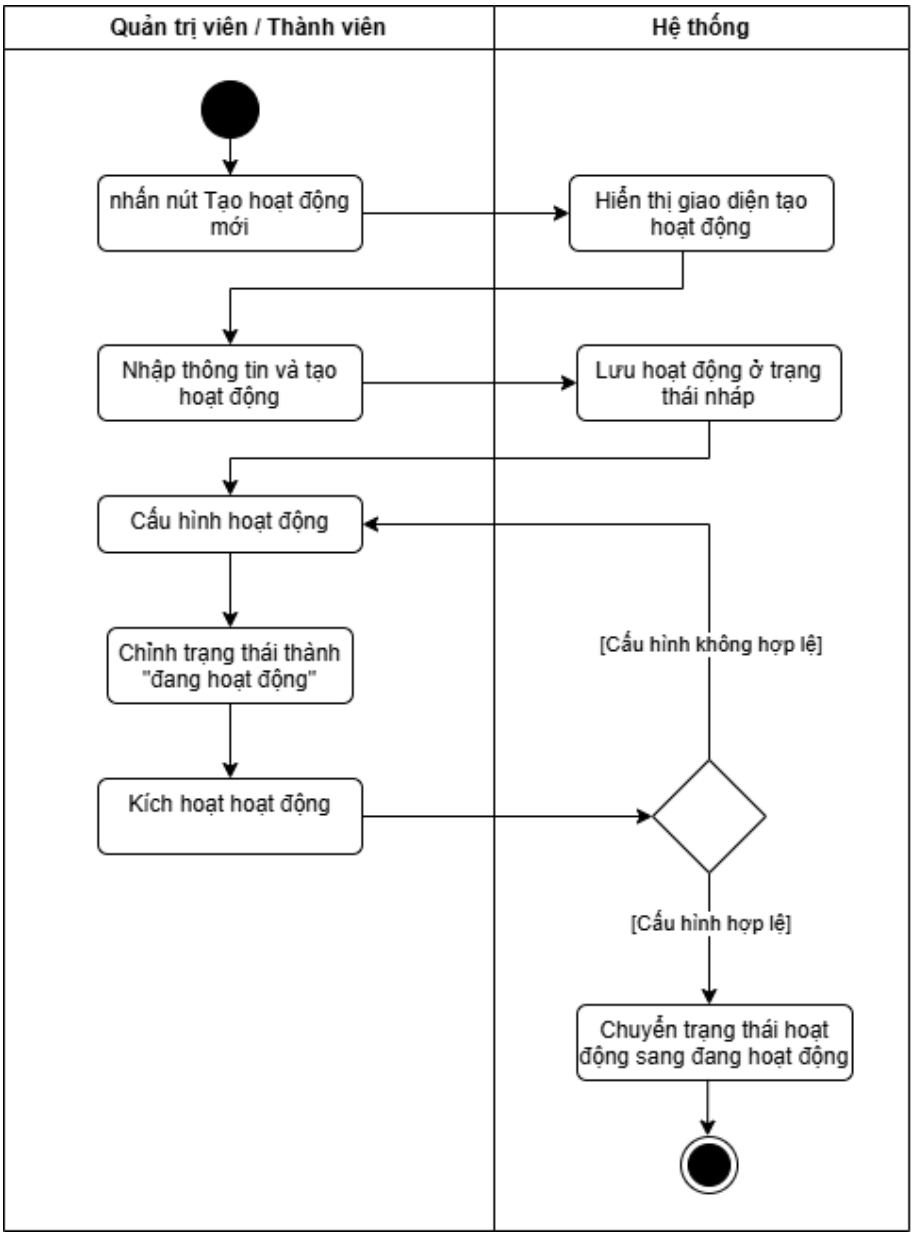


Figure 16: *Activity Diagram - Tạo và chỉnh sửa hoạt động*

Activity Diagram trên minh họa luồng tạo và chỉnh sửa hoạt động CTXH với logic quản lý trạng thái. Điểm quan trọng: khi chọn trạng thái ”Đang hoạt động”, hệ thống yêu cầu validate đầy đủ thông tin (địa điểm, thời gian, số ngày), trong khi trạng thái ”Nháp” cho phép lưu không đầy đủ để tiếp tục sau. Hệ thống kiểm tra cấu hình giới hạn số ngày theo tổ chức nhưng chỉ hiển thị cảnh báo, vẫn cho phép tạo hoạt động vượt giới hạn để linh hoạt với các trường hợp đặc biệt.

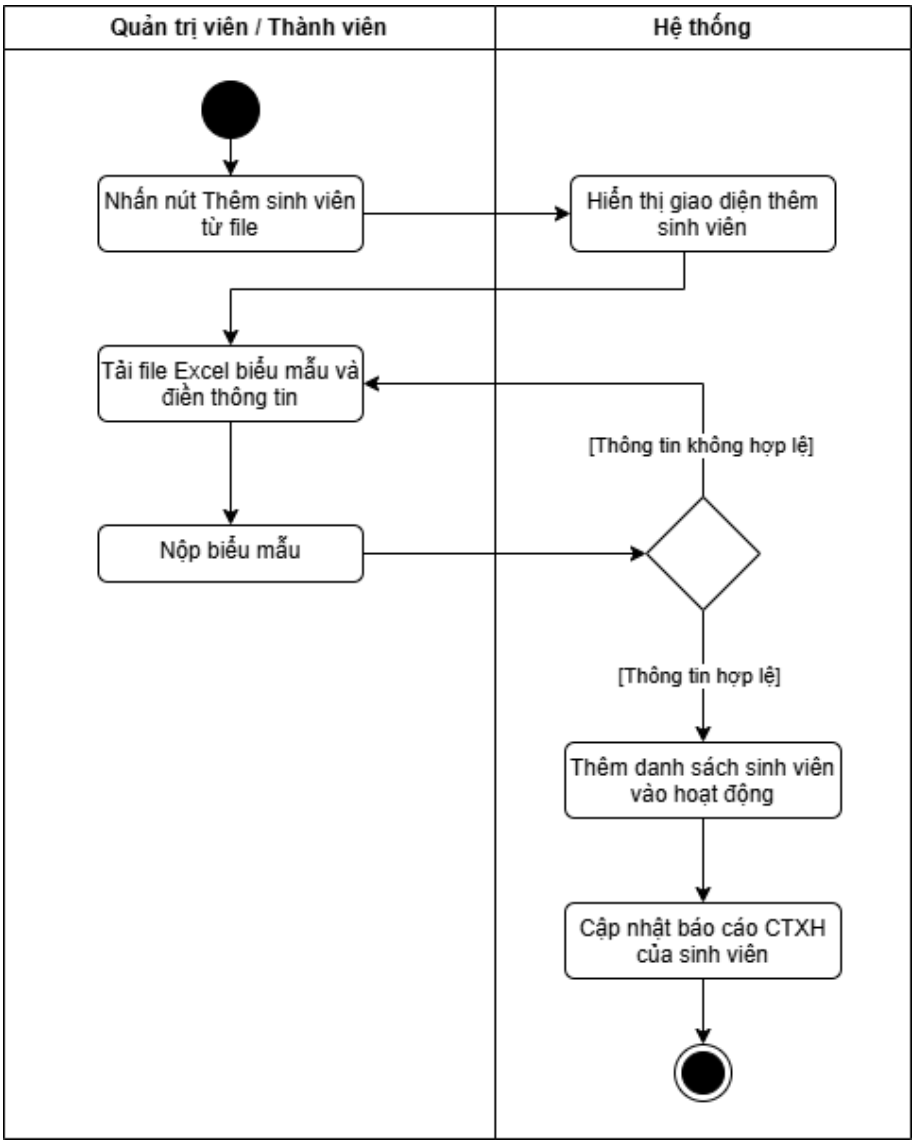


Figure 17: *Activity Diagram - Chấm điểm sinh viên từ file Excel*

Activity Diagram thể hiện luồng xử lý hàng loạt điểm CTXH từ file Excel theo pipeline: parse file → validate từng row → preview kết quả → batch update. Hệ thống xử lý chunk 500 rows để tối ưu hiệu năng với file lớn, validate từng row theo các quy tắc nghiệp vụ (số ngày là bội 0.5, không vượt giới hạn hoạt động, sinh viên tồn tại và đã đăng ký). Trong màn hình preview, các row lỗi được đánh dấu rõ ràng với lý do cụ thể, chỉ các row hợp lệ được cập nhật vào database. Nếu có lỗi trong quá trình batch update, toàn bộ transaction sẽ được rollback để đảm bảo tính nhất quán dữ liệu.

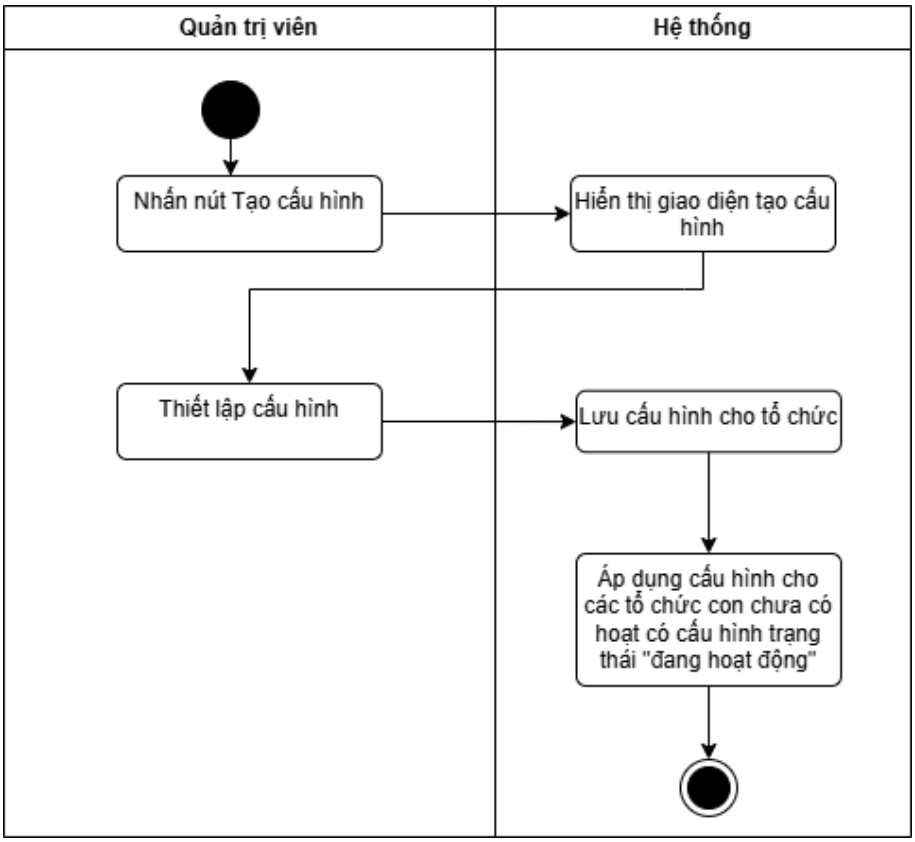


Figure 18: *Activity Diagram - Tạo cấu hình giới hạn ngày CTXH*

Activity Diagram minh họa luồng tạo cấu hình giới hạn số ngày CTXH với cơ chế cascade tự động. Sau khi tạo cấu hình cho tổ chức cha, hệ thống tự động duyệt qua tất cả tổ chức con cấp 1 và cấp 2 theo cấu trúc phân cấp. Với mỗi tổ chức con, hệ thống kiểm tra: nếu chưa có cấu hình thì tạo mới, nếu đã có cấu hình đang hoạt động thì cập nhật số ngày tối đa. Logic này đảm bảo cấu hình được áp dụng nhất quán cho toàn bộ cây tổ chức mà không cần quản trị viên phải thiết lập thủ công từng tổ chức con.

4.2.4 Activity Diagram - Module Biểu mẫu

Để minh họa chi tiết luồng xử lý của các chức năng chính trong module Biểu mẫu, báo cáo trình bày 2 Activity Diagram đại diện cho các pattern quan trọng trong hệ thống:

- **Template parsing & configuration:** Tải lên và phân tích file Word template, trích xuất placeholder động, cấu hình trường dữ liệu (UC8.1)
- **Dynamic form generation:** Sinh form động từ cấu hình, điền dữ liệu, thay thế placeholder và tạo file Word/PDF tự động (UC8.2)

Mỗi diagram thể hiện luồng xử lý hoàn chỉnh từ đầu vào của người dùng đến kết quả cuối cùng, bao gồm các điểm phân nhánh (decision point) và xử lý ngoại lệ quan trọng.

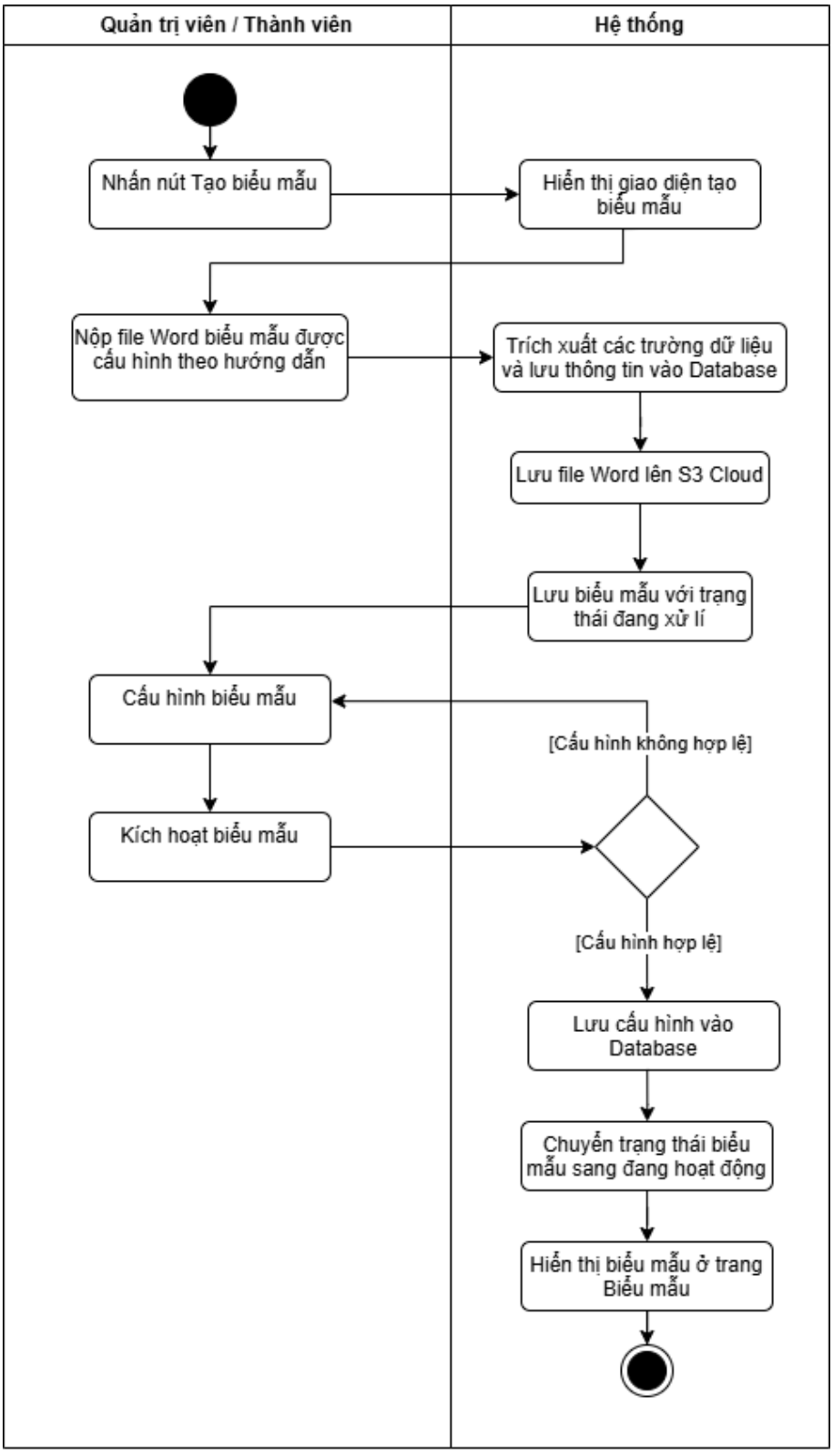


Figure 19: *Activity Diagram - Tạo và cấu hình biểu mẫu*

Activity Diagram trên minh họa luồng tải lên và cấu hình biểu mẫu Word động. Hệ thống nhận file .docx, parse nội dung và trích xuất các placeholder theo cú pháp {{tên_trường}}. Nếu không tìm thấy placeholder nào, hệ thống vẫn cho phép lưu template (có thể là biểu mẫu tĩnh) nhưng hiển thị cảnh báo. Sau khi trích xuất thành công, người dùng cấu hình cho từng placeholder: loại dữ liệu (text, number, date), nhãn hiển thị, và trường bắt buộc/tùy chọn. File template được lưu trên S3 Cloud và cấu hình trường được lưu vào database để sử dụng khi sinh form.

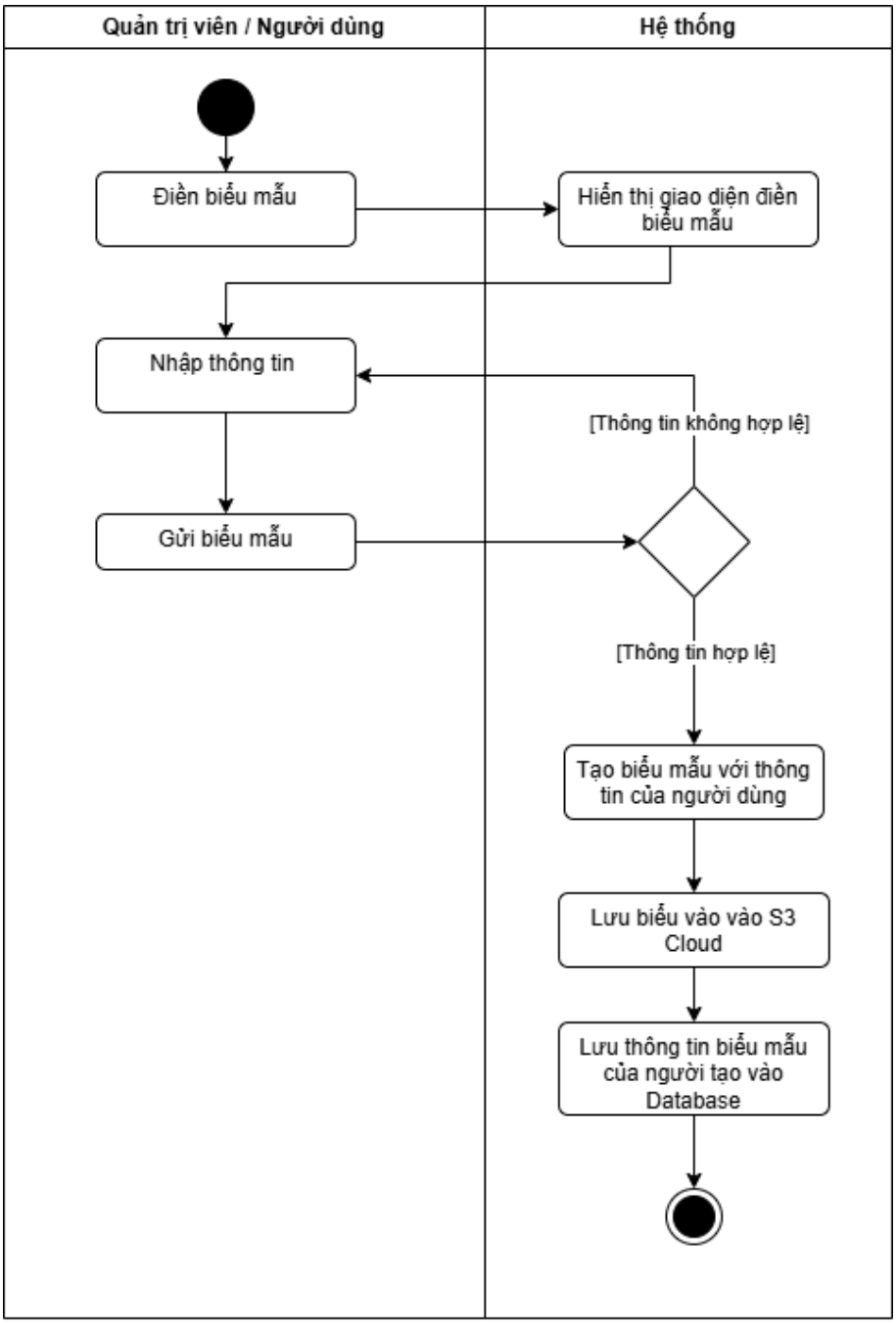


Figure 20: *Activity Diagram - Điền thông tin vào biểu mẫu*

Activity Diagram thể hiện luồng điền biểu mẫu động và sinh file tự động. Hệ thống lấy cấu hình trường từ database để sinh form với các input phù hợp (text box, number input, date picker). Sau khi người dùng điền thông tin, hệ thống validate theo cấu hình (kiểm tra trường bắt buộc, định dạng dữ liệu). Nếu hợp lệ, hệ thống tải template từ S3, thay thế các placeholder bằng dữ liệu người dùng nhập, và sinh file Word/PDF mới. File được lưu lại trên S3 để người dùng có thể tải xuống hoặc xem trước trong browser. Toàn bộ quá trình được lưu thành bản ghi nộp (submission) trong database để tra cứu sau này.

4.2.5 Activity Diagram - Chức năng tạo đợt đóng phí

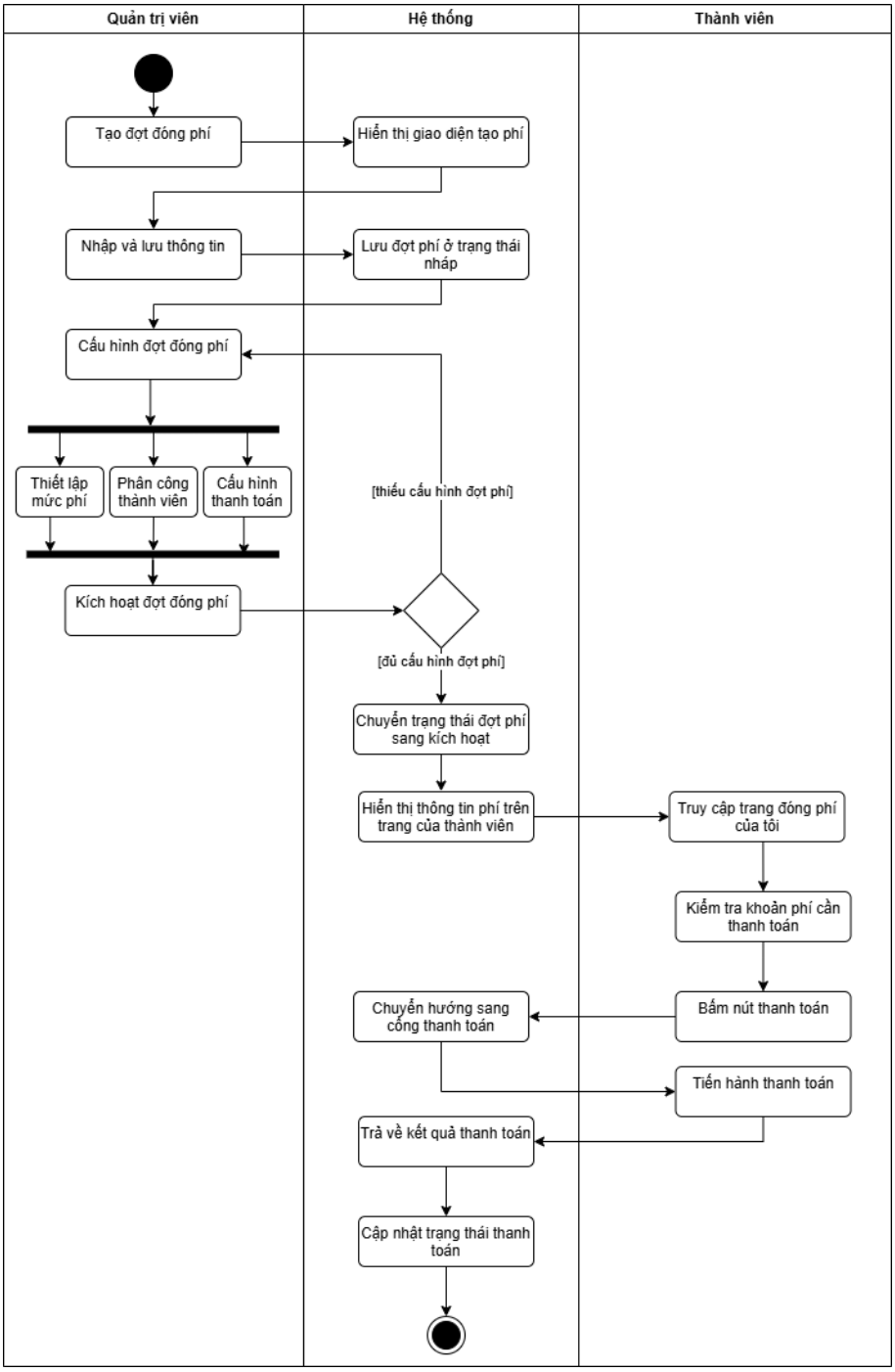


Figure 21: *Activity Diagram - Quản lý đóng phí*

Đầu tiên quản trị viên tạo đợt đóng phí theo tổ chức, sau khi bấm lưu đợt đóng phí sẽ được tạo mới ở bản nháp. Để kích hoạt, quản trị viển cần thiết lập mức phí và phân công thành viên. Sau khi đã thiết lập đầy đủ thì hệ thống sẽ cho phép kích hoạt. Khi đợt đóng phí ở trạng thái kích hoạt thì đợt đóng phí sẽ được công bố cho toàn bộ thành viên được phân công có thể xem và thực hiện thanh toán.

4.2.6 Activity Diagram - Chức năng thêm dữ liệu từ file Excel

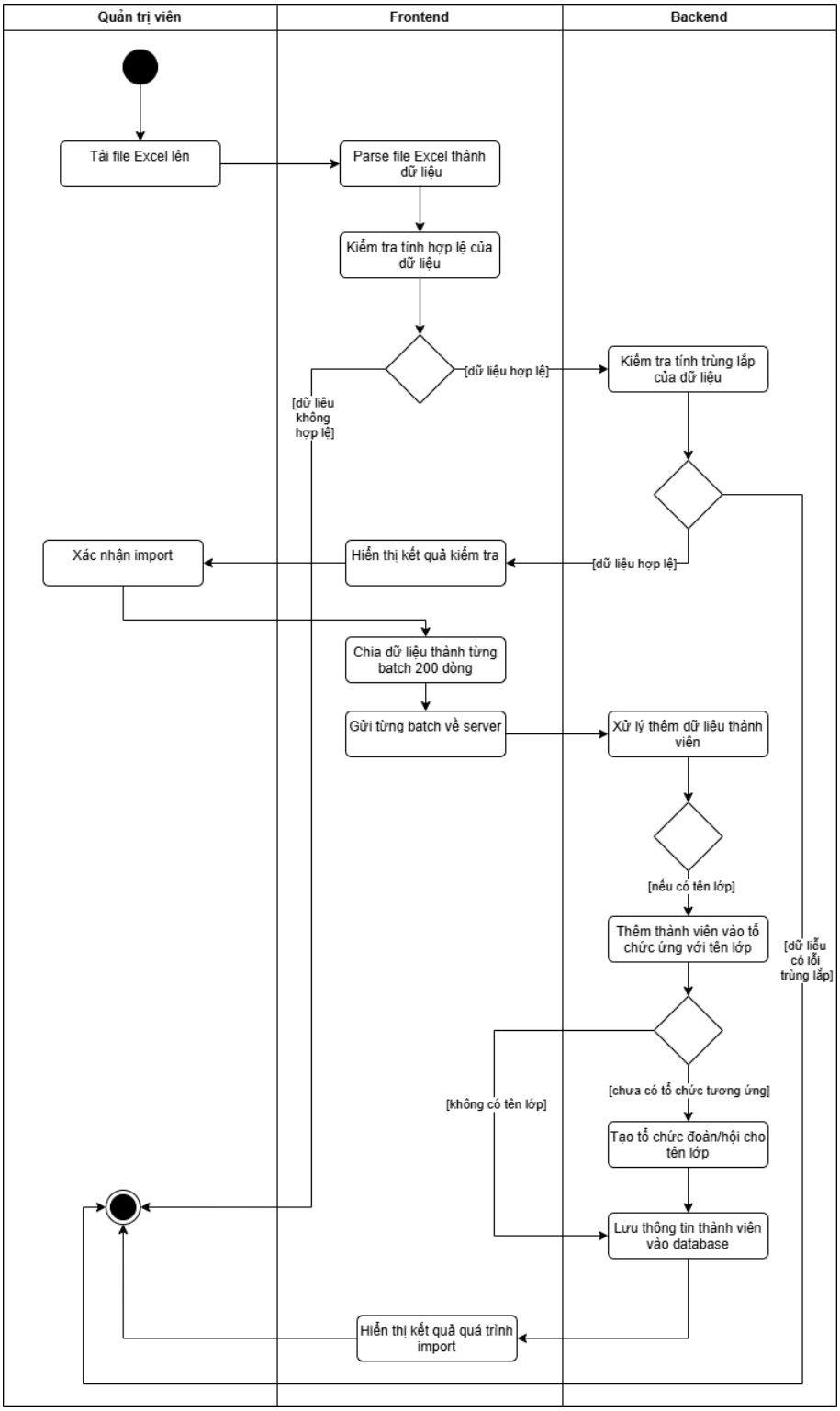


Figure 22: *Activity Diagram - Thêm nhiều thành viên từ file Excel*

Chức năng thêm dữ liệu từ file Excel trong module User được xây dựng nhằm hỗ trợ việc nhập nhanh số lượng lớn dữ liệu sinh viên vào hệ thống, đặc biệt trong giai đoạn đầu mỗi năm học khi cần cập nhật thông tin cho nhiều khóa và lớp khác nhau. Do đặc thù dữ liệu của các module quản lý như User, Department và Class là không đồng nhất, mỗi module có quy trình import riêng phù hợp với cấu trúc dữ liệu tương ứng. Trong phạm vi chức năng này, nhóm tập trung mô tả quy trình import dữ liệu tại module User do đây là module có tần suất sử dụng cao và đóng vai trò trung tâm trong việc quản lý thành viên.

Khi quản trị viên tải lên file Excel, hệ thống tiến hành đọc, kiểm tra dữ liệu đầu vào và xử lý từng dòng tương ứng với một người dùng. Với mỗi bản ghi, hệ thống tự động xác định và tạo cấu trúc tổ chức phù hợp dựa trên thông tin lớp học và khoa đào tạo. Cụ thể, hệ thống kiểm tra sự tồn tại của các tổ chức cấp khoa (level 2) và cấp lớp (level 3); trong trường hợp chưa tồn tại, hệ thống sẽ tự động khởi tạo mới. Tổ chức cấp cao nhất (level 1) là Đoàn trường hoặc Hội Sinh viên luôn được sử dụng làm tổ chức gốc để đảm bảo tính thống nhất trong mô hình tổ chức.

Sau khi hoàn tất việc xác định và tạo tổ chức, người dùng được tự động gán vào tất cả các tổ chức liên quan, bao gồm cấp trường (level 1), cấp khoa (level 2) và cấp lớp (level 3), với vai trò mặc định là *MEMBER*. Hệ thống áp dụng logic xác định khoa đào tạo dựa trên mã lớp: nếu tên lớp bắt đầu bằng các tiền tố *CC*, *CN*, *CT* hoặc *TT*, sinh viên sẽ được gán vào *Văn phòng Đào tạo Quốc tế*; các trường hợp còn lại sẽ xác định khoa dựa trên mã *trainingDepartment* trong dữ liệu đầu vào.

Ví dụ minh họa:

- **Mã lớp *MT22KH02***: Hệ thống xác định khoa đào tạo là *Khoa Khoa học và Kỹ thuật Máy tính*, từ đó tạo (hoặc lấy) tổ chức cấp 2 là *Liên chi đoàn/ Liên chi hội Khoa Khoa học và Kỹ thuật Máy tính* và tổ chức cấp 3 là *Chi đoàn/ Chi hội lớp MT22KH02*.
- **Mã lớp *CC23KTO1***: Do mã lớp bắt đầu bằng tiền tố *CC*, hệ thống xác định khoa đào tạo là *Văn phòng Đào tạo Quốc tế*, đồng thời tạo (hoặc lấy) tổ chức cấp 2 là *Liên chi đoàn/ Liên chi hội Văn phòng Đào tạo Quốc tế* và tổ chức cấp 3 là *Chi đoàn/ Chi hội lớp CC23KTO1*.

4.3 Mô hình hóa Sequence Diagram

4.3.1 Sequence Diagram - Module Quản lý tổ chức

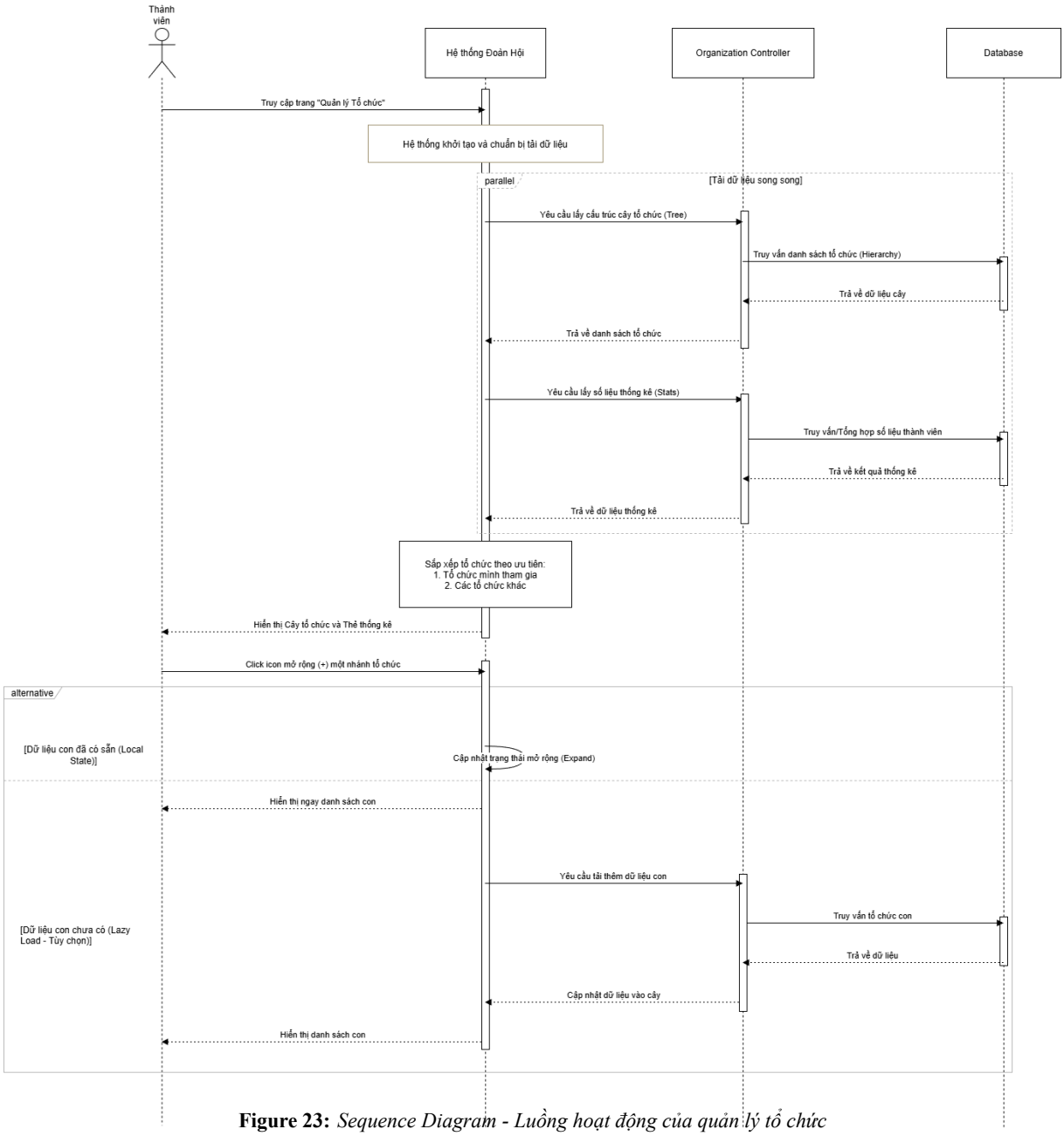


Figure 23: *Sequence Diagram - Luồng hoạt động của quản lý tổ chức*

Quy trình hiển thị và tương tác với cấu trúc tổ chức (Organization Tree) là một trong những nghiệp vụ quan trọng, yêu cầu tốc độ phản hồi nhanh và trải nghiệm người dùng mượt mà. Biểu đồ tuần tự (Sequence Diagram) tại Hình 23 mô tả chi tiết luồng tương tác giữa Thành viên, Hệ thống Frontend và Backend Controller. Quy trình xử lý được chia thành hai giai đoạn chính nhằm tối ưu hóa hiệu năng hệ thống:

4.3.1.1 Giai đoạn 1: Khởi tạo và Tải dữ liệu song song

Khi người dùng truy cập vào trang quản lý, hệ thống không thực hiện các lệnh gọi API tuần tự mà sử dụng kỹ thuật **Parallel Processing** (Xử lý song song) để giảm thiểu tổng thời gian phản hồi (Total Response Time).

- **Truy vấn cấu trúc cây (Tree Structure):** Hệ thống gửi yêu cầu đến *Organization Controller* để lấy danh sách tổ chức phân cấp.

- **Truy vấn thống kê (Statistics):** Đồng thời, một luồng xử lý khác truy vấn số liệu tổng hợp về thành viên từ cơ sở dữ liệu.

Sau khi nhận đủ dữ liệu từ cả hai luồng, Frontend thực hiện thuật toán sắp xếp cục bộ (*Client-side Sorting*). Các tổ chức mà người dùng đang là thành viên sẽ được ưu tiên hiển thị lên đầu danh sách (Priority 1), giúp người dùng tiếp cận thông tin quan trọng nhanh nhất.

4.3.1.2 Giai đoạn 2: Tương tác và Cơ chế Lazy Loading

Để giảm tải băng thông và dung lượng bộ nhớ ban đầu, hệ thống áp dụng cơ chế **Lazy Loading** (Tải dữ liệu theo nhu cầu) cho các thao tác mở rộng nhánh cây:

1. Khi người dùng kích hoạt mở rộng một nhánh tổ chức, hệ thống kiểm tra trạng thái dữ liệu cục bộ (*Local State*).
2. **Trường hợp Cache Hit:** Nếu dữ liệu nhánh con đã tồn tại, giao diện được cập nhật tức thì mà không cần gọi về Server.
3. **Trường hợp Cache Miss:** Nếu dữ liệu chưa có, hệ thống mới gửi yêu cầu tải bổ sung (*Fetch additional data*) tới Controller. Dữ liệu mới sau đó được cập nhật vào cây và hiển thị cho người dùng.

Giải pháp này giúp hệ thống đảm bảo khả năng mở rộng tốt (Scalability) ngay cả khi số lượng tổ chức trong trường tăng lên hàng nghìn đơn vị, đồng thời duy trì trải nghiệm mượt mà cho người dùng cuối.

4.3.2 Sequence Diagram - Module Phân quyền

Đảm bảo an toàn bảo mật và phân quyền chính xác là yêu cầu cốt lõi của hệ thống. Biểu đồ tuần tự tại Hình 24 mô tả chi tiết quy trình đăng nhập và cơ chế tự động tải quyền hạn (Permissions) kết hợp với chiến lược Caching để tối ưu hiệu năng.

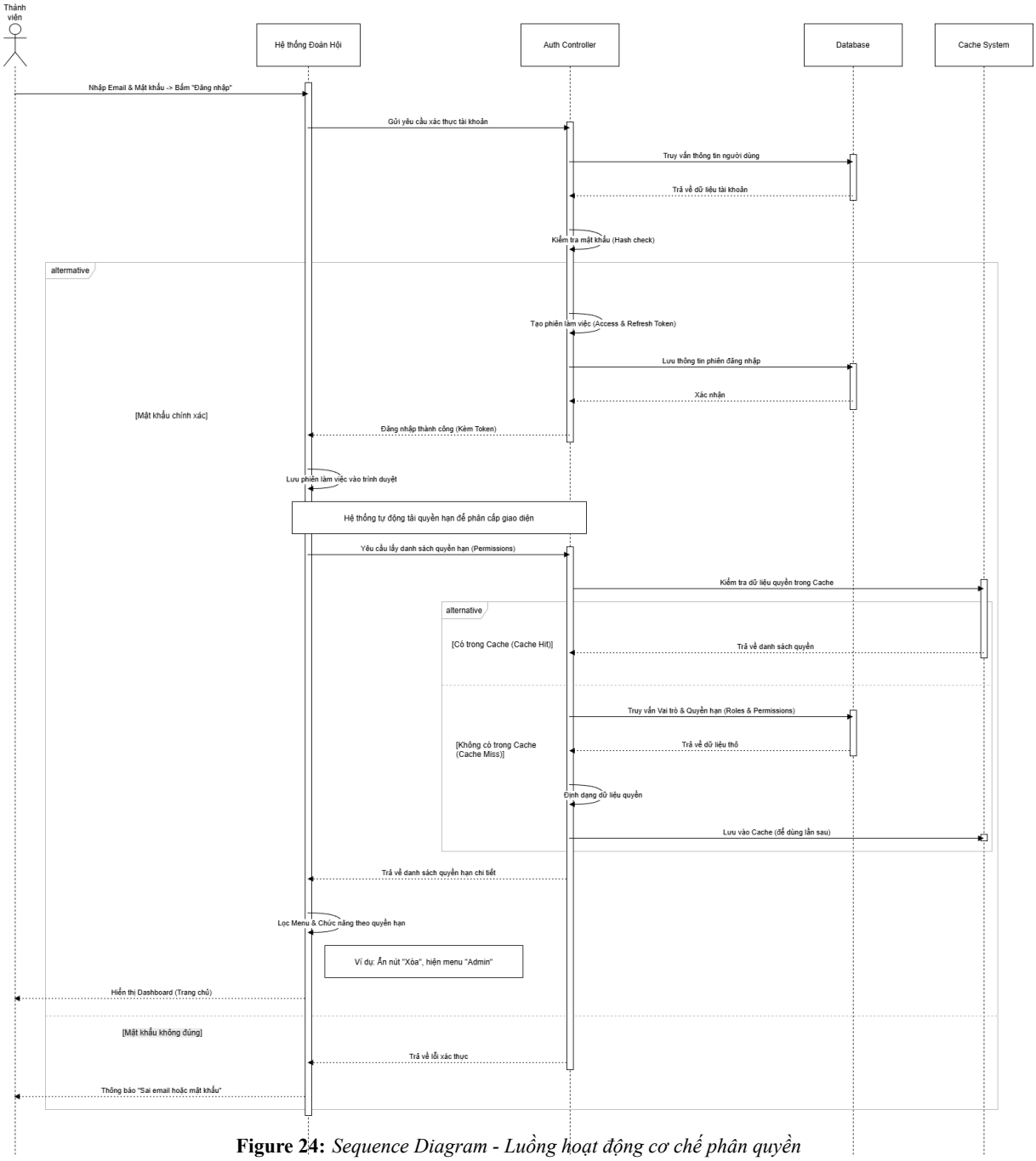


Figure 24: *Sequence Diagram - Luồng hoạt động cơ chế phân quyền*

Quy trình được chia thành ba giai đoạn xử lý chính:

4.3.2.1 Giai đoạn 1: Xác thực người dùng (Authentication)

Khi thành viên nhập thông tin đăng nhập, hệ thống thực hiện các bước kiểm tra bảo mật nghiêm ngặt:

- **Kiểm tra thông tin:** *Auth Controller* truy vấn thông tin từ cơ sở dữ liệu và thực hiện so khớp mật khẩu đã được mã hóa (*Password Hashing Check*).
- **Cấp phát phiên làm việc:** Nếu thông tin chính xác, hệ thống khởi tạo phiên làm việc bằng cách cấp phát cặp khóa *Access Token* và *Refresh Token*. Thông tin phiên này cũng được lưu vết vào cơ sở dữ liệu để quản lý bảo mật.

4.3.2.2 Giai đoạn 2: Tải quyền hạn và Tối ưu hóa với Cache

Sau khi đăng nhập thành công, hệ thống tự động thực hiện quy trình phân quyền (Authorization) để xác định phạm vi truy cập của người dùng. Để giảm tải cho cơ sở dữ liệu khi lượng truy cập lớn, nhóm áp dụng chiến lược

Caching (Lưu bộ nhớ đệm):

1. Hệ thống kiểm tra danh sách quyền hạn của người dùng trong *Cache System*.
2. **Trường hợp Cache Hit:** Nếu dữ liệu quyền đã có sẵn trong bộ nhớ đệm, hệ thống trả về ngay lập tức, giúp giảm độ trễ phản hồi xuống mức thấp nhất.
3. **Trường hợp Cache Miss:** Nếu chưa có trong Cache, hệ thống mới thực hiện truy vấn phức tạp (Join bảng Vai trò & Quyền hạn) từ Database, sau đó định dạng dữ liệu và lưu ngược vào Cache cho các lần truy cập sau.

4.3.2.3 Giai đoạn 3: Phân cấp giao diện (UI Adaptation)

Dựa trên danh sách quyền hạn nhận được, *Hệ thống Đoàn Hội* (Frontend) thực hiện lọc và hiển thị giao diện động:

- Các chức năng không thuộc quyền hạn của người dùng (ví dụ: nút ”Xóa”, menu ”Quản trị”) sẽ bị ẩn đi.
- Người dùng được chuyển hướng đến Trang chủ (Dashboard) với giao diện được cá nhân hóa theo đúng vai trò.

Quy trình này đảm bảo tính toàn vẹn dữ liệu và ngăn chặn các truy cập trái phép vào các chức năng nhạy cảm của hệ thống.

4.3.3 Sequence Diagram - Module Công tác Xã hội

Để minh họa chi tiết tương tác giữa các thành phần trong hệ thống, báo cáo trình bày 2 Sequence Diagram đại diện cho các luồng xử lý quan trọng trong module Công tác Xã hội:

- **Tạo và chỉnh sửa hoạt động:** Tương tác giữa người dùng, controller, service layer và database khi tạo/cập nhật hoạt động với logic quản lý trạng thái
- **Chấm điểm từ Excel:** Tương tác phức tạp giữa nhiều component trong quá trình xử lý file Excel hàng loạt, bao gồm file parsing, validation và batch update

Mỗi diagram thể hiện chi tiết các message trao đổi giữa các đối tượng, thứ tự thực hiện các phương thức, và cơ chế xử lý dữ liệu trong từng bước.

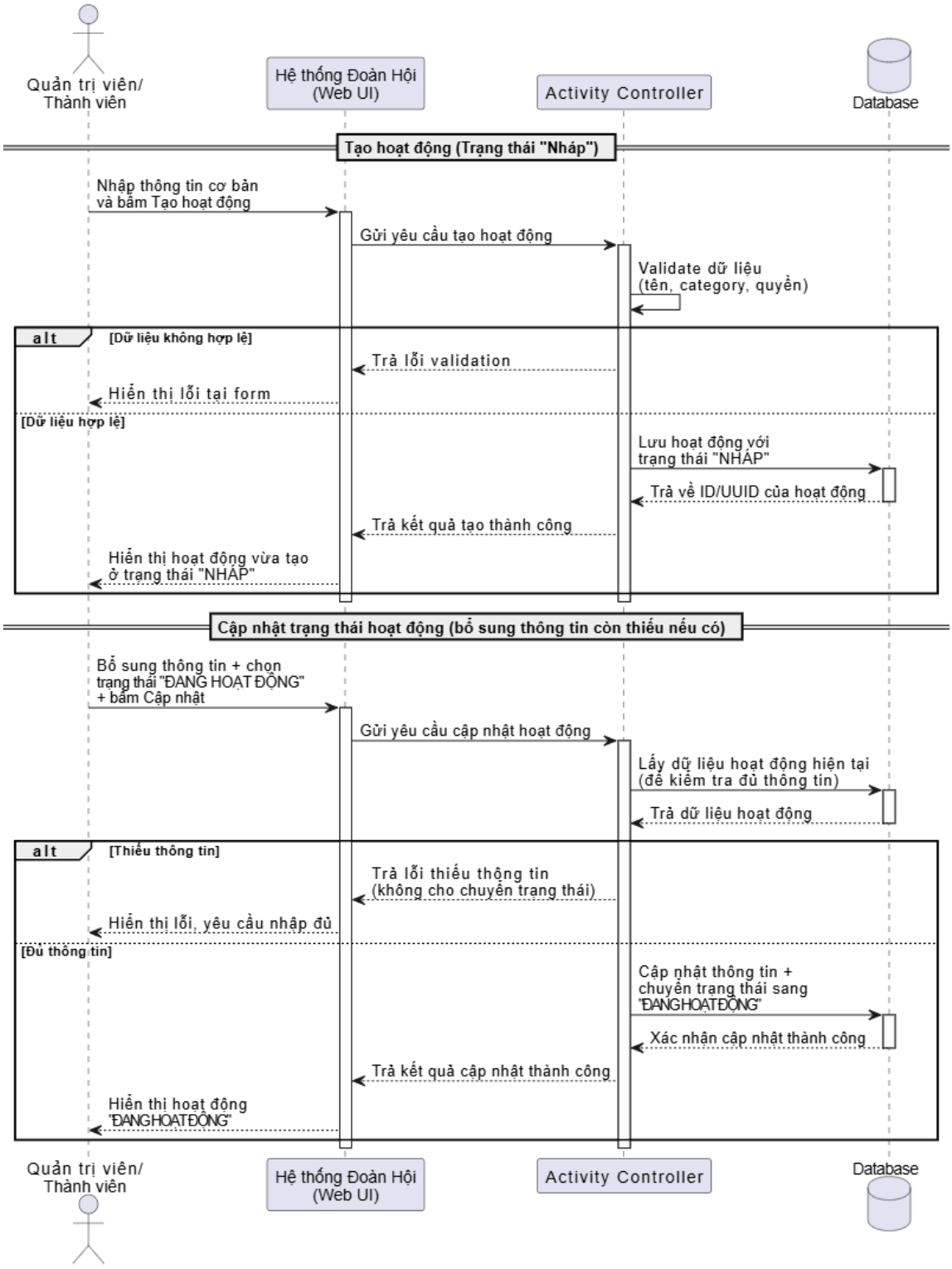


Figure 25: *Sequence Diagram - Tạo và chỉnh sửa hoạt động CTXH*

Sequence Diagram trên mô tả tương tác giữa các thành phần khi tạo hoặc chỉnh sửa hoạt động CTXH. Actor (Quản trị viên/Người dùng) gửi request qua Controller, Controller gọi Service Layer để xử lý business logic. Service kiểm tra quyền, validate dữ liệu đầu vào (tên không trống, thời gian hợp lệ, số ngày là bội 0.5), và kiểm tra cấu hình giới hạn từ ConfigService. Nếu trạng thái là ”Đang hoạt động”, Service validate đầy đủ thông tin bắt buộc. Sau khi validation thành công, Service gọi Repository để lưu vào database và trả về kết quả cho người dùng. Nếu có lỗi validation, hệ thống trả về error message cụ thể để người dùng sửa lại.

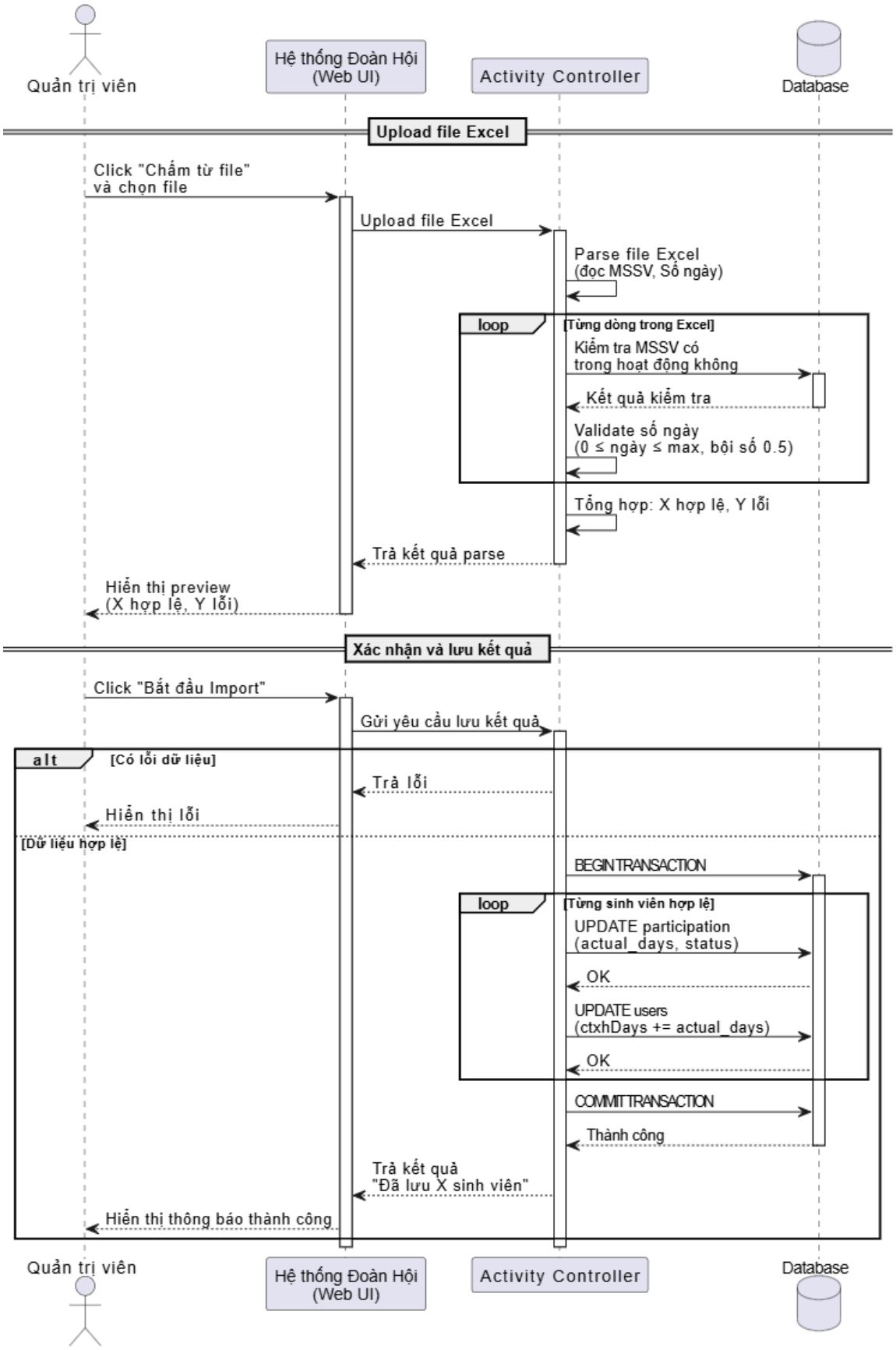


Figure 26: *Sequence Diagram - Chấm điểm sinh viên từ file Excel*

Sequence Diagram thể hiện luồng xử lý phức tạp khi chấm điểm hàng loạt từ Excel. Controller nhận file upload, gọi ExcelParserService để parse file thành chunks (500 rows/chunk). Với mỗi chunk, ValidationService validate

từng row: kiểm tra sinh viên tồn tại trong database, số ngày hợp lệ (bội 0.5, không âm), và không vượt giới hạn hoạt động. Kết quả validation được trả về dạng preview (danh sách row hợp lệ và row lỗi với lý do). Khi người dùng xác nhận, Service bắt đầu batch update trong transaction: với mỗi row hợp lệ, cập nhật điểm tham gia và cộng vào tổng CTXH của sinh viên. Nếu có lỗi trong quá trình update, toàn bộ transaction được rollback để đảm bảo tính nhất quán dữ liệu. Kết quả cuối cùng (số lượng sinh viên đã cập nhật) được trả về cho người dùng.

4.3.4 Sequence Diagram - Module Biểu mẫu

Để minh họa chi tiết tương tác giữa các thành phần trong hệ thống, báo cáo trình bày Sequence Diagram đại diện cho luồng xử lý quan trọng trong module Biểu mẫu:

- **Tạo và cấu hình biểu mẫu:** Tương tác giữa người dùng, controller, parser service, S3 storage và database trong quá trình tải lên, phân tích template và cấu hình trường động

Diagram thể hiện chi tiết các message trao đổi giữa các đối tượng, cơ chế parse file Word để trích xuất placeholder, và cách lưu trữ phân tán giữa S3 (file template) và database (cấu hình trường).

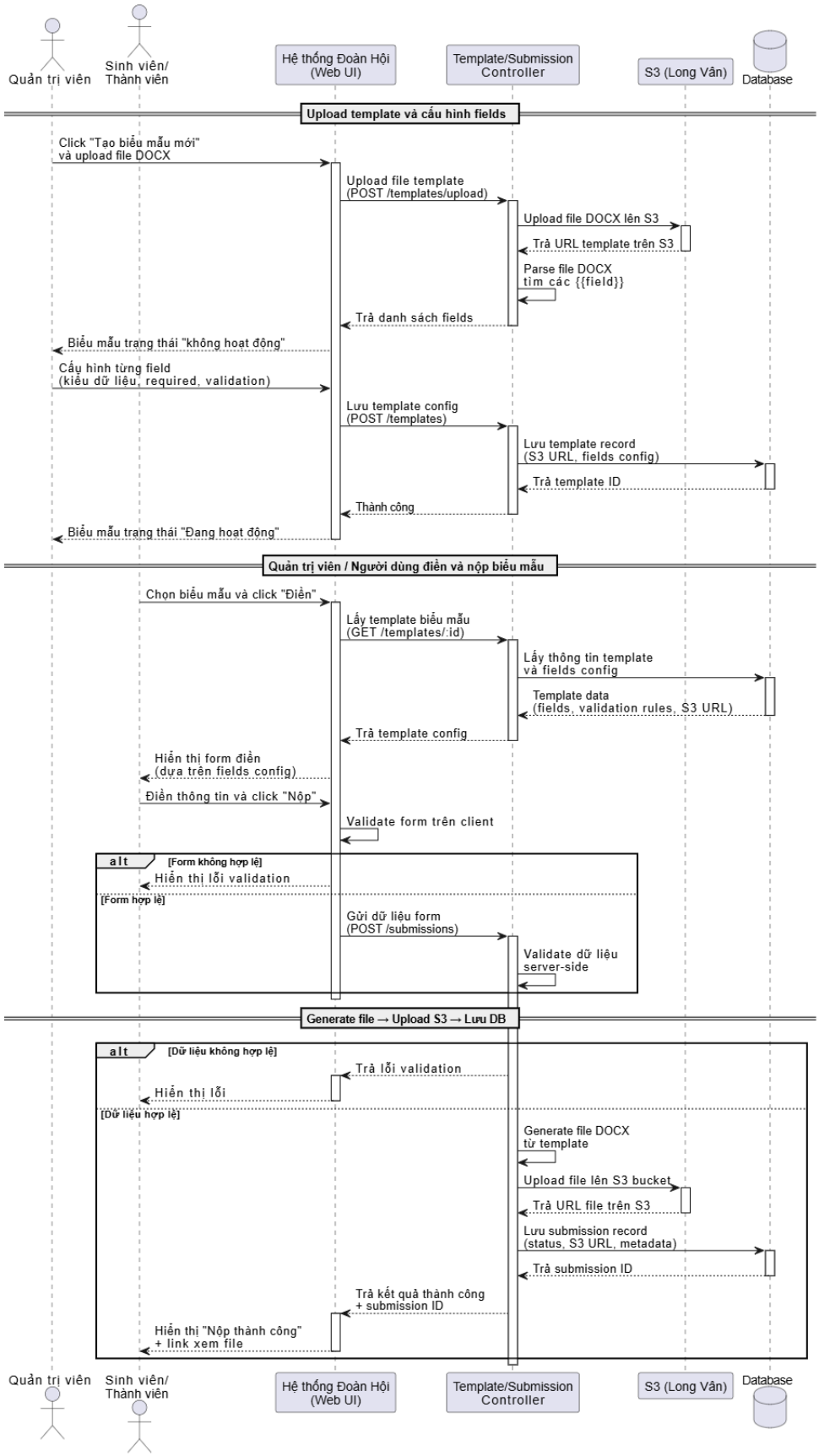


Figure 27: *Sequence Diagram - Tạo và cấu hình biểu mẫu*

Sequence Diagram trên mô tả tương tác giữa các thành phần khi tạo và cấu hình biểu mẫu động. Actor (Quản trị viên/Người dùng) upload file Word (.docx) qua Controller. Controller validate định dạng file và kích thước (tối đa 5MB), sau đó gọi WordParserService để phân tích nội dung. Parser sử dụng thư viện docx để đọc document

structure, tìm kiếm các placeholder theo pattern {{field_name}} trong toàn bộ paragraphs và tables. Danh sách placeholder được trả về cho người dùng để cấu hình (nếu không tìm thấy placeholder nào, hiển thị cảnh báo nhưng vẫn cho phép tiếp tục). Người dùng cấu hình từng placeholder: chọn loại dữ liệu, nhập nhãn hiển thị, đánh dấu bắt buộc/tùy chọn. Sau khi hoàn tất, Controller gọi S3StorageService để upload file template lên cloud storage, nhận về URL. Cuối cùng, TemplateService lưu thông tin template (tên, URL S3) và cấu hình trường vào database. Transaction đảm bảo cả file và metadata đều được lưu thành công, nếu có lỗi sẽ rollback và xóa file đã upload trên S3.

4.3.5 Sequence Diagram - Chatbot hỏi-đáp hoạt động CTXH

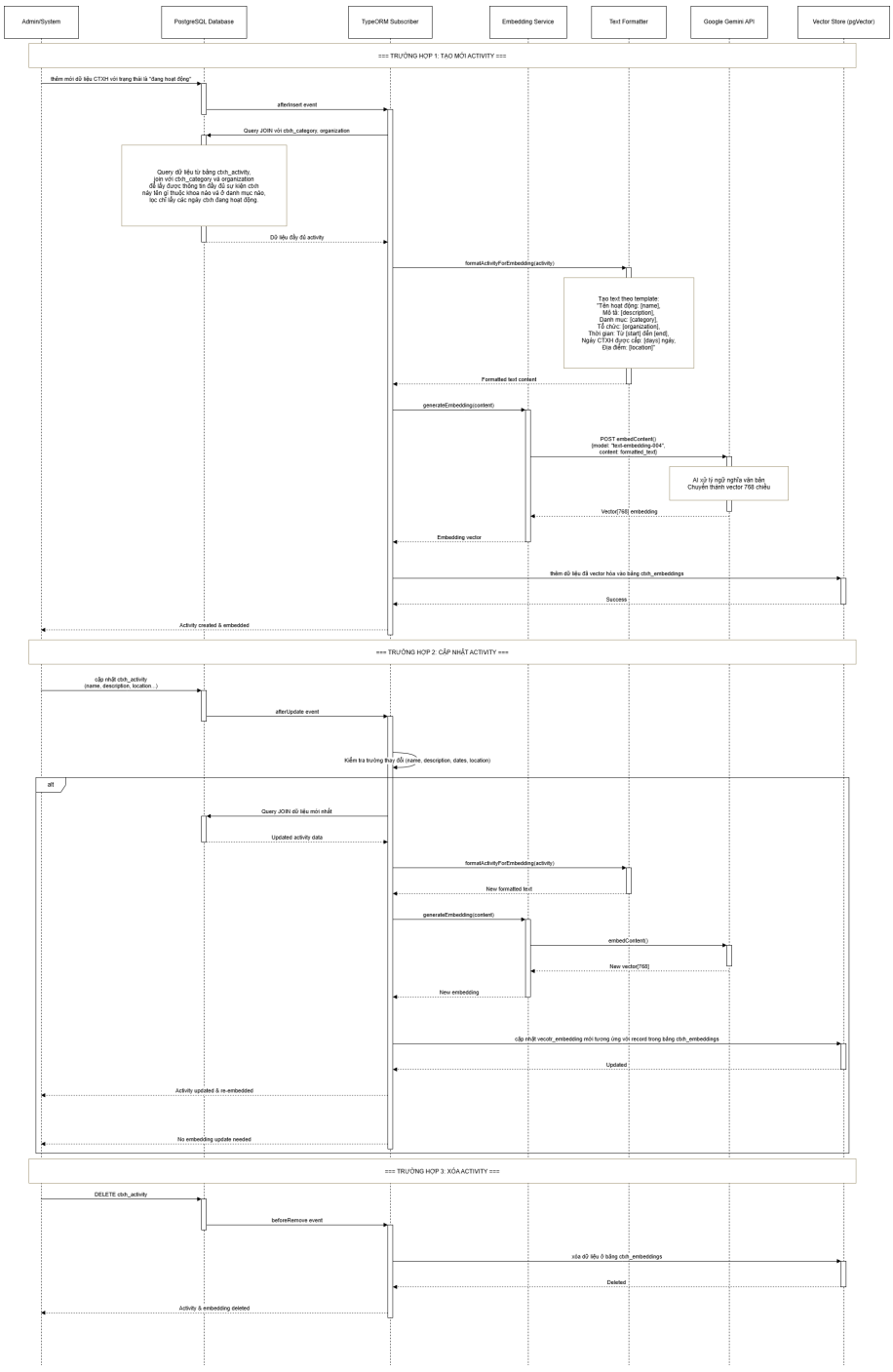


Figure 28: *Sequence Diagram - Pipeline tự động chuyển đổi dữ liệu CTXH sang dạng vector embedding)*

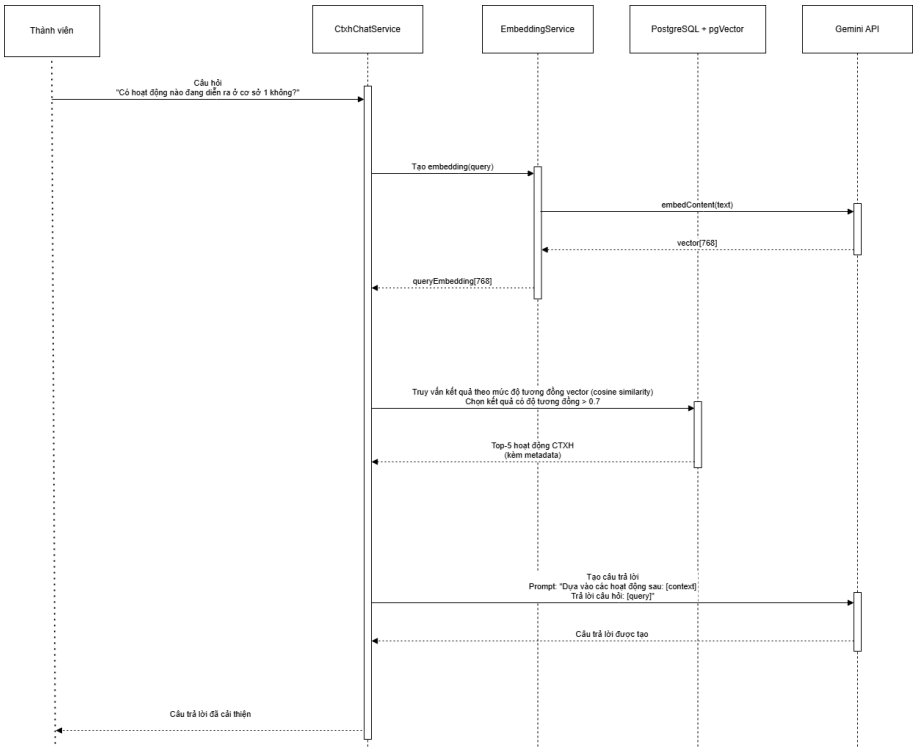


Figure 29: *Sequence Diagram - Luồng hoạt động của Chatbot RAG)*

Mô tả trong phần này sẽ được làm rõ ở phần 5.3 Thiết kế Chatbot RAG

5 Phân tích & Thiết kế hệ thống

5.1 Thiết kế dữ liệu

5.1.1 ERD tổng thể

5.1.1.1 Dữ liệu về thành viên

- **USER**: Thông tin chung
  - Họ tên (family_name, given_name)
  - Email, số điện thoại
  - CCCD (Căn cước công dân)
  - Ngày sinh, giới tính
  - **STUDENT**: Thông tin sinh viên
    - Năm nhập học (admission_year)
    - Năm chương trình đào tạo (curriculum_year)
    - Mã khoa (faculty_code)
    - Hệ đào tạo (training_system)
    - Bậc đào tạo (training_level)
    - Tổng số ngày CTXH (ctxh_days_total)
    - Số ngày CTXH từ hệ thống cũ (ctxh_days_from_old_system)
  - **STAFF**: Cán bộ Đoàn/Hội
    - Không có thêm attributes đặc biệt ngoài USER
- **DEPARTMENT**: Khoa/bộ môn
  - Mã khoa (department_code)
  - Tên khoa (department_name)
  - Mô tả (description)
- **CLASS**: Lớp học
  - Tên lớp (class_name)
  - Năm học (academic_year)

5.1.1.2 Cấu trúc tổ chức

- **ORGANIZATION**: Đơn vị tổ chức
  - Mã tổ chức (organization_code)
  - Tên tổ chức (organization_name)
  - Loại tổ chức (organization_type)
  - Mô tả (description)

5.1.1.3 Hệ thống phân quyền

- **ROLE**: Vai trò
  - Tên vai trò (role_name): Admin, Student, Manager, v.v.
  - Mô tả (description)
- **PERMISSION**: Quyền hạn cụ thể
  - Mã quyền (permission_code): users.create, organizations.access, ...
  - Tên quyền (permission_name)
  - Mô tả (description)
  - Danh mục (category)
  - Ngữ cảnh (context): xác định quyền này là toàn hệ thống hay theo tổ chức bằng ENUM('System', 'Organization')

5.1.1.4 Hệ thống Công tác xã hội

- **CTXH_CATEGORY**: Danh mục hoạt động
  - Tên danh mục (category_name): Tình nguyện, Khởi nghiệp, v.v.
  - Mô tả (description)
- **CTXH_ACTIVITY**: Hoạt động cụ thể
  - Tên hoạt động (activity_name)
  - Mô tả (description)
  - Thời gian (start_date, end_date)
  - Địa điểm (location)
  - Số ngày dự kiến (planned_total_days)
  - Trạng thái (activity_status): Draft, Active, Completed, Cancelled
- **PARTICIPATION**: Đăng ký tham gia của sinh viên
  - Trạng thái tham gia (participation_status)
  - Số ngày thực tế (actual_days)
  - Ghi chú (note)
- **CTXH_LIMIT_POLICY**: Chính sách giới hạn số ngày CTXH
  - Số ngày tối đa (max_days)
  - Trạng thái chính sách (policy_status): Active, Inactive
  - Mô tả (description)

5.1.1.5 Hệ thống biểu mẫu điện tử

- **TEMPLATE**: Template biểu mẫu
  - Tên template (template_name)
  - Mô tả (description)
  - Trạng thái (template_status): Active, Inactive

- **TEMPLATE_FIELD**: Trường dữ liệu trong template
  - Tên trường (field_name)
  - Tên hiển thị (display_name)
  - Loại trường (field_type): Text, Number, Date, Select, Checkbox, File
  - Placeholder, help text, giá trị mặc định
  - Thứ tự hiển thị (display_order)
  - Hiển thị hay không (is_visible)
- **TEMPLATE_SUBMISSION**: Đơn nộp của người dùng
  - Trạng thái (submission_status): Draft, Submitted, Approved, Rejected
  - Ngày nộp (submitted_date)
- **GENERATED_DOCUMENT**: Văn bản Word được sinh tự động
  - Tên file (file_name)
  - Kích thước file (file_size)

5.1.1.6 Hệ thống đóng phí

- **PAYMENT_CAMPAIGN**: Chiến dịch thu phí
  - Tên chiến dịch (campaign_name)
  - Mô tả (description)
  - Thời gian (start_date, end_date)
  - Trạng thái (campaign_status): Draft, Active, Completed, Cancel
- **FEE_LEVEL**: Mức phí
  - Tên mức (level_name)
  - Số tiền (amount)
  - Mô tả (description)
- **PAYMENT_ASSIGNMENT**: Phân công thu phí cho sinh viên
  - Trạng thái (assignment_status): Pending, Paid, Overdue
  - Hạn chót (due_date)
  - Ghi chú (note)
- **PAYMENT_CAMPAIGN_PAYOS_CONFIG**: Cấu hình cổng thanh toán PayOS cho đợt đóng phí
  - ClientID
  - APIKey
  - ChecksumKey
- **PAYMENT_HISTORY**: Lịch sử giao dịch thanh toán
  - Phương thức thanh toán (payment_method)
  - Mã đơn hàng (order_code)

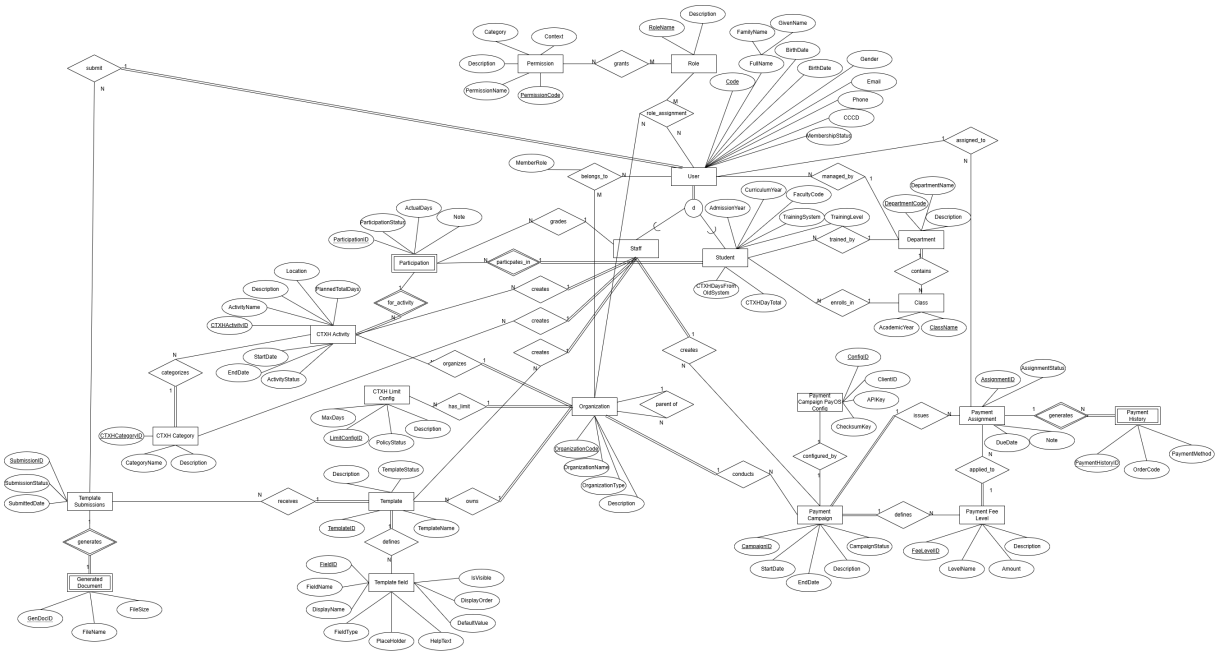


Figure 30: *Entity-Relationship Model cho toàn bộ hệ thống*

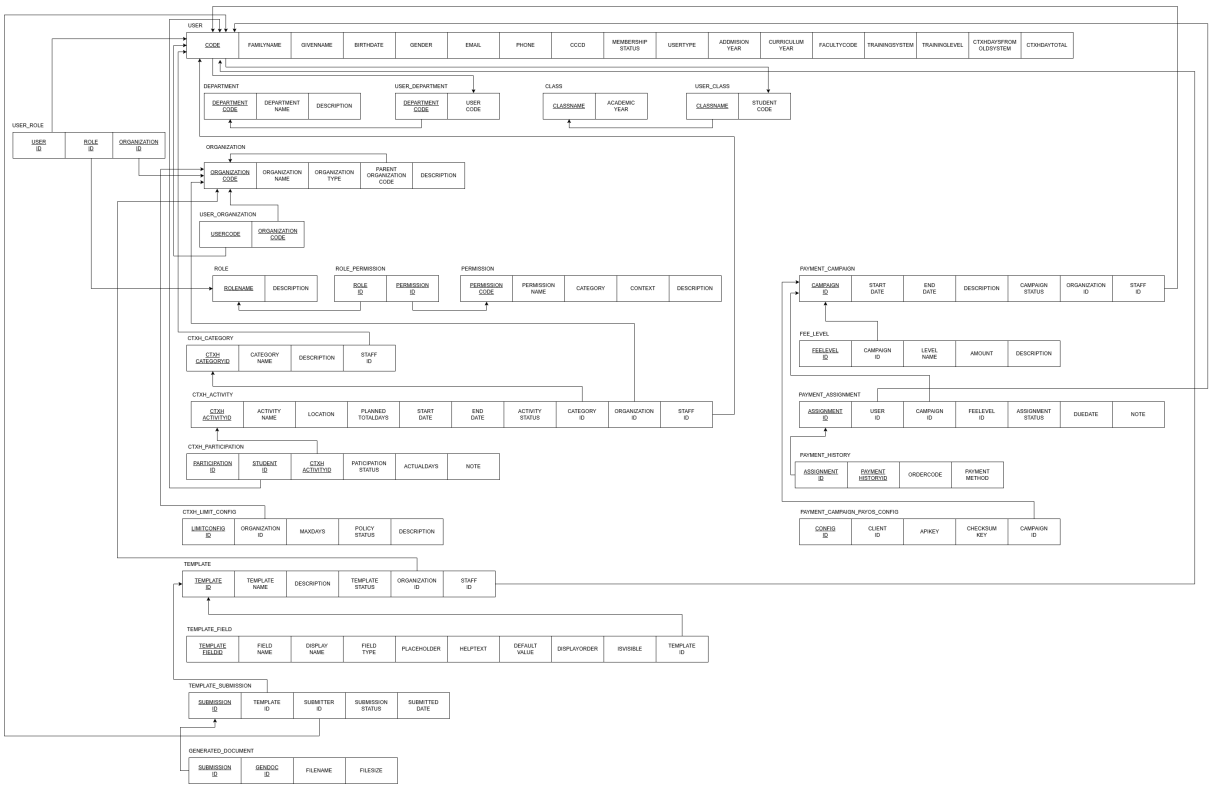


Figure 31: *Relational Data Model của toàn bộ hệ thống*

5.1.2 Dữ liệu phục vụ Chatbot RAG

STT	Thuộc tính	Kiểu dữ liệu	Mô tả
1	id	number	Mã định danh hoạt động CTXH
2	name	string	Tên hoạt động
3	description	string	Mô tả chi tiết hoạt động
4	category.id	number	Mã danh mục hoạt động
5	category.name	string	Tên danh mục hoạt động
6	organization.id	string	Mã đơn vị tổ chức
7	organization.name	string	Tên đơn vị tổ chức
8	startDate	Date	Ngày bắt đầu hoạt động
9	endDate	Date	Ngày kết thúc hoạt động
10	totalDays	number	Tổng số ngày công tác xã hội
11	location	string	Địa điểm tổ chức
12	status	ActivityStatus	Trạng thái hoạt động (ACTIVE, UPCOMING, ...)

Table 19: *Cấu trúc dữ liệu CtxhActivity*

Template văn bản chuẩn cho embedding:

```
Tên hoạt động: [name], Mô tả: [description], Danh mục: [category.name],
Tổ chức: [organization.name], Thời gian: Từ [start_date] đến [end_date],
Ngày CTXH được cấp: [total_days] ngày, Địa điểm: [location]
```

Ví dụ cụ thể:

Tên hoạt động: CHIẾN DỊCH MÙA HÈ XANH CSE 2025, Mô tả: Năm nay, đội hình của khoa sẽ dừng chân tại mảnh đất Đồng Tháp xinh đẹp. Ở đây, chúng mình sẽ dành một tháng sinh hoạt cùng nhau, đồng thời cùng bà con cải thiện chất lượng cuộc sống tại địa phương, Danh mục: Mùa hè xanh, Tổ chức: Đoàn Khoa Khoa học và Kỹ thuật Máy tính, Thời gian: 7/2025, Ngày CTXH được cấp: 12 ngày, Địa điểm: tỉnh Đồng Tháp

5.2 Thiết kế kiến trúc hệ thống

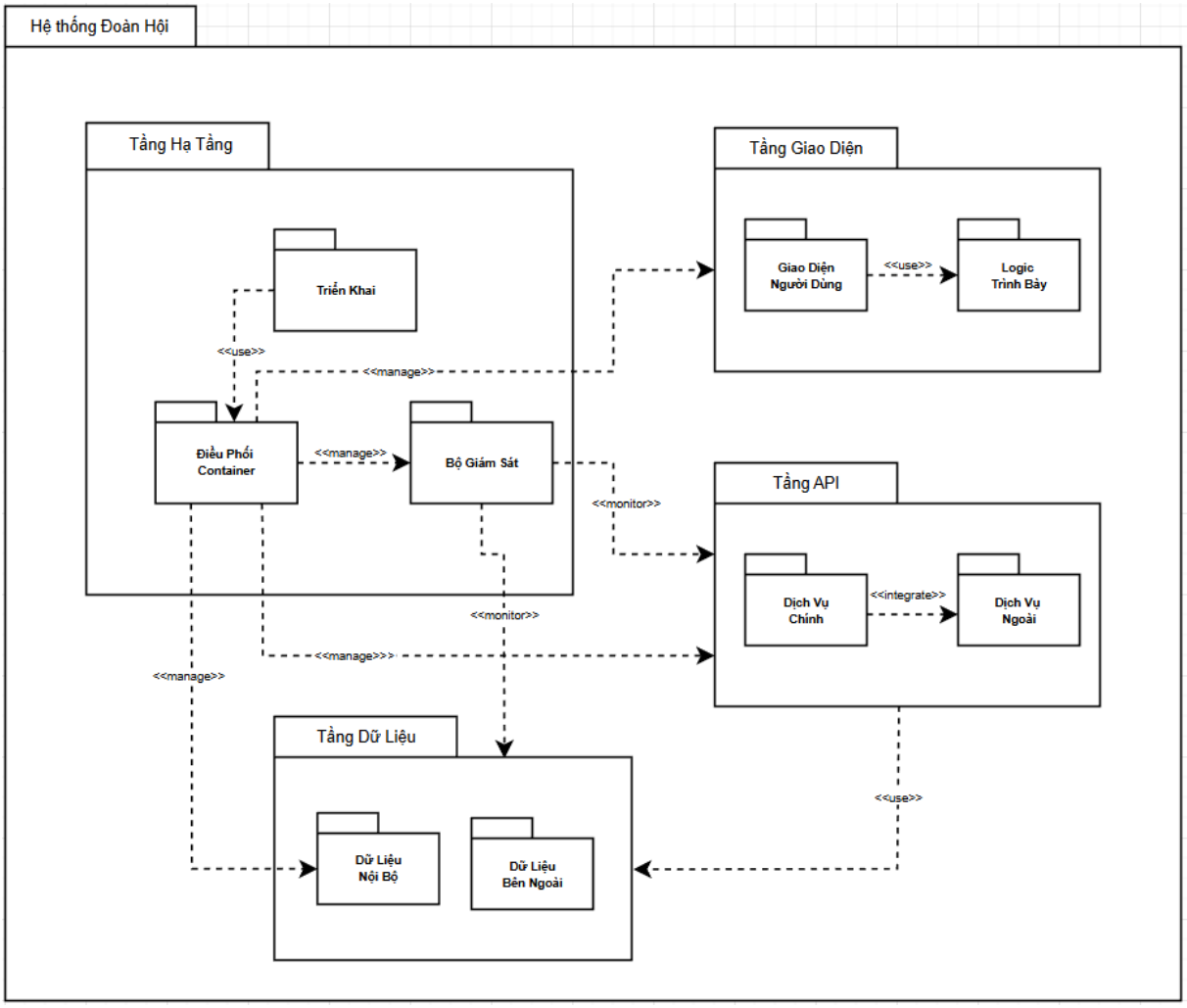


Figure 32: *Component Diagram- Kiến trúc hệ thống*

Để đảm bảo tính mở rộng (Scalability) và khả năng bảo trì (Maintainability) lâu dài, nhóm phát triển quyết định áp dụng mô hình **Layered Application Architecture** (Kiến trúc phân tầng). Hệ thống được chia thành 4 tầng chính với trách nhiệm riêng biệt, giao tiếp chặt chẽ với nhau như mô tả trong Hình 32. Chi tiết các tầng trong kiến trúc hệ thống bao gồm:

- **Tầng Giao Diện (Presentation Layer):** Đây là điểm tiếp xúc trực tiếp với người dùng cuối (Sinh viên và Cán bộ Đoàn - Hội). Tầng này bao gồm:
  - *Giao diện người dùng (User Interface):* Cung cấp trải nghiệm trực quan.
  - *Logic trình bày (Presentation Logic):* Xử lý các thao tác hiển thị và điều hướng trước khi gọi xuống tầng dưới.
- **Tầng API (API Layer):** Đóng vai trò là "bộ não" trung tâm xử lý các yêu cầu nghiệp vụ. Tầng này được chia nhỏ thành:
  - *Dịch vụ chính (Core Services):* Xử lý các logic nghiệp vụ cốt lõi của hệ thống quản lý Đoàn Hội.
  - *Dịch vụ ngoài (External Services):* Module chuyên trách giao tiếp với các hệ thống bên thứ ba (như cổng thanh toán, hệ thống xác thực SSO của nhà trường) thông qua cơ chế tích hợp (*integrate*).
- **Tầng Dữ Liệu (Data Layer):** Chịu trách nhiệm lưu trữ và truy xuất thông tin bền vững, bao gồm:

  - *Dữ liệu nội bộ (Internal Data):* Cơ sở dữ liệu chính của hệ thống.
  - *Dữ liệu bên ngoài (External Data):* Các kho lưu trữ mở rộng dùng cho tài liệu, hình ảnh minh chứng hoạt động.

- **Tầng Hạ Tầng (Infrastructure Layer):** Đây là nền tảng vận hành của toàn bộ hệ thống, được chú trọng đầu tư công nghệ hiện đại:
  - *Container Orchestration (Điều phối Container):* Quản lý vòng đời và sự vận hành của các dịch vụ, giúp hệ thống triển khai linh hoạt.
  - *Observability Suite (Bộ giám sát):* Tích hợp các công cụ giám sát (*monitor*) liên tục để phát hiện lỗi sớm và đảm bảo độ ổn định cao nhất, đặc biệt là trong các đợt cao điểm khi lượng sinh viên đăng ký hoạt động tăng đột biến.

5.3 Thiết kế Chatbot RAG

5.3.1 Tổng quan Chatbot

Hệ thống quản lý Đoàn – Hội tại Trường Đại học Bách khoa TP.HCM hiện đang quản lý hàng trăm hoạt động Công tác xã hội (CTXH) diễn ra mỗi năm, được tổ chức bởi các đơn vị khác nhau như Đoàn trường, các Đoàn Khoa, Liên Chi hội sinh viên. Mỗi hoạt động có nhiều thuộc tính thông tin: tên hoạt động, mô tả chi tiết, danh mục (xuân tình nguyện, mùa hè xanh, hiến máu, …), tổ chức đơn vị, thời gian tổ chức, số ngày CTXH được công nhận, địa điểm, và trạng thái hoạt động.

Sinh viên thường gặp khó khăn khi:

- Tìm kiếm hoạt động CTXH phù hợp với sở thích và thời gian của bản thân
- Tra cứu thông tin chi tiết về các hoạt động đang diễn ra
- Lọc hoạt động theo tiêu chí cụ thể (địa điểm, đơn vị tổ chức, loại hình hoạt động)
- Nhận được tư vấn nhanh chóng về các hoạt động mà không cần liên hệ trực tiếp với ban tổ chức.

Phương pháp tìm kiếm truyền thống dựa trên từ khóa (keyword search) không đủ linh hoạt để xử lý các câu hỏi dạng ngôn ngữ tự nhiên như ”Có hoạt động nào đang diễn ra ở cơ sở 1 không?” hay ”Hoạt động hiến máu tình nguyện được bao nhiêu ngày CTXH”.

Để giải quyết vấn đề trên, module chatbot hỏi–đáp CTXH được phát triển với các mục tiêu:

- Hỗ trợ sinh viên tra cứu thông tin hoạt động CTXH bằng ngôn ngữ tự nhiên
- Tìm kiếm ngữ nghĩa (semantic search) các hoạt động phù hợp với ý định câu hỏi
- Cung cấp câu trả lời chi tiết kèm nguồn thông tin và gợi ý liên quan
- Duy trì lịch sử hội thoại để hỗ trợ các câu hỏi nhiều vòng (multi-turn conversation)
- Đảm bảo câu trả lời chỉ dựa trên dữ liệu thực tế của hệ thống, tránh tạo ra thông tin sai lệch

Retrieval Augmented Generation (RAG) được lựa chọn vì các lý do sau:

- **Tính chính xác:** RAG đảm bảo câu trả lời luôn dựa trên dữ liệu thực tế từ cơ sở dữ liệu hoạt động CTXH, giảm thiểu hiện tượng ”hallucination” (tạo ra thông tin không có thật) thường gặp khi sử dụng LLM thuần túy.
- **Tính cập nhật:** Khi dữ liệu hoạt động CTXH thay đổi (thêm mới, cập nhật), hệ thống tự động cập nhật vector store mà không cần huấn luyện lại mô hình ngôn ngữ.
- **Khả năng kiểm soát:** Mỗi câu trả lời đều có thể truy vết nguồn gốc (source attribution), giúp người dùng và quản trị viên xác minh tính chính xác.
- **Hiệu quả về chi phí:** Không cần fine-tune mô hình LLM lớn, chỉ cần sử dụng embedding và generation API.

5.3.2 Tổng quan giải pháp Chatbot RAG

RAG là kiến trúc kết hợp hai bước chính:

- **Retrieval (Truy xuất):** Tìm kiếm các tài liệu/thông tin liên quan nhất đến câu hỏi từ vector store
- **Generation (Sinh câu trả lời):** Sử dụng Large Language Model (LLM) để tạo câu trả lời dựa trên thông tin đã truy xuất

Pipeline RAG đảm bảo mô hình ngôn ngữ không tự tạo ra thông tin mà phải dựa vào context cụ thể được cung cấp từ cơ sở dữ liệu. Điều này là một cải tiến lớn so với LLM thông thường vì dữ liệu CTXH trả lời cho người dùng cần phải chính xác, không được tự ý bịa đặt. Nhóm đã lập bảng so sánh giữa 2 mô hình AI như sau:

Tiêu chí	**LLM thuần túy**	**RAG (Retrieval-Augmented Generation)**
Nguồn thông tin	Dữ liệu huấn luyện sẵn có, mang tính chung chung và có thể đã lỗi thời	Cơ sở dữ liệu nội bộ hoặc nguồn dữ liệu thời gian thực được truy xuất khi cần
Độ chính xác	Dễ xảy ra hiện tượng hallucination (tạo thông tin không có thật)	Độ chính xác cao do câu trả lời dựa trên dữ liệu thực tế được truy xuất
Khả năng cập nhật	Cần huấn luyện lại mô hình hoặc fine-tuning khi dữ liệu thay đổi	Dữ liệu được cập nhật linh hoạt thông qua vector store mà không cần huấn luyện lại
Khả năng truy vết nguồn	Không thể truy vết nguồn thông tin cụ thể của câu trả lời	Có thể truy vết rõ ràng nguồn dữ liệu được sử dụng để tạo câu trả lời
Chi phí	Chi phí cao do yêu cầu tài nguyên lớn cho quá trình huấn luyện hoặc fine-tuning	Chi phí thấp hơn, chủ yếu sử dụng API của mô hình ngôn ngữ và hệ thống truy xuất

Table 20: *So sánh giữa LLM thuần túy và RAG*

5.3.3 Quy trình tự động chuyển đổi dữ liệu sang vector embedding

Hệ thống sử dụng **TypeORM Entity Subscriber** để tự động theo dõi và đồng bộ embeddings khi có thay đổi dữ liệu. **Hình 28** minh họa chi tiết quy trình tự động chuyển đổi dữ liệu CTXH sang dạng vector embedding. Trigger tự động embedding như sau:

1. **Activity mới được tạo (INSERT):**
   - Điều kiện: `status = 'ACTIVE'`
   - Hành động: Tự động tạo embedding và lưu vào vector store
2. **Activity được cập nhật (UPDATE):**
   - Điều kiện: Có thay đổi ở các trường: `name`, `description`, `start_date`, `end_date`, `location`, hoặc `total_days`
   - Hành động: Tạo lại embedding mới và cập nhật vector store
3. **Activity bị xóa (DELETE):**
   - Hành động: Xóa embedding tương ứng khỏi vector store

Hiệu suất AI Processing:

- Thời gian tạo embedding: ∼200-300ms/activity (Gemini API)
- Vector dimension: 768 chiều (text-embedding-004)
- Batch processing: Có thể xử lý 100 activities/batch khi khởi tạo hệ thống

5.3.4 Quy trình xử lý câu hỏi của người dùng với AI

Luồng xử lý câu hỏi của Chatbot RAG được mô tả chi tiết trong **hình 29**. Quy trình bao gồm các bước sau:
Bước 1: Quản lý Session

- Input: `query`, `userId`, `sessionId` (optional)
- Nếu có `sessionId`: Lấy session từ Redis
- Nếu không có: Tạo session mới với UUID
- Session chứa: userId, messages[], summary, metadata

Bước 2: Tạo Query Embedding bằng AI

- Input: Câu hỏi người dùng (text)
- AI Processing:
  1. Gọi Google Gemini API `embedContent()`
  2. Model: `text-embedding-004` chuyển văn bản thành vector 768 chiều
  3. AI hiểu ngữ nghĩa câu hỏi, không phụ thuộc từ khóa
- Output: Query vector[768] đại diện cho ý nghĩa câu hỏi

Bước 3: Vector Similarity Search với AI

- Input: Query vector từ bước 2
- Process:
  1. Query PostgreSQL với pgVector (similarity > 0.7)
  2. Lambda = 0.7 (cân bằng độ liên quan vs độ đa dạng)
  3. Chọn 5 documents tốt nhất
- Output: Top-5 hoạt động đa dạng, phù hợp nhất

Bước 4: Lấy Context Hội thoại

- Lấy lịch sử tin nhắn từ session
- Nếu hội thoại quá dài (> 20 messages):
  1. Gọi AI Gemini để tóm tắt lịch sử
  2. Giữ lại 10 messages gần nhất + summary
- Output: Lịch sử hội thoại được tối ưu

Bước 5: AI Generate Câu trả lời (Core RAG)

- Xây dựng prompt cho AI
- Gọi **Google Gemini 2.5 Pro API** `generateContent()`
- AI xử lý:
  - Phân tích câu hỏi và context
  - Tổng hợp thông tin từ 5 activities
  - Tạo câu trả lời tự nhiên, dễ hiểu
  - Trả lời đúng ngữ cảnh hội thoại nhiều vòng
- Output: Câu trả lời AI sinh ra

5.3.5 Kết quả đạt được

a. Các chức năng chatbot đã thực hiện

- **Hỏi–đáp ngôn ngữ tự nhiên:** Sinh viên có thể đặt câu hỏi tự do như "Có hoạt động nào dành cho sinh viên khoa học máy tính không?", "Tôi muốn tham gia hoạt động về môi trường".
- **Tìm kiếm ngữ nghĩa:** Hệ thống hiểu được ý định câu hỏi mà không yêu cầu từ khóa chính xác.
- **Lọc theo tiêu chí:** Hỗ trợ lọc theo đơn vị tổ chức, trạng thái hoạt động và khoảng thời gian.
- **Source attribution:** Mỗi câu trả lời đều kèm danh sách hoạt động được sử dụng làm nguồn, bao gồm liên kết chi tiết và *confidence score*.
- **Follow-up suggestions:** Đề xuất tối đa 3 hành động tiếp theo như đăng ký tham gia hoặc xem các hoạt động liên quan.

b. Kết quả kiểm thử API backend

Unit Tests: 59/59 test cases đạt (100%).

Service	Tests	Status	Coverage
CtxhRetrievalService	12/12	PASS	Vector search, MMR, filters
ConversationService	14/14	PASS	Session CRUD, summarization
CtxhChatService	16/16	PASS	RAG pipeline, error handling
ResponseFormatterService	17/17	PASS	Markdown, sources, suggestions

Các API endpoint đã kiểm thử:

- `POST /api/ctxh-chat/message`: Gửi câu hỏi và nhận câu trả lời.
- `GET /api/ctxh-chat/history/:sessionId`: Lấy lịch sử hội thoại.
- `DELETE /api/ctxh-chat/session/:sessionId`: Xóa phiên làm việc.

c. Khả năng trả lời câu hỏi Công tác xã hội (CTXH)

- **Theo chủ đề:** "Có hoạt động nào về môi trường không?", "Tôi muốn tham gia hoạt động hiến máu".
- **Theo địa điểm:** "Hoạt động nào diễn ra ở cơ sở 1?", "Có hoạt động nào ở quận 10 không?".
- **Theo thời gian:** "Hoạt động nào trong tháng này?", "Hoạt động sắp diễn ra".
- **Theo đơn vị tổ chức:** "Đoàn khoa Khoa học Máy tính có hoạt động gì?".
- **Tìm kiếm kết hợp:** "Có hoạt động về cộng đồng ở cơ sở 2 trong tháng 12 không?", "Tôi muốn tham gia hoạt động 2 ngày CTXH về môi trường".

d. Đánh giá độ chính xác và mức độ phù hợp

Độ chính xác về nội dung:

- Chatbot chỉ trả lời dựa trên dữ liệu thực tế từ cơ sở dữ liệu.
- Không ghi nhận hiện tượng sinh thông tin sai lệch (hallucination) trong các kịch bản kiểm thử.
- Cơ chế trích dẫn dữ liệu nguồn cho phép xác minh đầy đủ nguồn thông tin.

Độ phù hợp của kết quả:

- Ngưỡng tương đồng (similarity threshold) 0.7 giúp lọc các kết quả có chất lượng.
- Trong Top-5 kết quả trả về, thường có từ 3 đến 4 hoạt động thực sự phù hợp với câu hỏi.

5.3.6 Giới hạn của Chatbot

Theo thiết kế hiện tại, Chatbot không trả lời được các câu hỏi sau:

- **Thông tin về deadline đăng ký**
  Lý do: Thông tin deadline thường thay đổi linh hoạt và phụ thuộc vào việc quản trị viên có thêm dữ liệu về deadline đăng ký trong mục mô tả khi tạo hoạt động CTXH hay không
- **Thông tin về quy trình cập nhật ngày CTXH**
  Lý do: Thông tin về quy trình quản lý nội bộ không phải dữ liệu CTXH
- **Câu hỏi ngoài phạm vi CTXH**
  Lý do: Chatbot RAG chỉ thiết kế cho việc truy vấn thông tin hoạt động CTXH nên không trả lời câu hỏi nằm ngoài phạm vi dữ liệu hoạt động CTXH

5.3.7 Kết luận

1. **Về mặt kỹ thuật:**
   - Áp dụng kiến trúc RAG hiện đại với vector search và LLM
   - Tích hợp đầy đủ: PostgreSQL pgVector, Google Gemini API
   - Đảm bảo chất lượng code với 100% unit test coverage
2. **Về mặt chức năng:**
   - Giải quyết bài toán tra cứu thông tin CTXH bằng ngôn ngữ tự nhiên
   - Tìm kiếm ngữ nghĩa vượt trội so với keyword search truyền thống
   - Cung cấp dữ liệu CTXH chi tiết khi trả lời để đảm bảo tính minh bạch

Chatbot RAG còn là một thành phần quan trọng trong Cổng thông tin Đoàn – Hội:

1. **Tăng cường trải nghiệm người dùng:**
   - Sinh viên không cần duyệt qua hàng trăm hoạt động
   - Tra cứu nhanh chóng bằng ngôn ngữ tự nhiên
   - Nhận gợi ý hoạt động phù hợp
2. **Giảm tải công việc quản lý:**
   - Tự động trả lời câu hỏi thường gặp
   - Giảm số lượng email/tin nhắn gửi cho quản trị viên để hỏi thông tin
   - Ban tổ chức tập trung vào công việc quan trọng hơn
3. **Nền tảng cho tương lai:**
   - Có thể mở rộng sang các lĩnh vực khác
   - Tích lũy dữ liệu hành vi tra cứu của người dùng để cải thiện
   - Nền tảng cho các tính năng AI tiếp theo

5.3.8 Định hướng phát triển Chatbot

- Tích hợp vào giao diện người dùng
- Triển khai hệ thống CTXH để có thêm dữ liệu cho Chatbot
- Thiết lập kịch bản kiểm thử

6 Tổng kết

6.1 Nhận xét

Sau quá trình phát triển và triển khai hệ thống Đoàn Hội Portal tại Trường Đại học Bách Khoa TP.HCM, nhóm đã đạt được những kết quả đáng ghi nhận và rút ra một số nhận xét về ưu nhược điểm của hệ thống.

6.1.1 Ưu điểm

- **Số hóa toàn diện quy trình quản lý:**
  - Hệ thống đã thay thế thành công các quy trình thủ công bằng giải pháp số hóa: từ quản lý hồ sơ Đoàn viên/Hội viên, đăng ký và chấm điểm hoạt động CTXH, đến thu hội phí trực tuyến.
  - Giảm thiểu đáng kể khối lượng công việc giấy tờ, nâng cao hiệu quả làm việc cho cán bộ Đoàn – Hội.
  - Sinh viên có thể tra cứu thông tin cá nhân, điểm CTXH, và thanh toán hội phí mọi lúc mọi nơi thông qua giao diện web responsive.
- **Kiến trúc hệ thống hiện đại và khoa học:**
  - Áp dụng kiến trúc modular với backend NestJS và frontend ReactJS, tách biệt rõ ràng giữa presentation layer và business logic.
  - Sử dụng TypeORM giúp xây dựng query an toàn, dễ dàng bảo trì và mở rộng schema database.
  - Triển khai Docker container hóa toàn bộ stack, đảm bảo tính nhất quán giữa các môi trường development, staging và production.
  - Tích hợp Prometheus và Grafana cho phép giám sát real-time các metrics quan trọng của hệ thống.
- **Hệ thống phân quyền linh hoạt:**
  - Mô hình RBAC với context theo tổ chức cho phép quản lý quyền truy cập chi tiết và phù hợp với cơ cấu tổ chức thực tế.
  - Một user có thể có nhiều vai trò khác nhau ở các tổ chức khác nhau, phản ánh đúng thực tế hoạt động Đoàn – Hội.
  - PermissionGuard tự động kiểm tra quyền trước khi thực thi API, giảm thiểu lỗi bảo mật.
- **Tính năng import/export Excel linh hoạt:**
  - Hỗ trợ import hàng loạt user với 18 cột thông tin, tự động tạo tổ chức/khoa/lớp nếu chưa tồn tại.
  - Upload Excel để chấm điểm CTXH hàng loạt với validation chặt chẽ, giảm thời gian xử lý từ hàng giờ xuống còn vài phút.
  - Parse file theo chunk giúp xử lý được file lớn (hàng nghìn dòng) mà không bị timeout.
- **Tích hợp thanh toán trực tuyến:**
  - PayOS integration cho phép thu hội phí tự động, giảm gánh nặng quản lý tiền mặt.
  - Webhook tự động cập nhật trạng thái thanh toán, đảm bảo dữ liệu đồng bộ real-time.
  - Lưu trữ đầy đủ lịch sử giao dịch phục vụ đối soát và audit.
- **Hệ thống biểu mẫu động:**
  - Docxtemplater cho phép tạo template Word với placeholder, tự động generate document từ dữ liệu form.
  - Admin có thể tạo và cấu hình biểu mẫu mới mà không cần developer can thiệp.
  - Upload file output lên S3, tách biệt storage khỏi database server.

6.1.2 Hạn chế

- **Hiệu năng khi xử lý dữ liệu lớn:**
  - Import Excel file có hơn 5000 dòng có thể mất thời gian xử lý lâu (30-60 giây) do thực hiện đồng bộ, gây trải nghiệm chờ đợi cho người dùng.
  - Generate document từ template với nhiều field (> 50 placeholder) có thể mất 10-20 giây, chưa có progress indicator rõ ràng.
  - Query danh sách hoạt động khi có nhiều filter và sort có thể chậm (2-3 giây) do chưa optimize index đầy đủ.
- **Giao diện người dùng:**
  - Responsive design trên mobile browser còn chưa mượt mà ở một số màn hình phức tạp (form tạo hoạt động, dashboard).
  - Một số thông báo lỗi validation chưa đủ rõ ràng, người dùng mới có thể gặp khó khăn khi điền form.
  - Dark mode chưa được tối ưu hoàn toàn, một số component còn hiển thị màu không đồng nhất.
- **Kiểm thử và đảm bảo chất lượng:**
  - Test coverage hiện tại còn thấp (backend < 30%, frontend < 20%), chủ yếu dựa vào manual testing.
  - Chưa có automated testing trong quy trình deployment, tăng rủi ro khi release version mới.
  - Chưa thực hiện performance testing và load testing để xác định giới hạn của hệ thống.

6.2 Kế hoạch phát triển

Dựa trên nền tảng đã xây dựng, kế hoạch phát triển tiếp theo của nhóm trong giai đoạn sau tập trung vào việc hoàn thiện, tối ưu hóa và nâng cao chất lượng hệ thống hiện tại, thay vì phát triển các tính năng mới phức tạp.

6.2.1 Hoàn thiện và tối ưu các tính năng hiện có (Tuần 1-6)

- **Cải thiện hiệu năng:**
  - Triển khai Redis để cache các query lặp lại thường xuyên (danh sách tổ chức, thông tin user, danh sách hoạt động).
  - Implement Bull Queue để xử lý bất đồng bộ các tác vụ nặng: import Excel, generate document, gửi email hàng loạt.
  - Phân tích và tối ưu các slow query, thêm index phù hợp cho các bảng lớn.
- **Nâng cao tính bảo mật:**
  - Triển khai rate limiting cho các endpoint quan trọng (login, upload file, payment) để chống tấn công brute-force.
  - Thêm 2FA (Two-Factor Authentication) cho tài khoản admin bằng Google Authenticator.
  - Cải thiện audit logging: ghi nhận đầy đủ các thao tác quan trọng với thông tin user, IP, timestamp.
- **Cải thiện trải nghiệm người dùng:**
  - Tối ưu responsive design cho các màn hình tablet và mobile.
  - Cải thiện các thông báo lỗi, loading states, validation messages để user-friendly hơn.
  - Thêm hướng dẫn sử dụng (tooltips, help text) cho các tính năng phức tạp.
  - Tối ưu performance frontend: lazy loading, code splitting, giảm bundle size.

6.2.2 Triển khai Testing toàn diện (Tuần 7-10)

- **Backend Testing:**
  - Unit tests cho các service và controller quan trọng (auth, users, activities, payment).
  - Integration tests cho các API endpoint với database test.
  - Mục tiêu: đạt test coverage ít nhất 60% cho backend code.
- **Frontend Testing:**
  - Component tests cho các UI component chính bằng React Testing Library.
  - Integration tests cho các user flows quan trọng (login, đăng ký hoạt động, thanh toán).
  - Mục tiêu: test coverage 50% cho các component và pages quan trọng.
- **End-to-End Testing:**
  - Viết E2E tests cho các user journey quan trọng bằng Playwright hoặc Cypress.
  - Test cases: đăng nhập, quản lý hoạt động, import Excel, thanh toán, export báo cáo.

6.2.3 Cải thiện DevOps và Deployment (Tuần 11-14)

- **CI/CD Pipeline:**
  - Setup GitHub Actions hoặc GitLab CI để tự động chạy tests khi có pull request.
  - Automated build và push Docker images lên registry khi merge vào branch main.
  - Automated deployment lên staging environment để test trước khi production.
- **Quản lý thông tin bí mật:**
  - Triển khai HashiCorp Vault để quản lý secrets (database credentials, API keys, JWT secrets).
  - Loại bỏ hardcoded secrets trong codebase và environment files.
  - Implement dynamic secrets rotation cho database credentials.
- **Monitoring, Alerting và Logging:**
  - Cấu hình Alertmanager tích hợp với Prometheus để gửi cảnh báo qua email/Telegram khi có sự cố.
  - Setup centralized logging với ELK Stack (Elasticsearch, Logstash, Kibana) hoặc Loki để tổng hợp log từ các container.
  - Tạo runbook cho các incident scenarios phổ biến (high CPU, database connection timeout, API errors).
- **Backup và Disaster Recovery:**
  - Automated daily backup cho PostgreSQL database với retention policy 30 ngày.
  - S3 versioning và lifecycle policy để backup file uploads, tự động archive sang Glacier sau 90 ngày.
  - Viết và test disaster recovery procedure, đo lường RTO (Recovery Time Objective) và RPO (Recovery Point Objective).
- **Security Scanning:**
  - Tích hợp Trivy vào CI/CD pipeline để scan Docker images và dependencies trước khi deploy.
  - Sử dụng OWASP Dependency Check để phát hiện lỗ hổng bảo mật trong các thư viện npm/Python.
  - Thiết lập policy fail build nếu phát hiện vulnerabilities ở mức CRITICAL hoặc HIGH.

6.2.4 Documentation và Knowledge Transfer (Tuần 15-16)

- **Technical Documentation:**
  - Cập nhật README với hướng dẫn setup đầy đủ cho developer mới.
  - Viết API documentation đầy đủ trên Swagger/OpenAPI.
  - Document các design decisions, architecture choices, trade-offs đã thực hiện.
- **User Documentation:**
  - Viết user manual cho admin: hướng dẫn quản lý user, tạo hoạt động, import Excel, tạo biểu mẫu.
  - Viết hướng dẫn cho sinh viên: đăng ký hoạt động, thanh toán hội phí, xem điểm CTXH.
  - Tạo video tutorials cho các tính năng chính.
- **Deployment Guide:**
  - Document chi tiết quy trình deployment production.
  - Hướng dẫn troubleshooting cho các vấn đề thường gặp.
  - Maintenance checklist cho việc vận hành hệ thống.

6.2.5 Các mục tiêu đo lường được

- Backend test coverage: $\geq 60\%$
- Frontend test coverage: $\geq 50\%$
- E2E test scenarios: ≥ 10 user journeys quan trọng
- CI/CD pipeline: 100% automated testing và deployment
- Response time với Redis: giảm $\geq 50\%$ so với hiện tại
- Downtime khi deployment: < 5 phút
- Documentation completeness: 100% các module chính được document

7 Tài liệu tham khảo

1. NestJS (2025). *NestJS - A progressive Node.js framework*. `https://nestjs.com/` (Truy cập: 15/09/2025)
2. Meta Platforms, Inc. (2025). *React - A JavaScript library for building user interfaces*. `https://react.dev/` (Truy cập: 22/09/2025)
3. SheetJS LLC (2025). *SheetJS Community Edition - Spreadsheet Data Toolkit*. `https://sheetjs.com/` (Truy cập: 08/10/2025)
4. Edgar Hipp (2025). *Docxtemplater Documentation*. `https://docxtemplater.com/` (Truy cập: 25/10/2025)
5. Open XML Templating (2025). *PizZip - A library for handling zip files*. `https://github.com/open-xml-templating/pizzip` (Truy cập: 03/11/2025)
6. Shailja Gupta, Rajesh Ranjan, Surya Narayan Singh (2024). *A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions*

7. Avthar Sewrathan (2023). *How to Build LLM Applications With Pgvector Vector Store in LangChain* https://www.tigerdata.com/blog/how-to-build-llm-applications-with-pgvector-vector-store- (Truy cập: 20/10/2025)

8. langchainjs (2025). *LangChain.js GitHub Repository* https://github.com/langchain-ai/langchainjs (Truy cập: 20/10/2025)

9. PGVectorStore (2025). *LangChain pgVector documentation* https://docs.langchain.com/oss/python/integrations/vectorstores/pgvectorstore (Truy cập 20/10/2025)